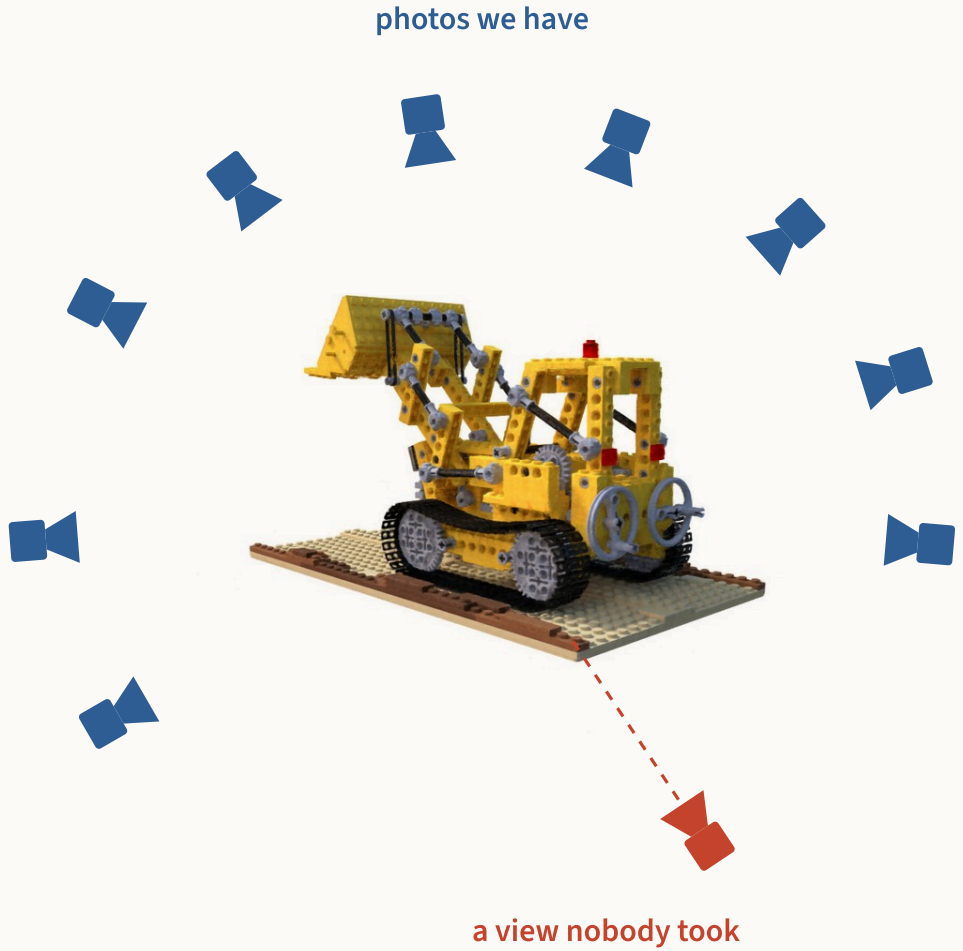
Neural Radiance Fields

Fit one small network to the photos of one scene, then render that scene from viewpoints no camera took. Then two papers that made it fast.



Ben Mildenhall, Pratul P. Srinivasan, Matthew Tancik, Jonathan T. Barron, Ravi Ramamoorthi, Ren Ng
UC Berkeley, Google Research and UC San Diego. ECCV 2020.
With Instant-NGP (Müller et al., SIGGRAPH 2022) and 3D Gaussian splatting (Kerbl et al., SIGGRAPH 2023).

Presented by **Ertugrul Senturk**
November 2026

The task: render a scene from a viewpoint nobody photographed

100 photos of one scene, with each camera's position and direction known

NeRF

one network, trained on these photos only

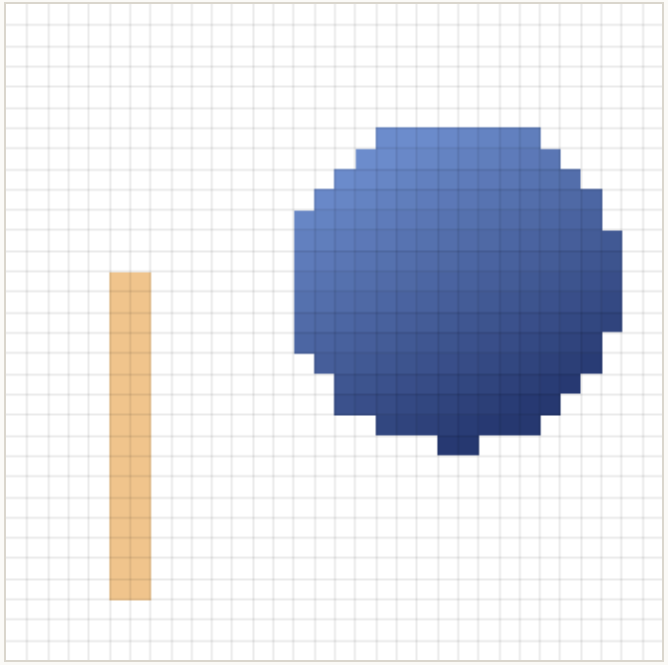
Two new views rendered by NeRF

The input is a set of ordinary photos of one scene, plus where each camera stood and where it pointed. The output is the photo a camera would take from any other position, with correct perspective, hidden parts and reflections.

This problem is called view synthesis. NeRF solves it by training one network per scene, from scratch, on that scene's photos alone. After training, the network is the scene.

Mildenhall et al. 2020, Fig. 1: the synthetic Drums scene, 100 input views on a hemisphere, two rendered new views.

Storing a scene: a grid of cubes or a function



A slice through a toy scene, drawn with the grid chosen below.

Shown for **32 cubes per side**. The next pages show 128 and 1,024.

Earlier methods store a scene as a 3D grid of small cubes, called voxels, each with a colour and an opacity. Fine detail needs small cubes, and halving their size multiplies their number by eight.

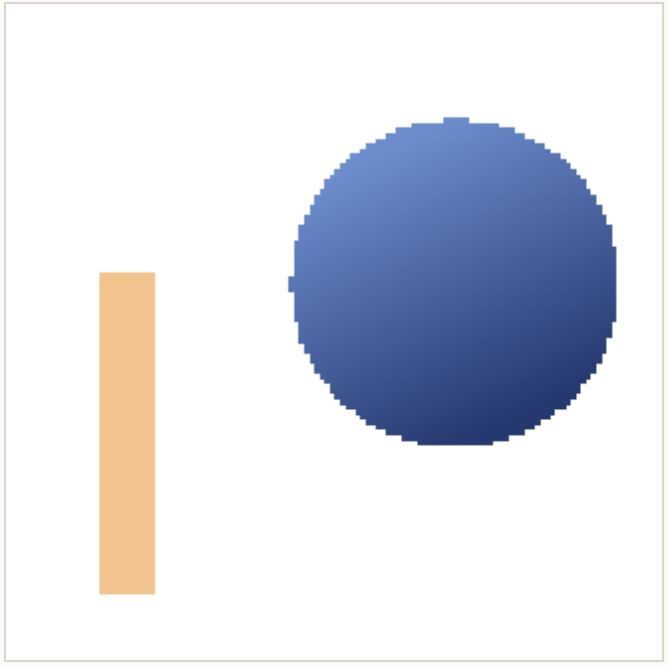
NeRF stores a function instead: a network that gives the colour and density at any point you ask about. Its size does not depend on a resolution, and for a whole scene it is about 5 MB.

5 MB
NeRF

131 KB
grid, 32^3 cubes

Grid memory: cubes per side cubed, 4 bytes each (red, green, blue, opacity). Squares are sized by area. Neural Volumes, a baseline in the paper, uses a 128^3 grid; NeRF's 5 MB is from Mildenhall et al. 2020, Sec. 6.3.

Storing a scene: a grid of cubes or a function



A slice through a toy scene, drawn with the grid chosen below.

Shown for **128 cubes per side**. Neural Volumes uses this grid.

Earlier methods store a scene as a 3D grid of small cubes, called voxels, each with a colour and an opacity. Fine detail needs small cubes, and halving their size multiplies their number by eight.

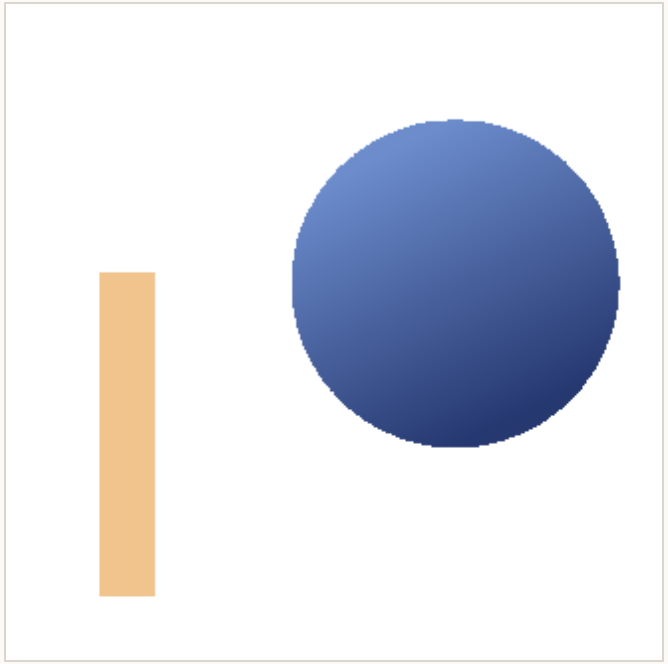
NeRF stores a function instead: a network that gives the colour and density at any point you ask about. Its size does not depend on a resolution, and for a whole scene it is about 5 MB.

5 MB
NeRF

8.4 MB
grid, 128^3 cubes

Grid memory: cubes per side cubed, 4 bytes each (red, green, blue, opacity). Squares are sized by area. Neural Volumes, a baseline in the paper, uses a 128^3 grid; NeRF's 5 MB is from Mildenhall et al. 2020, Sec. 6.3.

Storing a scene: a grid of cubes or a function

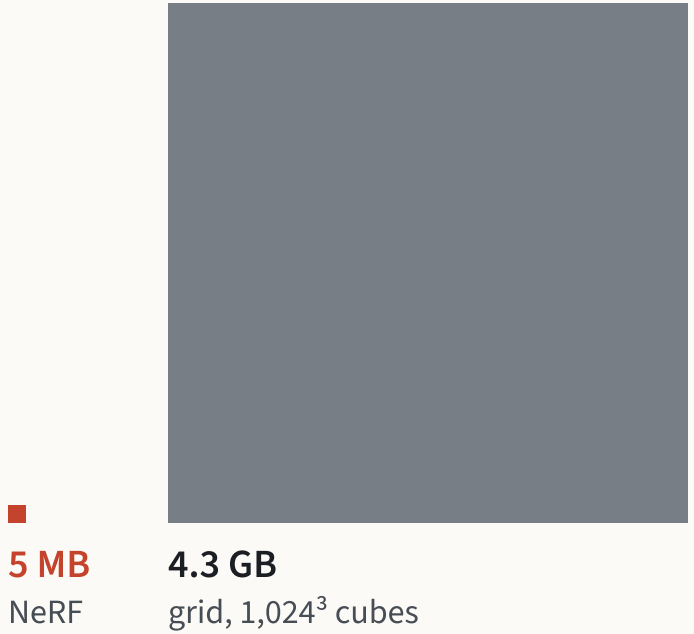


A slice through a toy scene, drawn with the grid chosen below.

Shown for **1,024 cubes per side**.

Earlier methods store a scene as a 3D grid of small cubes, called voxels, each with a colour and an opacity. Fine detail needs small cubes, and halving their size multiplies their number by eight.

NeRF stores a function instead: a network that gives the colour and density at any point you ask about. Its size does not depend on a resolution, and for a whole scene it is about 5 MB.

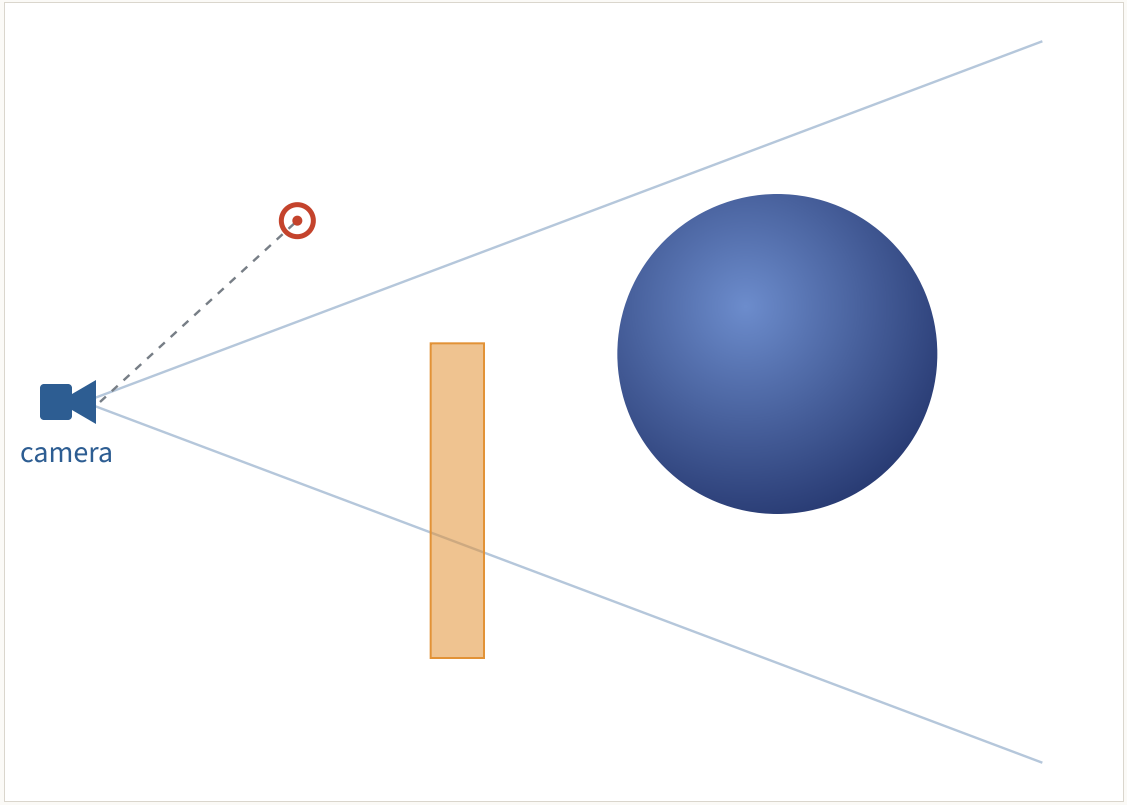


5 MB
NeRF

4.3 GB
grid, $1{,}024^3$ cubes

Grid memory: cubes per side cubed, 4 bytes each (red, green, blue, opacity). Squares are sized by area. Neural Volumes, a baseline in the paper, uses a 128^3 grid; NeRF's 5 MB is from Mildenhall et al. 2020, Sec. 6.3.

What the network takes in and gives back



position

$(2.0, 1.7, 0)$

view direction

(θ, ϕ)

F_Θ

density

$\sigma = 0$

colour

anything

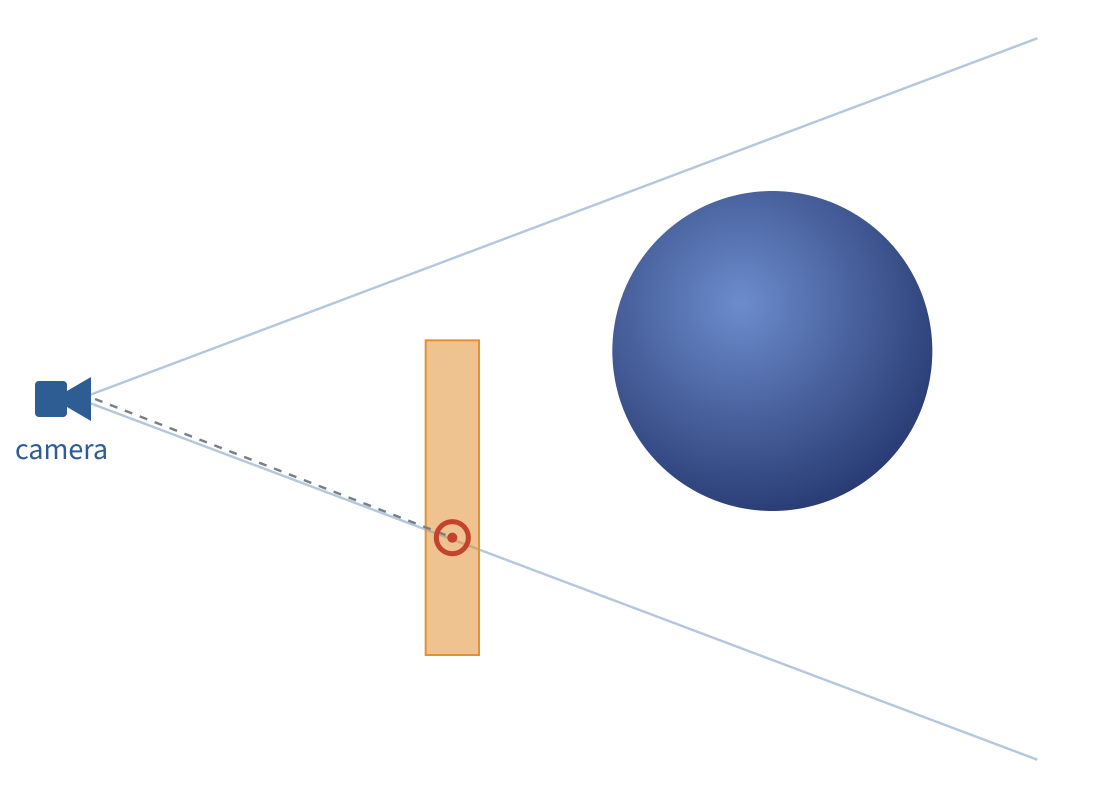
A point in empty air: density 0, so its colour never matters.

Density says how much matter is at the point: zero in empty air, small in something see-through, large in a solid. It does not depend on where you look from.

Colour is the light the point sends towards the camera, so it can change with the viewing direction.

A toy scene in two dimensions, with made-up density values; NeRF does the same in 3D. Mildenhall et al. 2020, Sec. 3: $F_\Theta : (x, d) \rightarrow (c, \sigma)$.

What the network takes in and gives back



position
(3.5, -1.3, 0)

view direction
(θ, ϕ)

F_Θ

density
σ = 1.6

colour
(226, 146, 54)

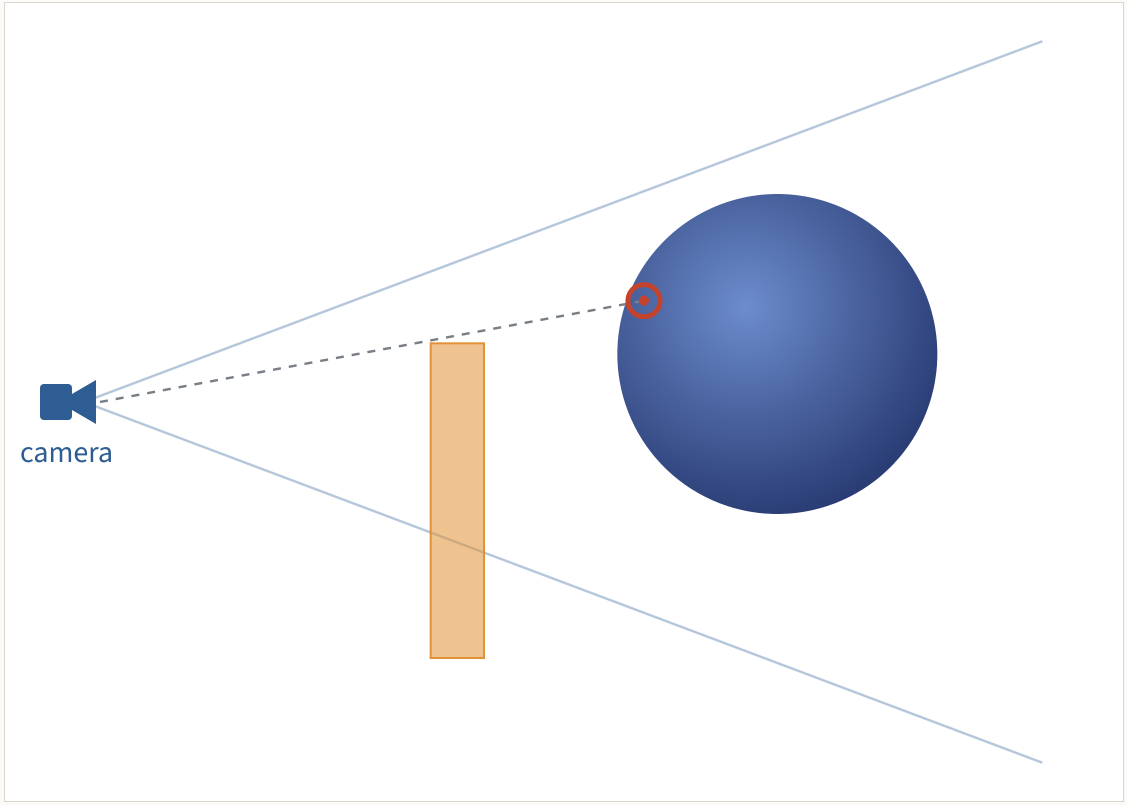
A point inside the orange pane: a small density, so light is dimmed but passes through.

Density says how much matter is at the point: zero in empty air, small in something see-through, large in a solid. It does not depend on where you look from.

Colour is the light the point sends towards the camera, so it can change with the viewing direction.

A toy scene in two dimensions, with made-up density values; NeRF does the same in 3D. Mildenhall et al. 2020, Sec. 3: $F_\Theta : (x, d) \to (c, \sigma)$.

What the network takes in and gives back



position
$(5.3, 0.9, 0)$

view direction
(θ, ϕ)

F_Θ

density
$\sigma = 18$

colour
$(99, 130, 193)$

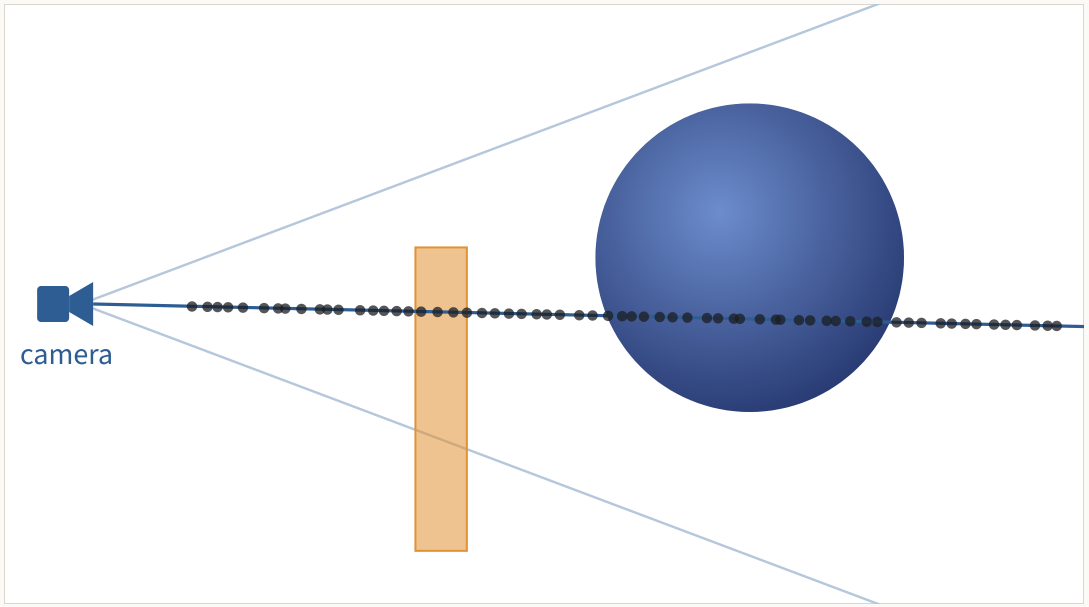
A point on the blue ball: a high density, so nothing behind it can be seen.

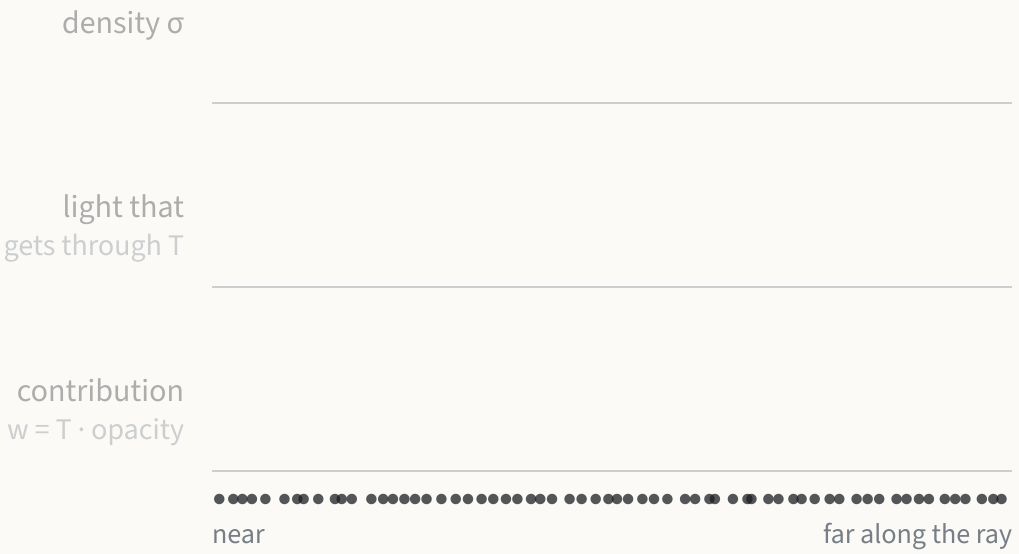
Density says how much matter is at the point: zero in empty air, small in something see-through, large in a solid. It does not depend on where you look from.

Colour is the light the point sends towards the camera, so it can change with the viewing direction.

A toy scene in two dimensions, with made-up density values; NeRF does the same in 3D. Mildenhall et al. 2020, Sec. 3: $F_\Theta : (x, d) \rightarrow (c, \sigma)$.

From densities and colours to one pixel



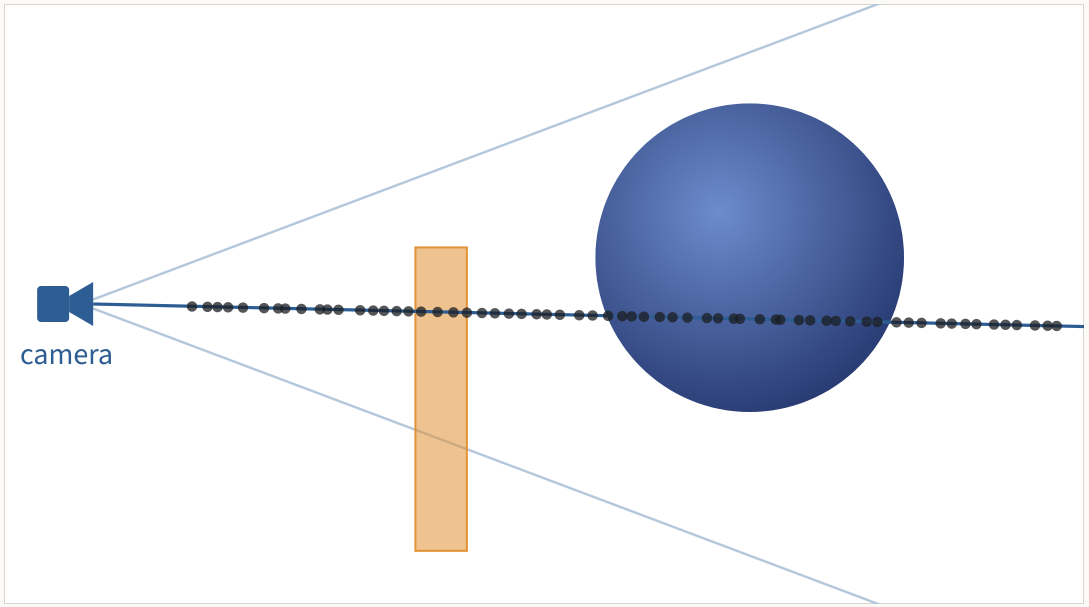


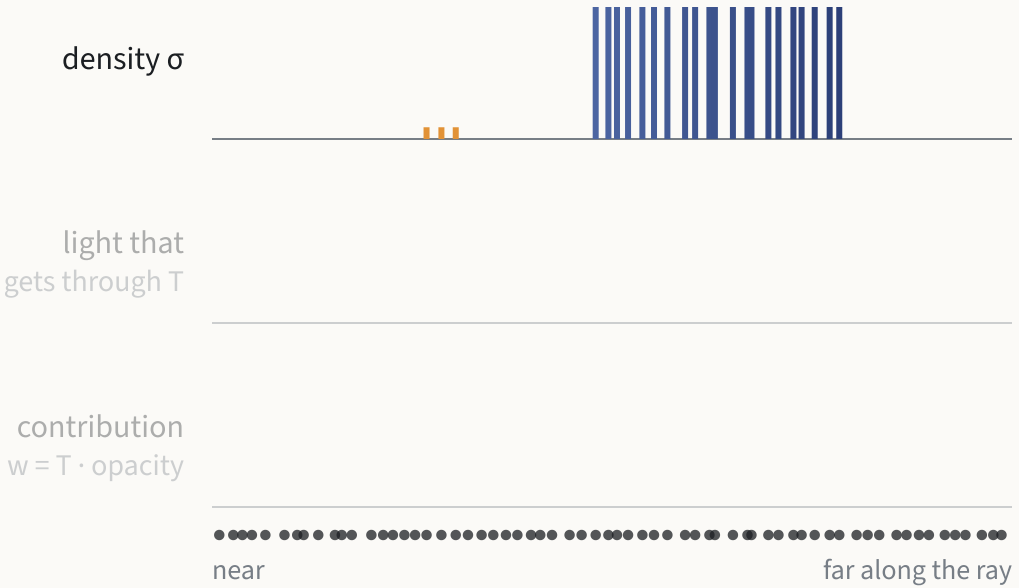
$$C = \Sigma_i \, T_i \, (1 - e^{-\sigma_i \delta_i}) \, c_i$$

Each pixel of the image looks along one ray. NeRF asks the network for density and colour at 64 points along it, spread evenly with a little random jitter.

Mildenhall et al. 2020, Eq. 3; δ_i is the gap between neighbouring points. Toy scene and density values as on the previous slide, 64 points per ray; the rendered strip is every pixel of the camera, computed with the same sum.

From densities and colours to one pixel



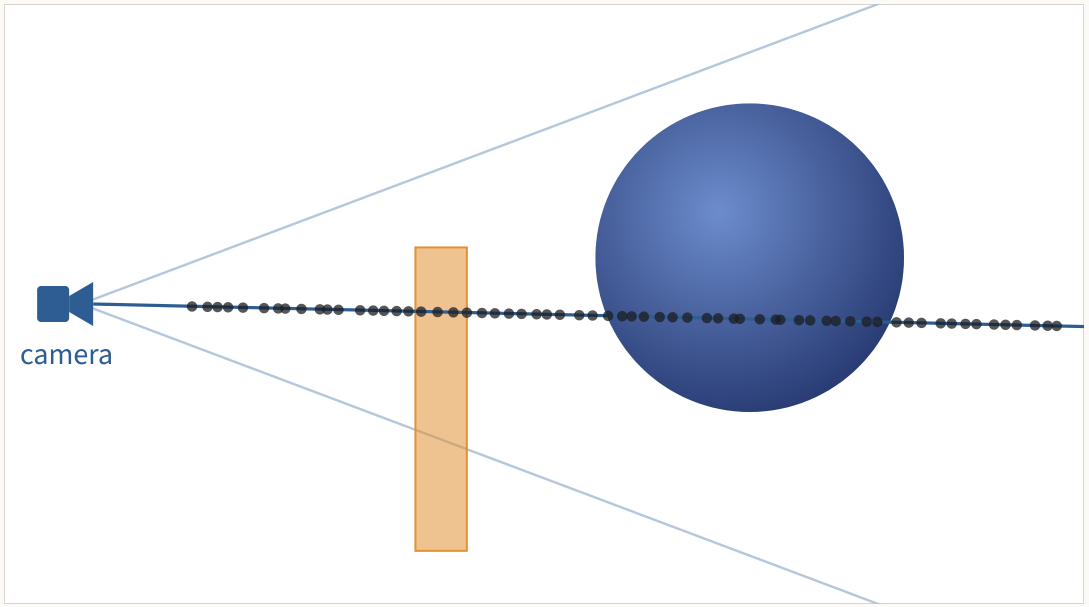


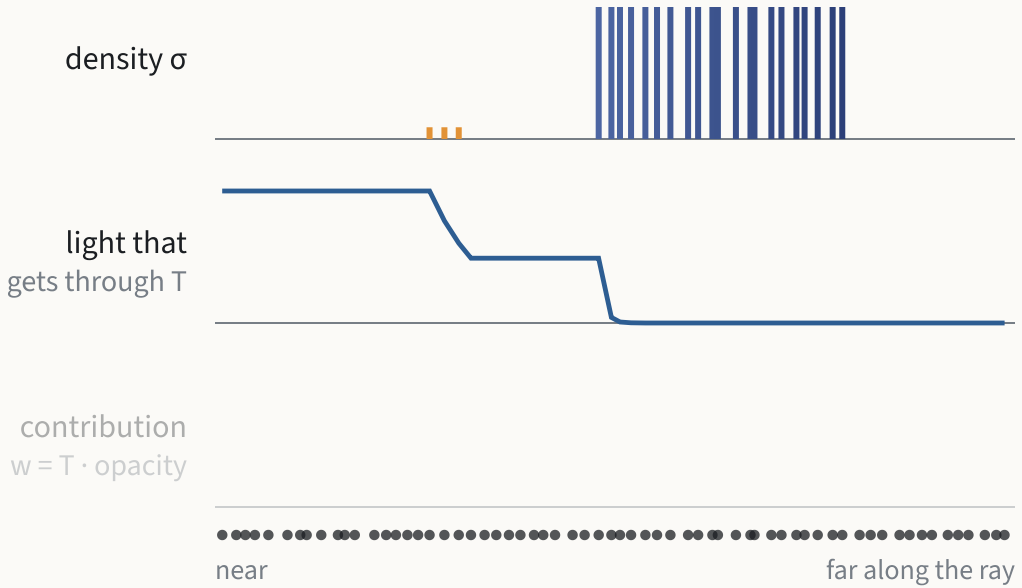
$$C = \Sigma_i \, T_i \, (1 - e^{-\sigma_i \delta_i}) \, c_i$$

Most points are in empty air, with density zero. The density rises inside the orange pane and is high inside the ball.

Mildenhall et al. 2020, Eq. 3; δ_i is the gap between neighbouring points. Toy scene and density values as on the previous slide, 64 points per ray; the rendered strip is every pixel of the camera, computed with the same sum.

From densities and colours to one pixel





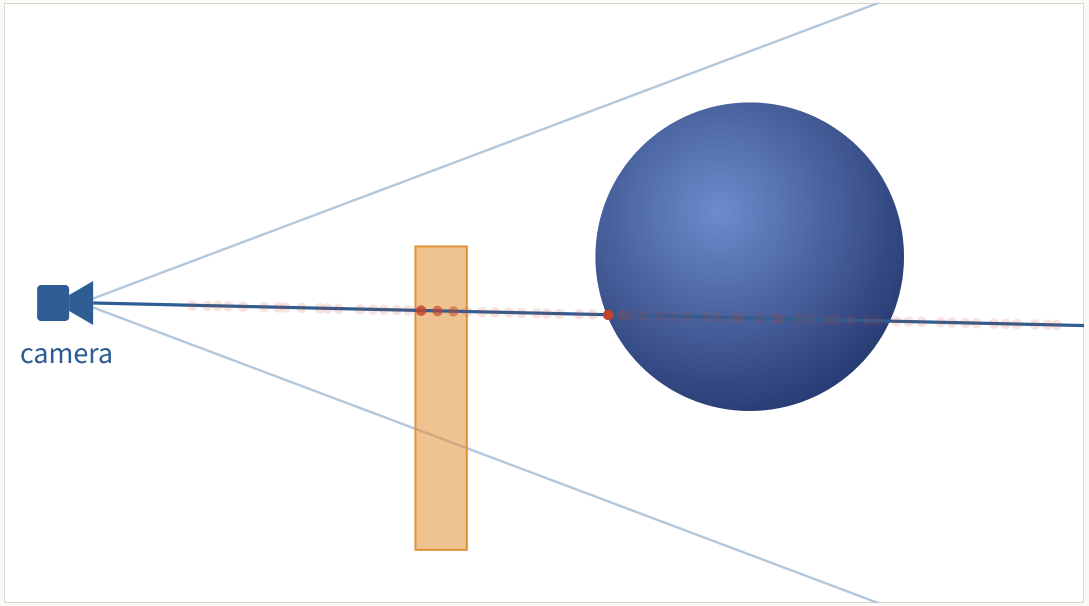
rendered

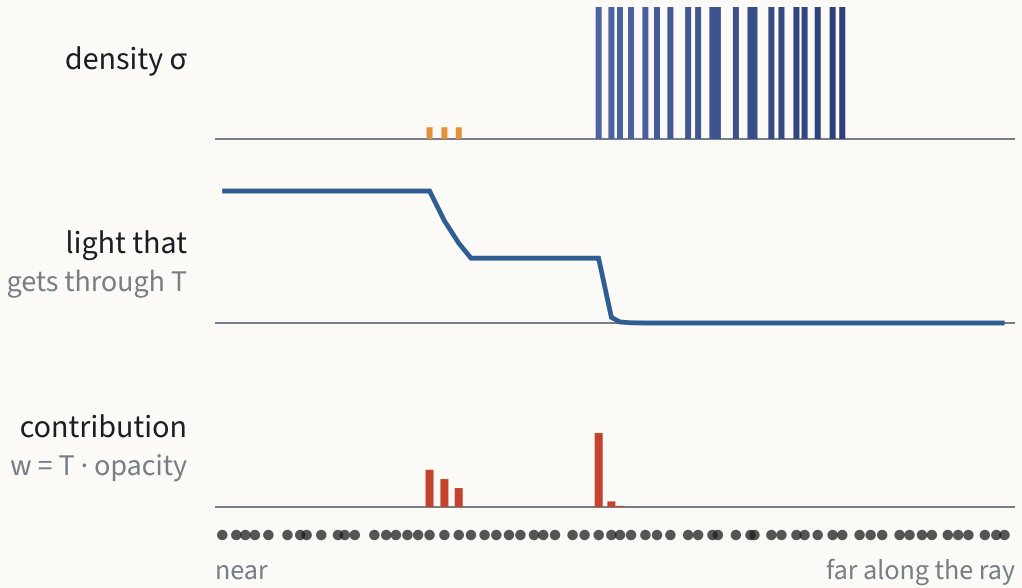
$$C = \Sigma_i \, T_i \, (1 - e^{-\sigma_i \delta_i}) \, c_i$$

T is the share of a point's light that still reaches the camera. It starts at 1 and falls each time the ray passes through matter, the same exponential law as X-ray attenuation in CT.

Mildenhall et al. 2020, Eq. 3; δ_i is the gap between neighbouring points. Toy scene and density values as on the previous slide, 64 points per ray; the rendered strip is every pixel of the camera, computed with the same sum.

From densities and colours to one pixel





rendered

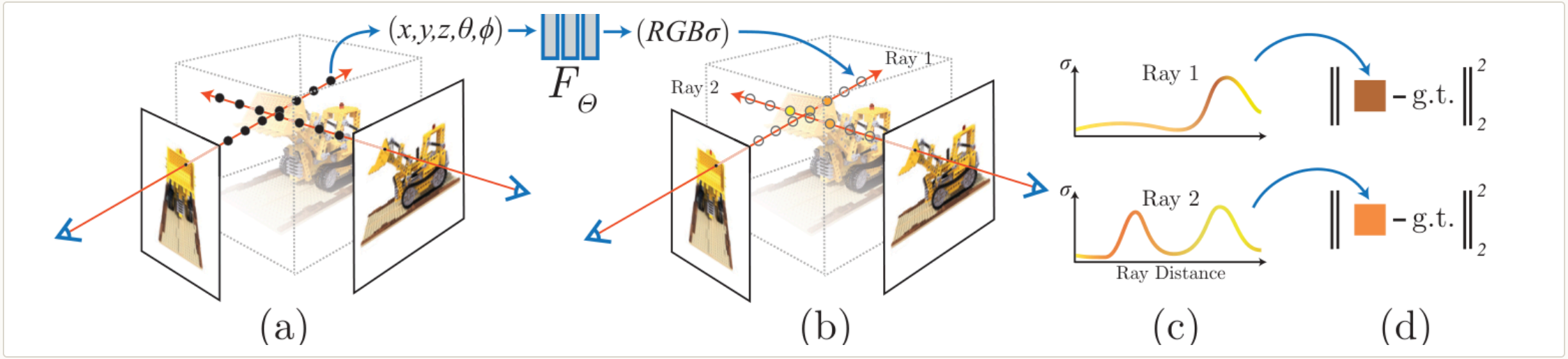
$$C = \Sigma_i \, T_i \, (1 - e^{-\sigma_i \delta_i}) \, c_i$$

pixel

Each point contributes its colour, weighted by its opacity and by T. The pixel is the sum, so the first surface dominates and whatever lies behind it hardly counts.

Mildenhall et al. 2020, Eq. 3; δ_i is the gap between neighbouring points. Toy scene and density values as on the previous slide, 64 points per ray; the rendered strip is every pixel of the camera, computed with the same sum.

Training: render pixels we know and compare



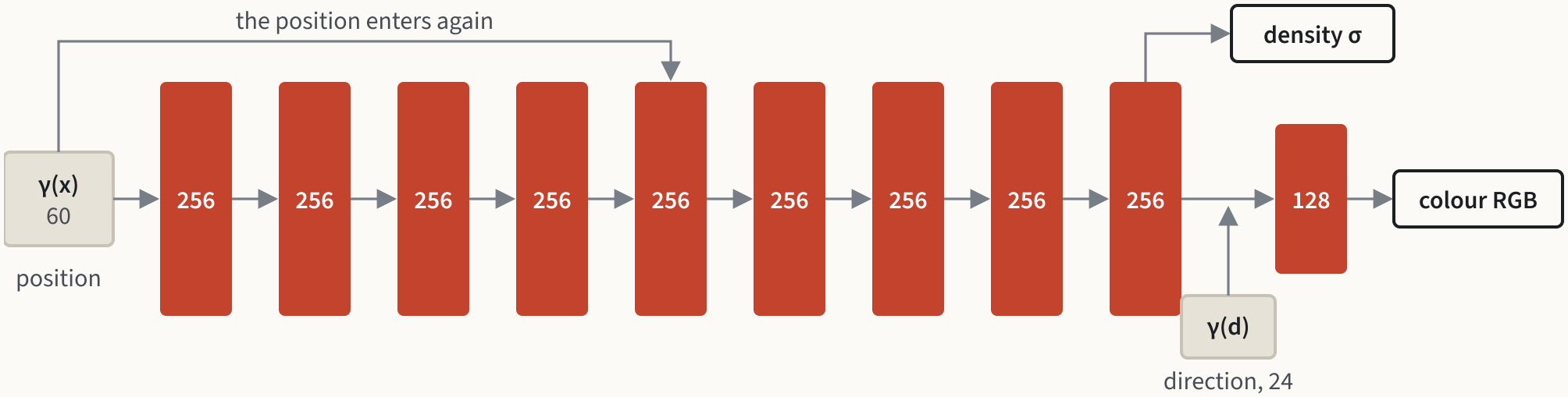
Each training step picks 4,096 random pixels from all the input photos. Every pixel is rendered with the current network, exactly as on the previous slide, and compared with its true colour.

$$\text{loss} = \Sigma_{\text{rays}} \, \| \, C_{\text{rendered}} - C_{\text{photo}} \, \|^2$$

The error flows back through the rendering sum into the network's weights. Nothing tells the network where the surfaces are: a point gets high density only because that makes the renders match the photos from many cameras at once.

Mildenhall et al. 2020, Fig. 2 and Eq. 6 (the loss is summed over the coarse and fine renders, introduced later). Batch of 4,096 rays: Sec. 5.3.

The network sees the direction only at the very end

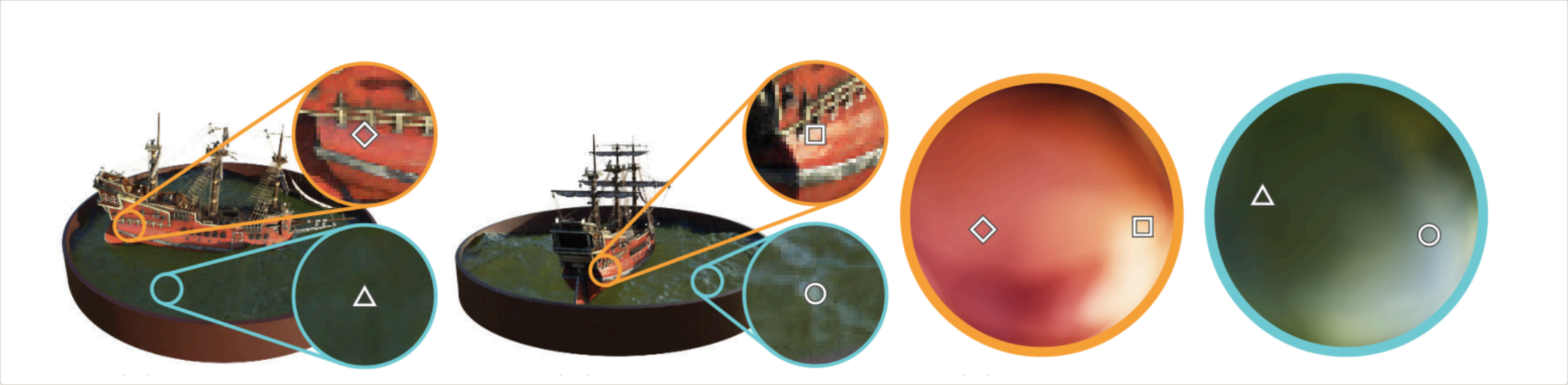


The position passes through eight layers before the network outputs the density. Because the direction is not used there, the shape is the same from every viewpoint, which keeps the views consistent.

The direction joins only for the last layer, enough to tint the colour for reflections and shine. Each network has about 0.6 million weights; with the second network introduced later, a scene takes 5 MB.

Redrawn from Mildenhall et al. 2020, Fig. 7 and Sec. 3. γ is the positional encoding (60 numbers for the position, 24 for the direction), explained shortly. Weight count computed from the layer sizes; 5 MB is from Sec. 6.3.

The same point can look different from different sides



The figure follows two fixed points of the Ship scene, one on the hull and one on the water. Seen from two different cameras, each point has a different colour, because the surfaces reflect light.

The discs on the right show the colour the network predicts for that point from every direction of the hemisphere. A model that ignores the direction has to average these into one colour.

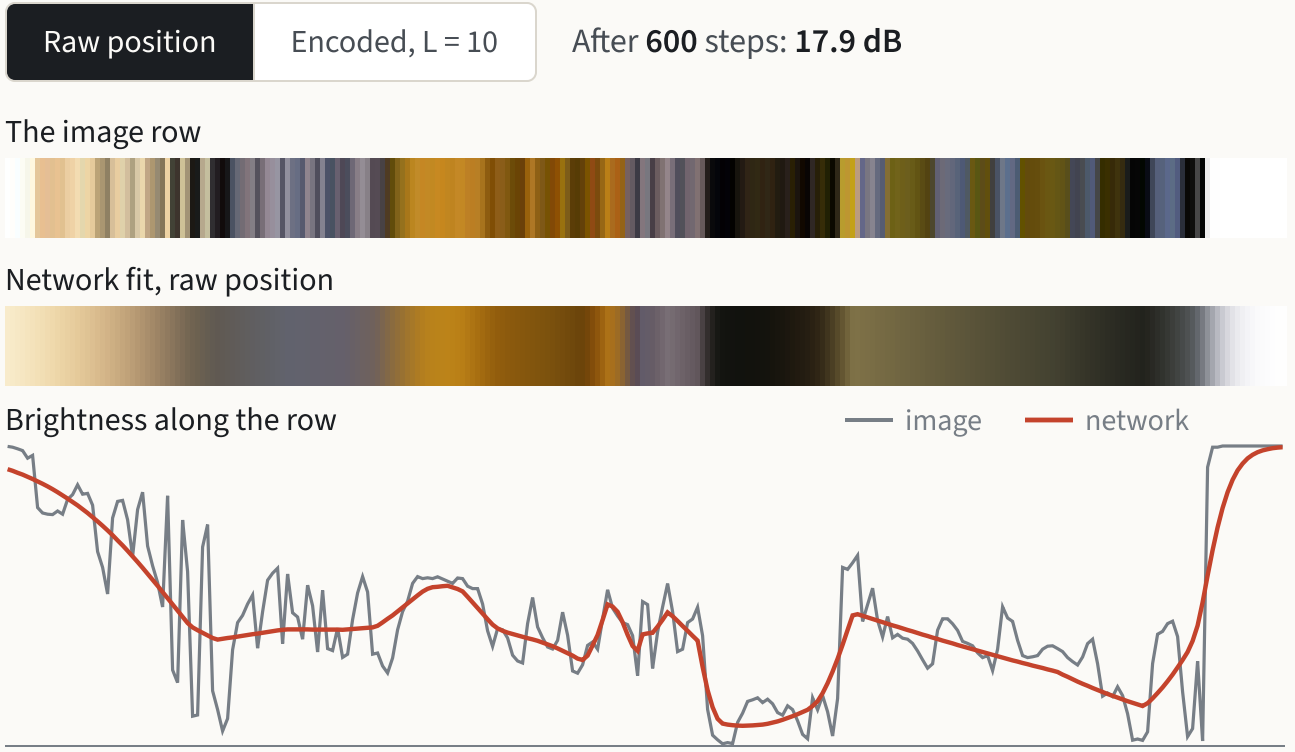
Mildenhall et al. 2020, Fig. 3: (a) view 1, (b) view 2, (c) predicted colour over all viewing directions for the hull point (orange ring) and the water point (blue ring).

Why raw coordinates give blurry results

A network fed raw coordinates learns smooth functions first and sharp edges very slowly. Here a small network is trained live, in this browser, to reproduce one row of pixels from the Lego image.

Feeding it sines and cosines of the position at ten frequencies, instead of the position itself, lets it fit edges as sharp as the image.

$$\gamma(p) = (\sin 2^0\pi p, \cos 2^0\pi p, \ldots, \sin 2^{L-1}\pi p, \cos 2^{L-1}\pi p)$$



Mildenhall et al. 2020, Sec. 5.1, Eq. 4; NeRF uses $L = 10$ for the position (60 numbers) and $L = 4$ for the direction (24). Demo: 256 pixels of row 440 of the Lego test image, a 3-layer network with 64 units, 600 Adam steps, same seed for both inputs. PSNR measures the fit to these pixels.

Why raw coordinates give blurry results

A network fed raw coordinates learns smooth functions first and sharp edges very slowly. Here a small network is trained live, in this browser, to reproduce one row of pixels from the Lego image.

Feeding it sines and cosines of the position at ten frequencies, instead of the position itself, lets it fit edges as sharp as the image.

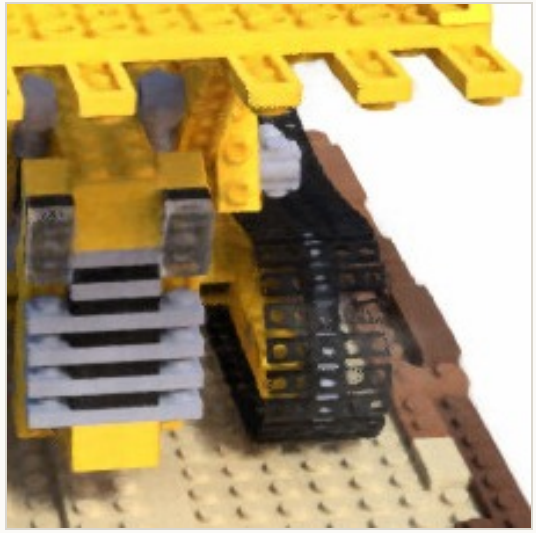
$$\gamma(p) = (\sin 2^0\pi p, \cos 2^0\pi p, \ldots, \sin 2^{L-1}\pi p, \cos 2^{L-1}\pi p)$$

Mildenhall et al. 2020, Sec. 5.1, Eq. 4; NeRF uses $L = 10$ for the position (60 numbers) and $L = 4$ for the direction (24). Demo: 256 pixels of row 440 of the Lego test image, a 3-layer network with 64 units, 600 Adam steps, same seed for both inputs. PSNR measures the fit to these pixels.

What the encoding and the direction input add

Ground truth

Complete model



Without the viewing direction

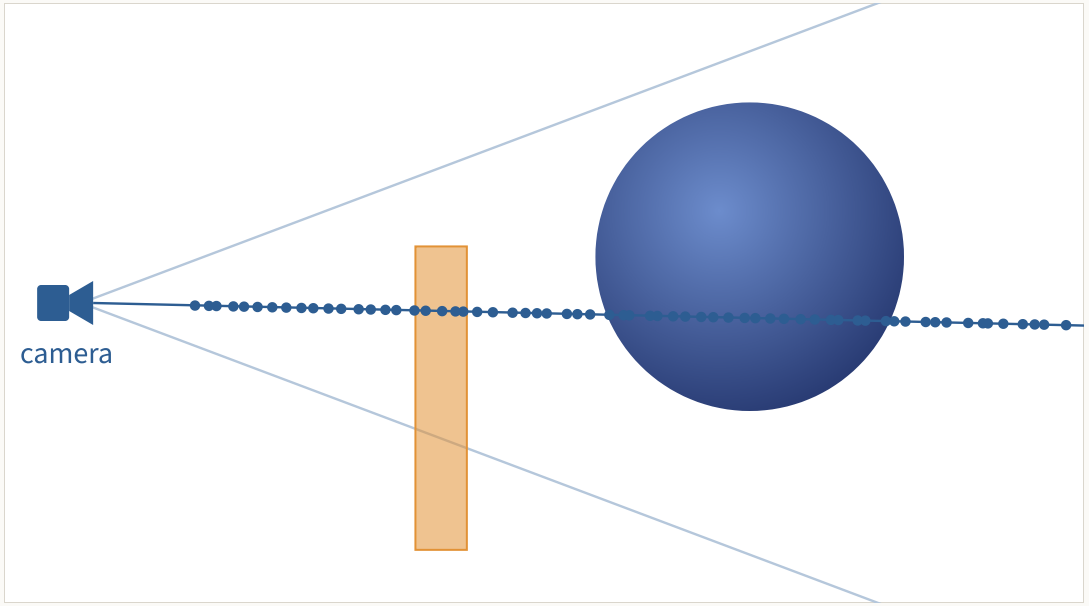
Without positional encoding

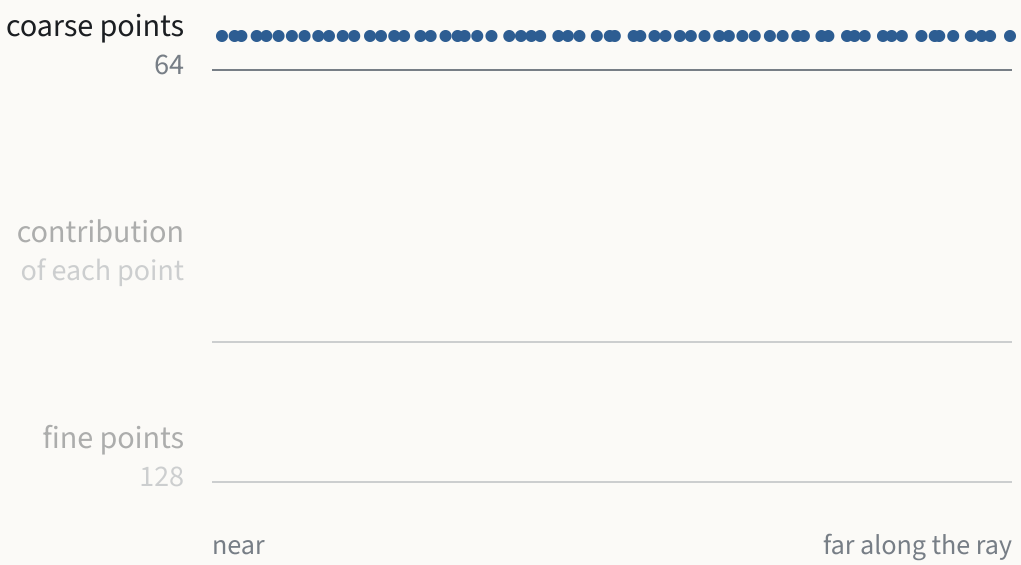
Without the viewing direction, the network still gets the shape right, but the bright reflections on the tread's metal pins are gone.

Without the positional encoding, the tread and the studs melt into a smooth blur: the same failure as the raw-position fit on the previous slide, now in 3D.

Mildenhall et al. 2020, Fig. 4: a zoom on the Lego bulldozer, a test view of the Realistic Synthetic 360° set. Images cut from the paper's full-resolution renders at the region the figure shows.

A coarse pass decides where to look closely



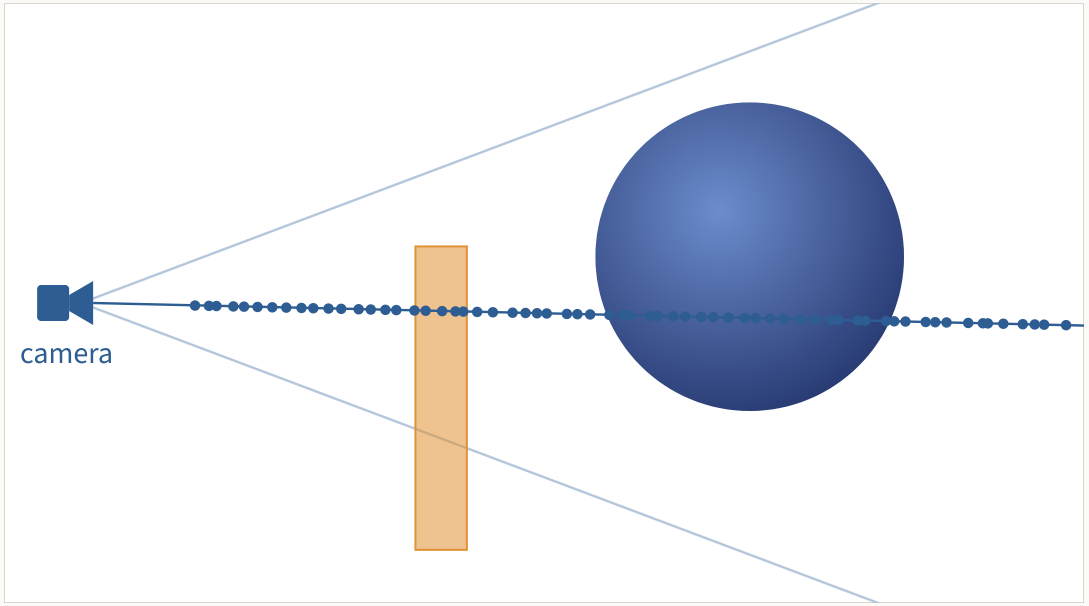


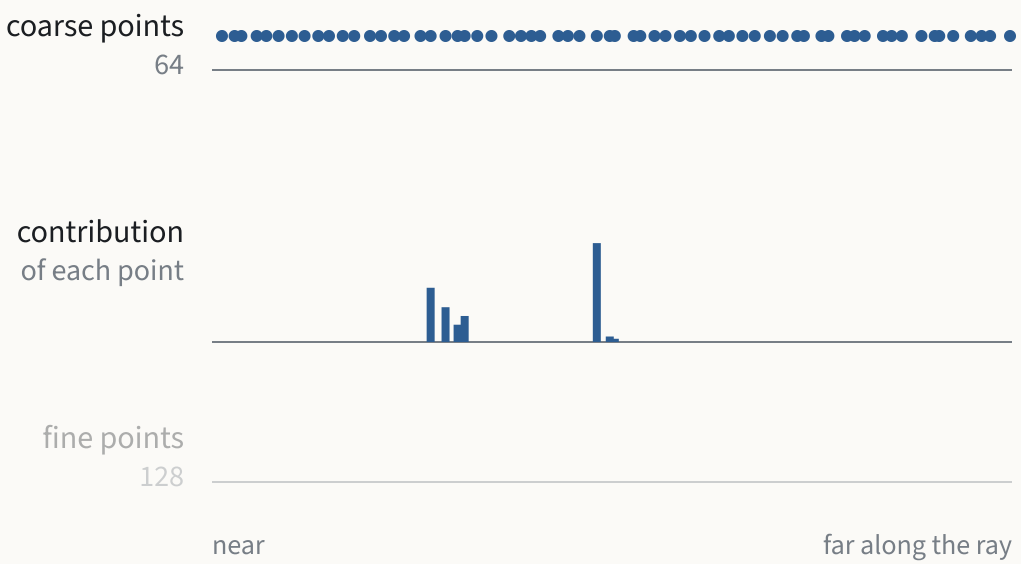
64 coarse points, evenly spread

Most of a ray is empty air or hidden behind the first surface, so evaluating the network there is wasted. NeRF first renders every ray with 64 evenly spread points, using a coarse network.

Mildenhall et al. 2020, Sec. 5.2, Eq. 5: $N_c = 64$ coarse and $N_f = 128$ fine points, drawn by inverse transform sampling from the coarse weights. Toy scene as before.

A coarse pass decides where to look closely



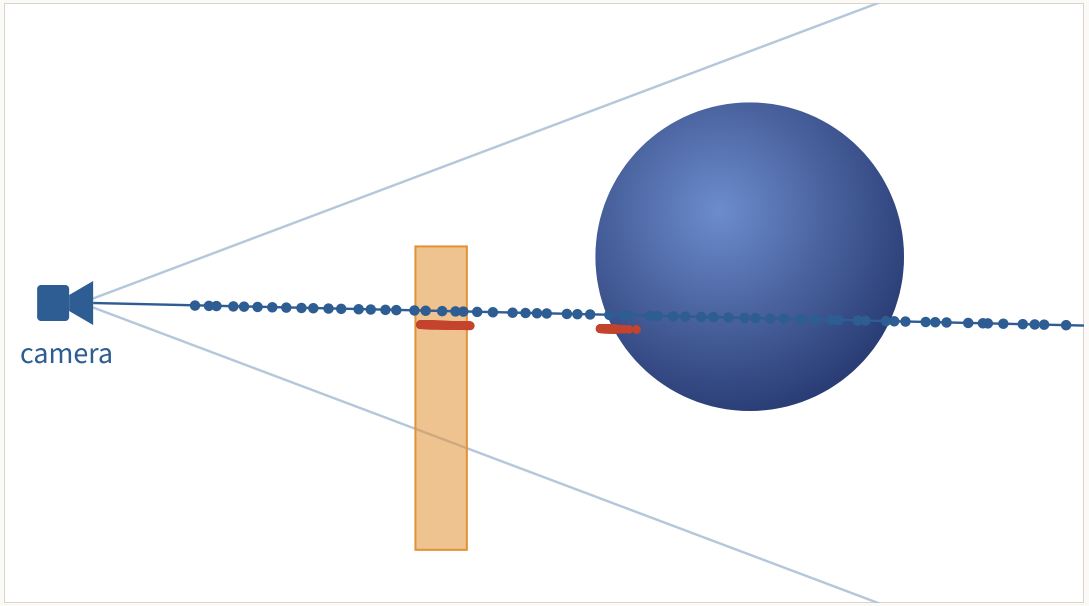


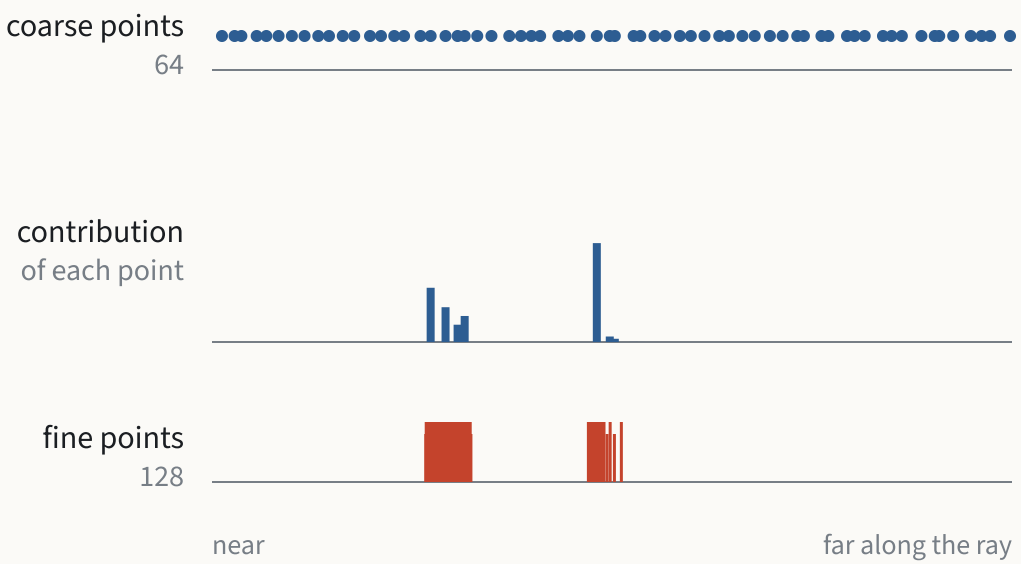
● 64 coarse points, evenly spread

The contributions of those 64 points show where the ray's colour came from: the pane and the front of the ball. Everything behind the ball contributes almost nothing.

Mildenhall et al. 2020, Sec. 5.2, Eq. 5: $N_c = 64$ coarse and $N_f = 128$ fine points, drawn by inverse transform sampling from the coarse weights. Toy scene as before.

A coarse pass decides where to look closely





64 coarse points, evenly spread 128 fine points

NeRF draws 128 more points from those contributions, so they land where the matter is. A second, fine network renders the ray from all 192 points.

Mildenhall et al. 2020, Sec. 5.2, Eq. 5: $N_c = 64$ coarse and $N_f = 128$ fine points, drawn by inverse transform sampling from the coarse weights. Toy scene as before.

One training step, and what a scene costs

```
# one optimisation step for one scene, simplified from Sec. 5.3
o, d, rgb = sample_rays(photos, poses, n=4096)              # random pixels from all photos
t_c = stratified(near, far, n=64)                           # coarse points, evenly spread with jitter
C_c, w = render(coarse_net, o, d, t_c)                      # Eq. 3, and the weights w
t_f = sort(t_c + sample_pdf(t_c, w, n=128))                 # 128 more where w is large
C_f, _ = render(fine_net, o, d, t_f)                        # all 192 points
loss = ((C_c - rgb)**2).sum() + ((C_f - rgb)**2).sum()     # Eq. 6
loss.backward(); adam.step()                                # learning rate 5e-4 decaying to 5e-5
```

Each scene is trained from scratch: 100,000 to 300,000 such steps, which takes one to two days on one NVIDIA V100 GPU.

For real photos, the camera positions come from COLMAP, a structure-from-motion program that matches features across the photos themselves.

Rendering one 800 × 800 image means 640,000 rays and 150 to 200 million network queries, about 30 seconds per image.

Mildenhall et al. 2020, Sec. 5.3 and Appendix A. The code is pseudo-code following the paper, not the official implementation. COLMAP: Schönberger & Frahm 2016.

The three test sets

The paper shows no images of this set.

Diffuse Synthetic 360°

Four simple objects with matte surfaces, from DeepVoxels. 479 training views at 512 × 512.

Realistic Synthetic 360°

Eight complex objects with shiny and see-through materials, made for this paper. 100 training views at 800 × 800, 200 test views.

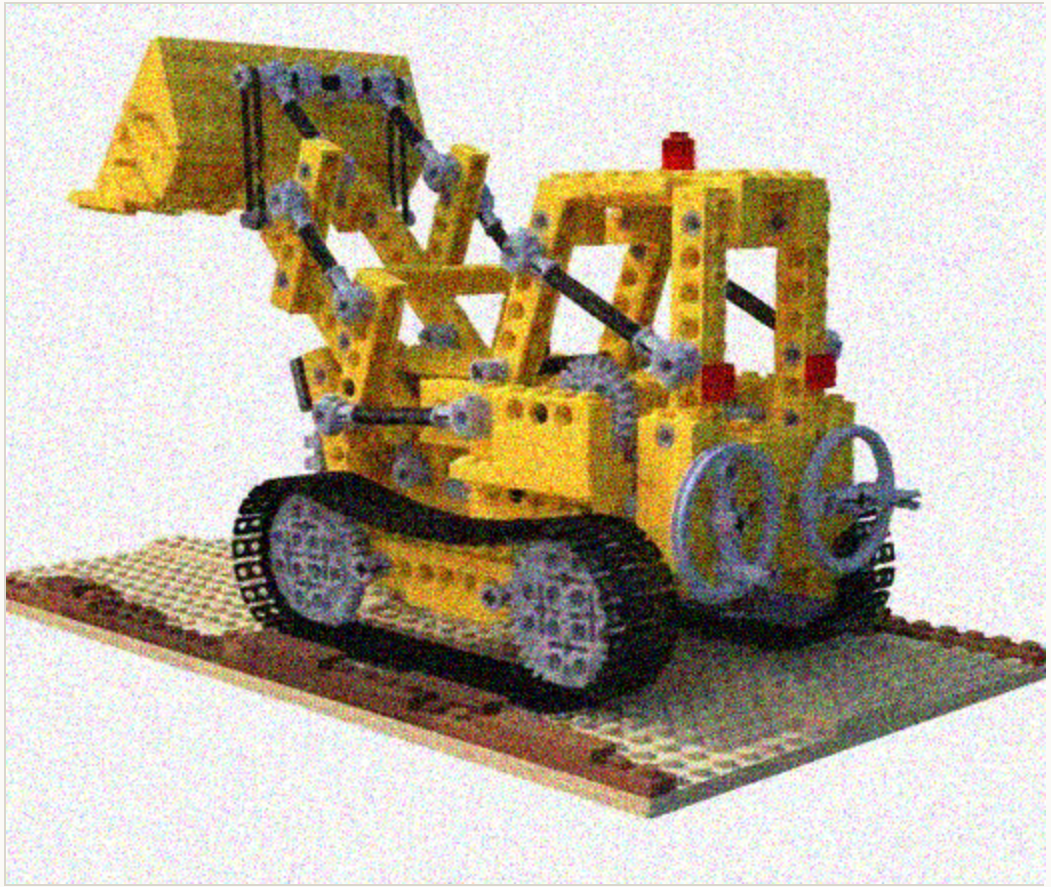
Real Forward-Facing

Eight real scenes shot with a phone, all from the front. 20 to 62 photos each, one in eight held out for testing.

Every method renders the held-out views, and each render is compared with the real image. PSNR is the pixel error in decibels, and 3 dB more means half the squared error. SSIM compares local structure (1 is perfect). LPIPS compares the images through a deep network, closer to human judgement; lower is better.

Mildenhall et al. 2020, Sec. 6.1 and Table 1. Images are test views from Fig. 5 and Fig. 6. DeepVoxels: Sitzmann et al. 2019; LPIPS: Zhang et al. 2018; SSIM: Wang et al. 2004.

What a decibel of PSNR means



A decibel measures how many times larger one power is than another: ten times the logarithm of their ratio. That is why it is used for sound, radio signals and images alike.

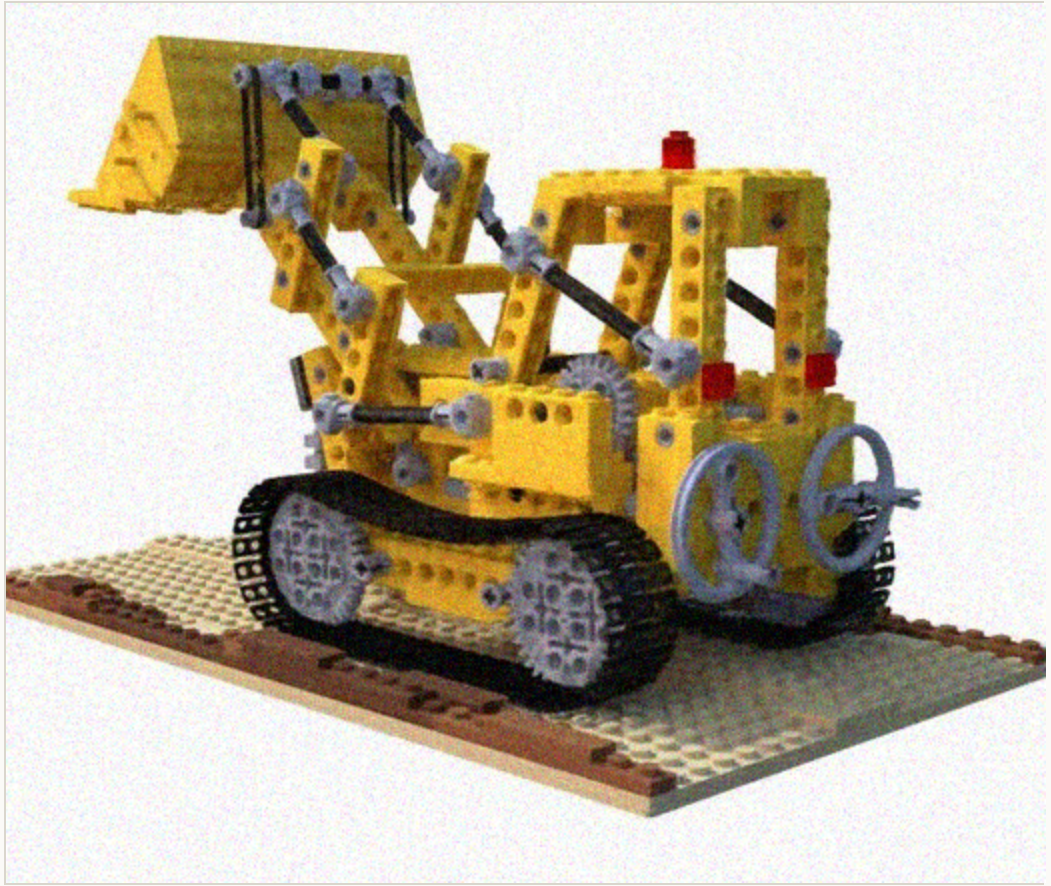
PSNR compares the brightest possible pixel with the average squared error between the render and the photo. Every 10 dB means ten times less squared error; 3 dB means half.

20 dB | 26 dB | 31 dB | 40 dB

At 20 dB a typical pixel is off by 26 of 255 brightness levels, and the error is obvious.

PSNR = 10 $\log_{10}$(1 / mean squared error) for pixel values from 0 to 1. Image: NeRF's Lego render from Mildenhall et al. 2020, with random noise added until its PSNR against the clean image is the value shown.

What a decibel of PSNR means



A decibel measures how many times larger one power is than another: ten times the logarithm of their ratio. That is why it is used for sound, radio signals and images alike.

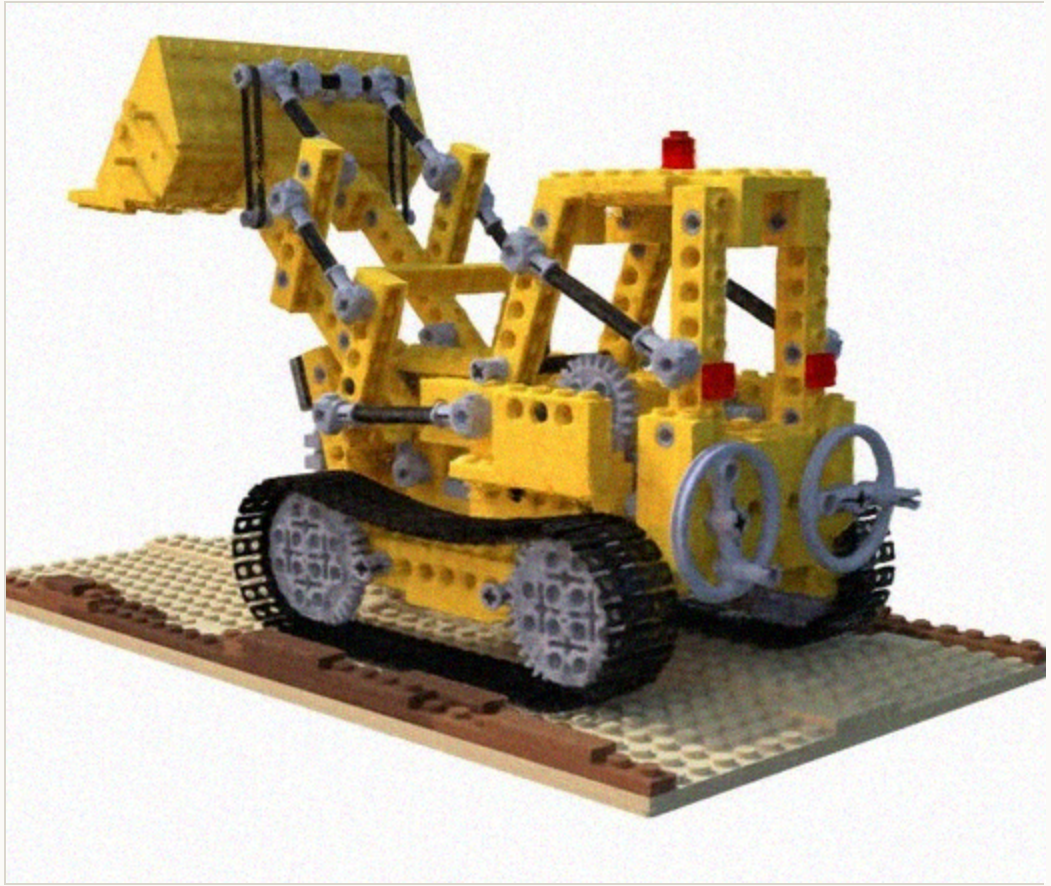
PSNR compares the brightest possible pixel with the average squared error between the render and the photo. Every 10 dB means ten times less squared error; 3 dB means half.

20 dB	**26 dB**	31 dB	40 dB

At 26 dB, about Neural Volumes on the synthetic scenes, a pixel is off by 13 levels.

PSNR = 10 $\log_{10}$(1 / mean squared error) for pixel values from 0 to 1. Image: NeRF's Lego render from Mildenhall et al. 2020, with random noise added until its PSNR against the clean image is the value shown.

What a decibel of PSNR means



A decibel measures how many times larger one power is than another: ten times the logarithm of their ratio. That is why it is used for sound, radio signals and images alike.

PSNR compares the brightest possible pixel with the average squared error between the render and the photo. Every 10 dB means ten times less squared error; 3 dB means half.

20 dB	26 dB	**31 dB**	40 dB

At 31 dB, NeRF on the same scenes, the error is 7.2 levels: visible only from close up.

PSNR = 10 $\log_{10}$(1 / mean squared error) for pixel values from 0 to 1. Image: NeRF's Lego render from Mildenhall et al. 2020, with random noise added until its PSNR against the clean image is the value shown.

What a decibel of PSNR means

A decibel measures how many times larger one power is than another: ten times the logarithm of their ratio. That is why it is used for sound, radio signals and images alike.

PSNR compares the brightest possible pixel with the average squared error between the render and the photo. Every 10 dB means ten times less squared error; 3 dB means half.

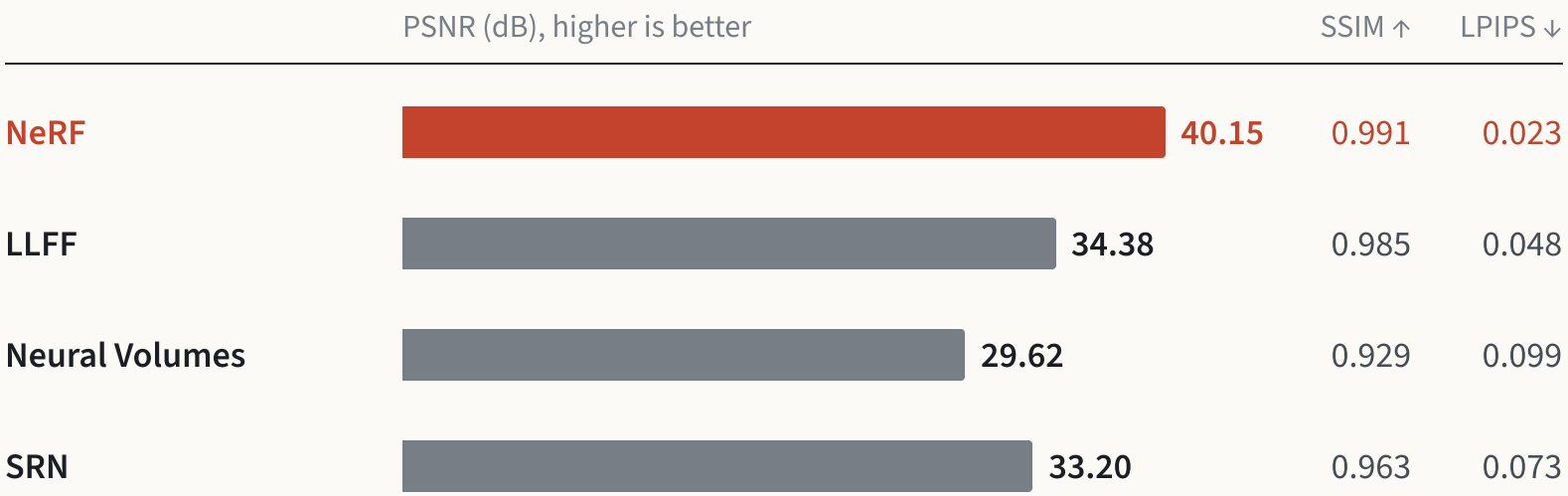
20 dB | 26 dB | 31 dB | **40 dB**

At 40 dB, NeRF on the matte objects, it is 2.6 levels and the image looks clean.

PSNR = 10 $\log_{10}$(1 / mean squared error) for pixel values from 0 to 1. Image: NeRF's Lego render from Mildenhall et al. 2020, with random noise added until its PSNR against the clean image is the value shown.

NeRF has the highest PSNR on all three test sets

Diffuse Synthetic | Realistic Synthetic | Real Forward-Facing

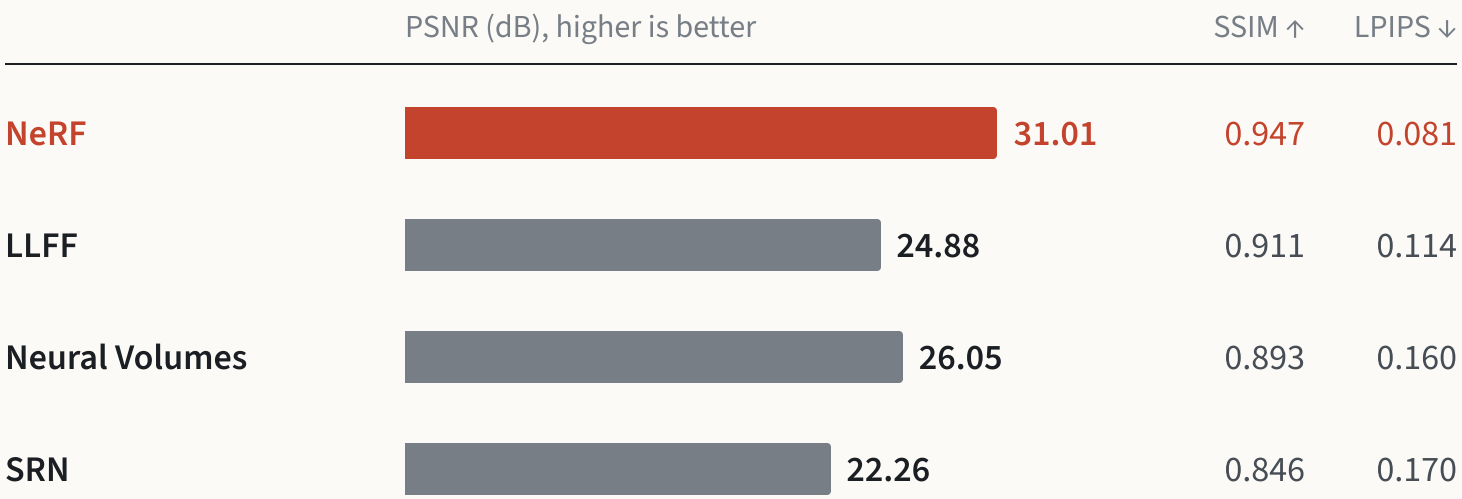


Four matte objects with simple shapes. Every method does well, but NeRF is almost 6 dB above the next best, LLFF: about a quarter of its squared error.

Mildenhall et al. 2020, Table 1. SRN: Sitzmann et al. 2019; Neural Volumes (NV): Lombardi et al. 2019; LLFF: Mildenhall et al. 2019. NV cannot run on the real scenes because it needs an object inside a bounded volume.

NeRF has the highest PSNR on all three test sets

Diffuse Synthetic | **Realistic Synthetic** | Real Forward-Facing



Eight objects with fine detail and shiny or see-through materials. NeRF reaches 31.0 dB, the next best, Neural Volumes, 26.1: about a third of the error.

Mildenhall et al. 2020, Table 1. SRN: Sitzmann et al. 2019; Neural Volumes (NV): Lombardi et al. 2019; LLFF: Mildenhall et al. 2019. NV cannot run on the real scenes because it needs an object inside a bounded volume.

NeRF has the highest PSNR on all three test sets

Diffuse Synthetic | Realistic Synthetic | **Real Forward-Facing**

	PSNR (dB), higher is better	SSIM ↑	LPIPS ↓
NeRF	**26.50**	0.811	0.250
LLFF	**24.13**	0.798	0.212
Neural Volumes	needs a bounded object	–	–
SRN	**22.84**	0.668	0.378

Real phone captures from the front, with 20 to 62 photos each. The margin shrinks to 2.4 dB over LLFF, which was designed for exactly these scenes and has the better LPIPS.

Mildenhall et al. 2020, Table 1. SRN: Sitzmann et al. 2019; Neural Volumes (NV): Lombardi et al. 2019; LLFF: Mildenhall et al. 2019. NV cannot run on the real scenes because it needs an object inside a bounded volume.

What the difference looks like

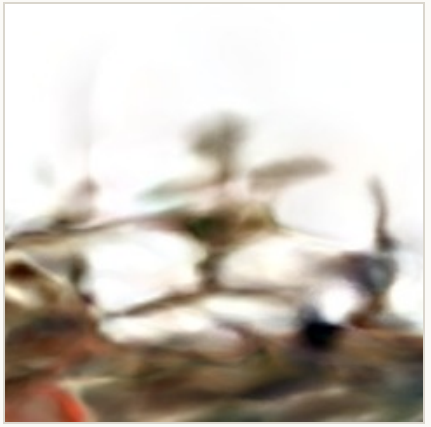
Ship | Lego | Microphone | Fern | T-Rex

Ground truth

NeRF

LLFF

Neural Volumes



SRN

The ship's rigging: thin ropes and masts against the sky. NeRF keeps most of them, Neural Volumes loses the ropes and SRN blurs everything.

Mildenhall et al. 2020, Fig. 5 (synthetic) and Fig. 6 (real): test views, cut from the paper's full-resolution renders. The same region is shown for every method.

What the difference looks like

Ship | **Lego** | Microphone | Fern | T-Rex

Ground truth

NeRF

LLFF

Neural Volumes

SRN

The gears and the tread. NeRF keeps the spokes and the tread links; LLFF shows ghosting, Neural Volumes and SRN smooth the detail away.

Mildenhall et al. 2020, Fig. 5 (synthetic) and Fig. 6 (real): test views, cut from the paper's full-resolution renders. The same region is shown for every method.

What the difference looks like

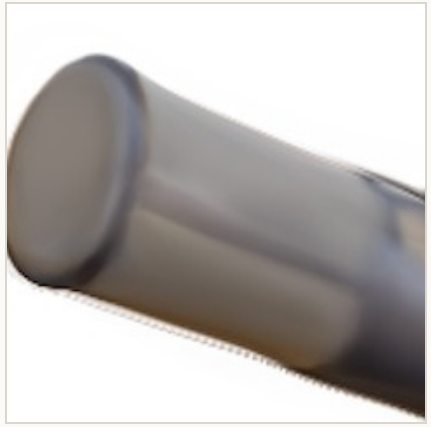
Ship | Lego | **Microphone** | Fern | T-Rex

Ground truth

NeRF

LLFF

Neural Volumes



SRN

The microphone's mesh grille. Only NeRF reproduces the fine mesh; Neural Volumes smears it and SRN draws a plain cylinder.

Mildenhall et al. 2020, Fig. 5 (synthetic) and Fig. 6 (real): test views, cut from the paper's full-resolution renders. The same region is shown for every method.

What the difference looks like

Ship | Lego | Microphone | **Fern** | T-Rex

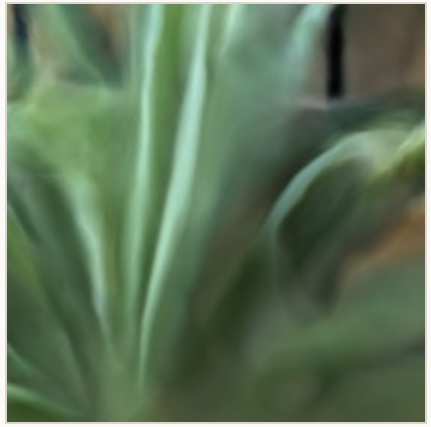
Ground truth

NeRF

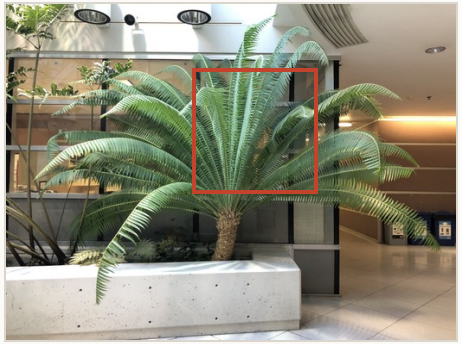
LLFF

not run on real scenes

Neural Volumes



SRN



A real photo. The fern's thin leaves stay sharp in NeRF and LLFF; SRN keeps only the rough shape. Neural Volumes cannot run on real scenes.

Mildenhall et al. 2020, Fig. 5 (synthetic) and Fig. 6 (real): test views, cut from the paper's full-resolution renders. The same region is shown for every method.

What the difference looks like

Ship | Lego | Microphone | Fern | **T-Rex**

Ground truth

NeRF

LLFF

not run on real scenes

Neural Volumes

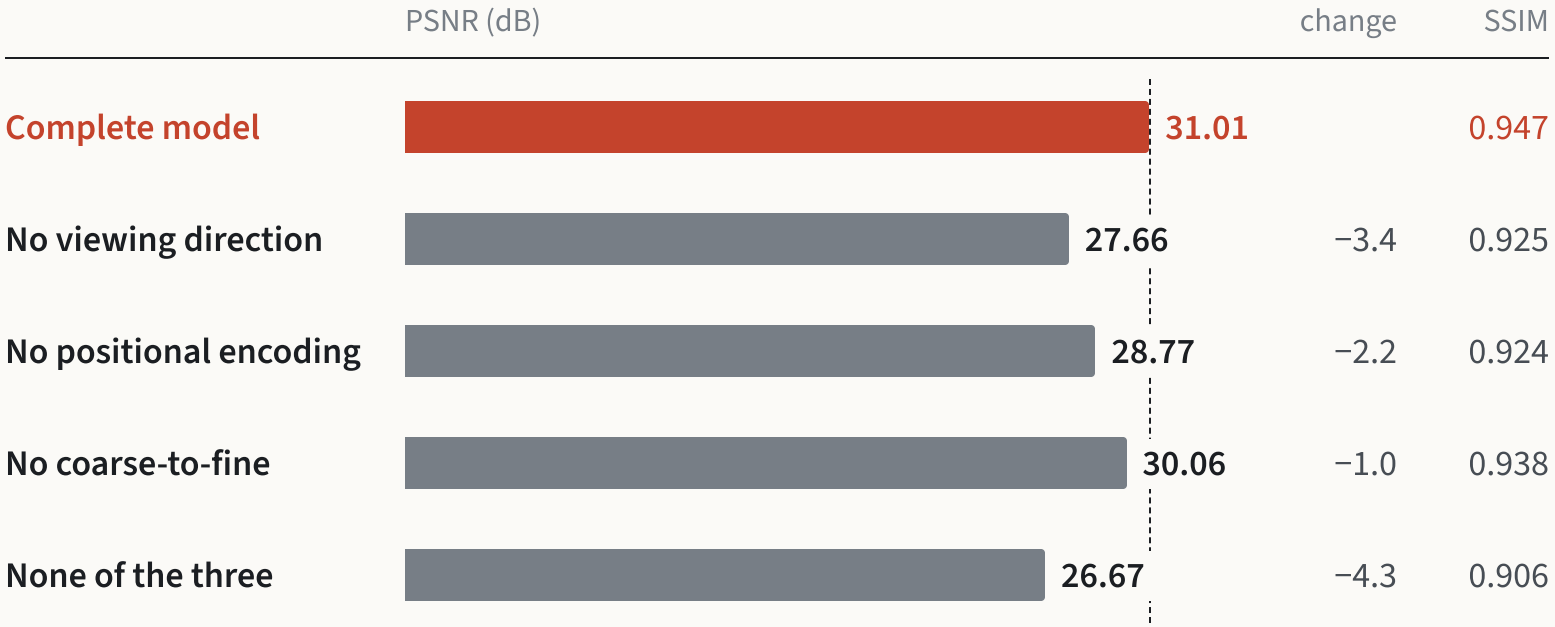
SRN

The skeleton's ribs and the railing behind them. NeRF keeps each rib separate; SRN merges them into one smooth shape.

Mildenhall et al. 2020, Fig. 5 (synthetic) and Fig. 6 (real): test views, cut from the paper's full-resolution renders. The same region is shown for every method.

What each part contributes

Remove a part | Fewer photos | Number of frequencies



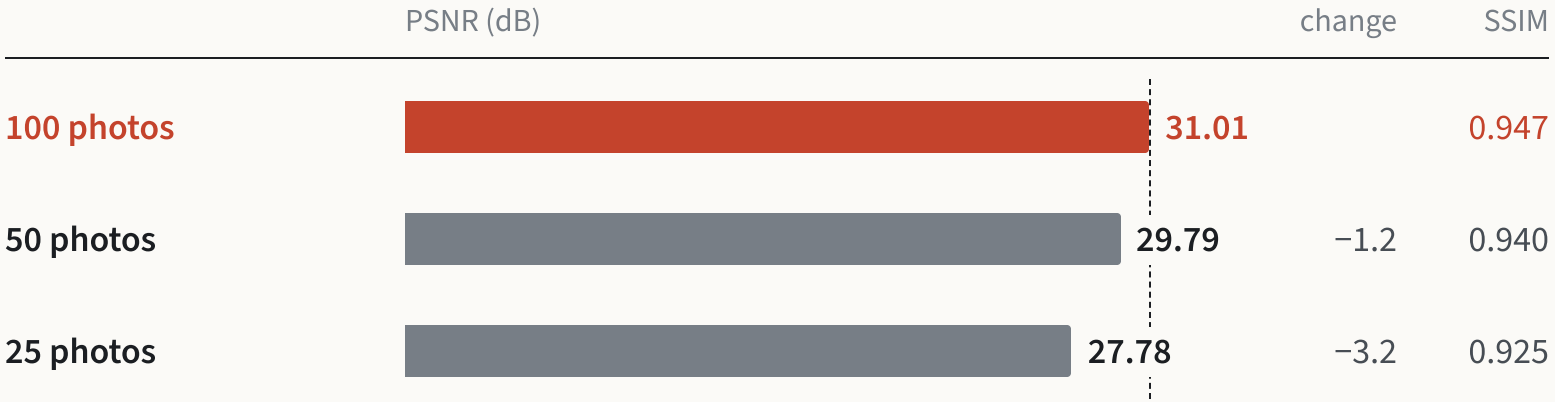
On these shiny scenes the viewing direction matters most: without it the score drops by 3.4 dB. The encoding is worth 2.2 dB and coarse-to-fine sampling 1.0 dB.

Without all three, the model loses 4.3 dB.

Mildenhall et al. 2020, Table 2: mean over the eight Realistic Synthetic 360° scenes. The dashed line is the complete model, 31.01 dB.

What each part contributes

Remove a part | **Fewer photos** | Number of frequencies



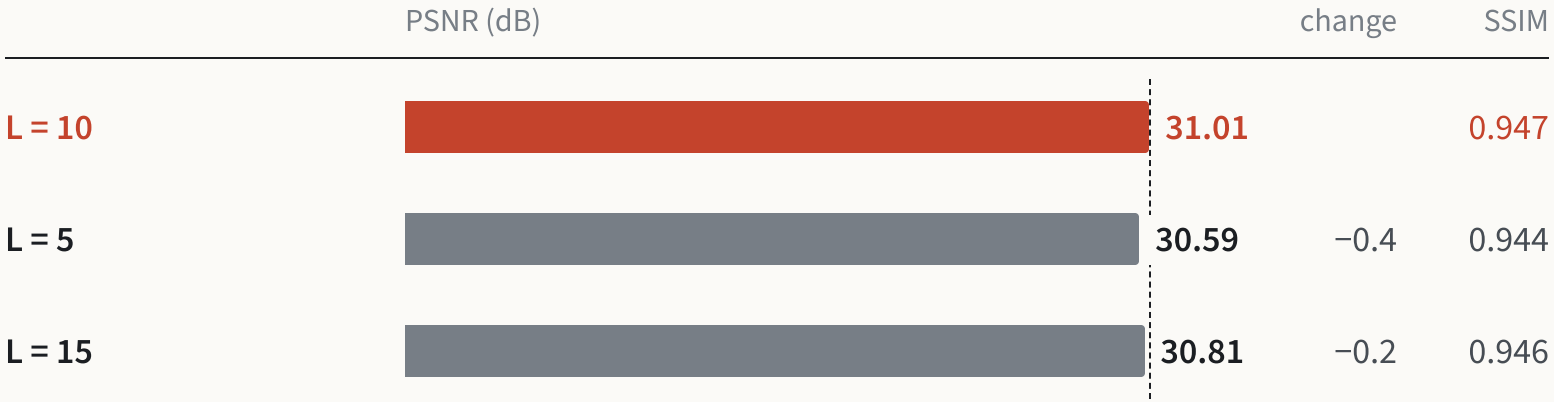
With 50 photos instead of 100 NeRF loses 1.2 dB, with 25 photos 3.2 dB.

Even with 25 photos it still beats every baseline that gets all 100.

Mildenhall et al. 2020, Table 2: mean over the eight Realistic Synthetic 360° scenes. The dashed line is the complete model, 31.01 dB.

What each part contributes

Remove a part | Fewer photos | **Number of frequencies**



Five frequencies are too few, and fifteen are no better than ten.

Once the finest sine is finer than the pixels, there is no more detail to fit.

Mildenhall et al. 2020, Table 2: mean over the eight Realistic Synthetic 360° scenes. The dashed line is the complete model, 31.01 dB.

Small to store, slow to train and to render

over 15 GB
LLFF, one scene

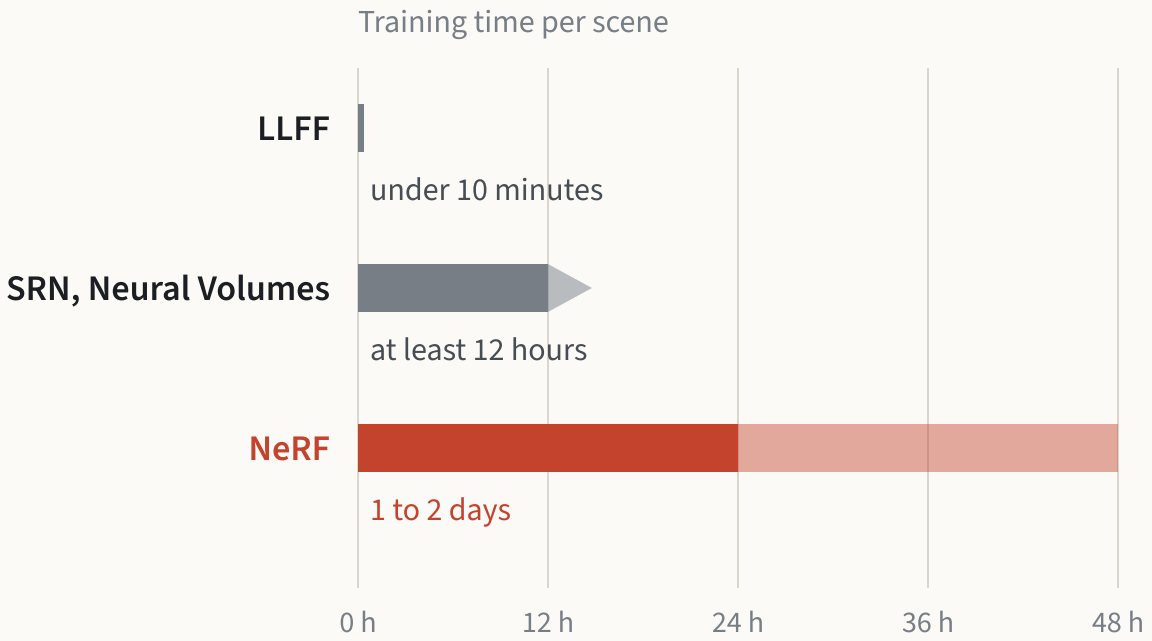
5 MB
NeRF

LLFF keeps a stack of image layers for every input view, over 15 GB for one synthetic scene.

NeRF's two networks take 5 MB, about 3,000 times less, and even less than the input photos themselves.

Mildenhall et al. 2020, Sec. 6.3 (storage, training time) and Appendix A (30 s per 800 × 800 image on a V100). Squares are sized by area.

Small to store, slow to train and to render



Rendering one 800 × 800 image with NeRF: about 30 sec

LLFF needs under ten minutes for a new scene because its network was trained beforehand on many scenes. Methods that fit one scene from scratch take at least 12 hours, and NeRF one to two days.

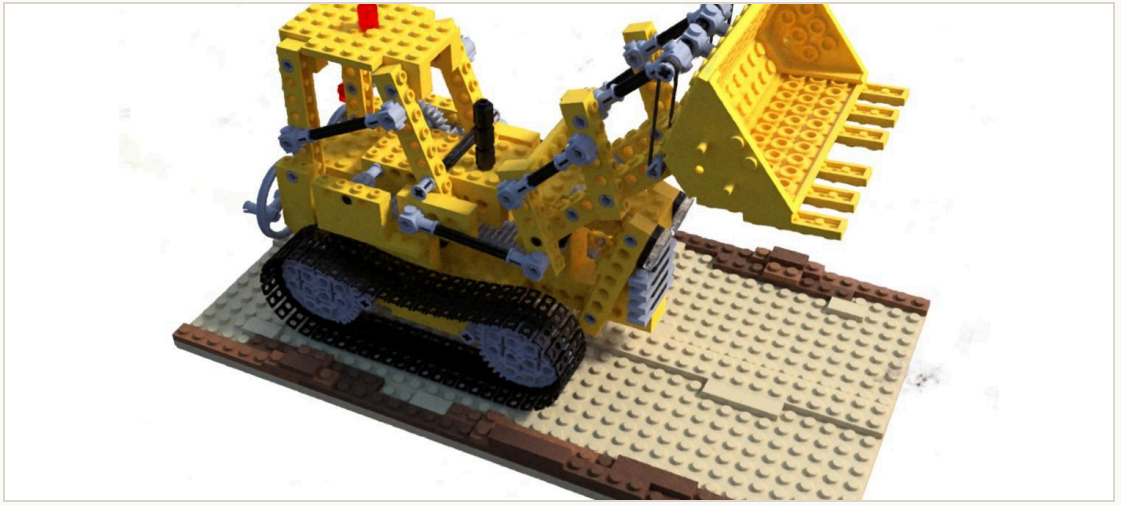
Rendering is slow too: every pixel needs 192 network queries.

Mildenhall et al. 2020, Sec. 6.3 (storage, training time) and Appendix A (30 s per 800 × 800 image on a V100). Squares are sized by area.

Making it fast

NeRF needs one to two days to train a scene and half a minute to render one image. Two later papers removed most of that cost.

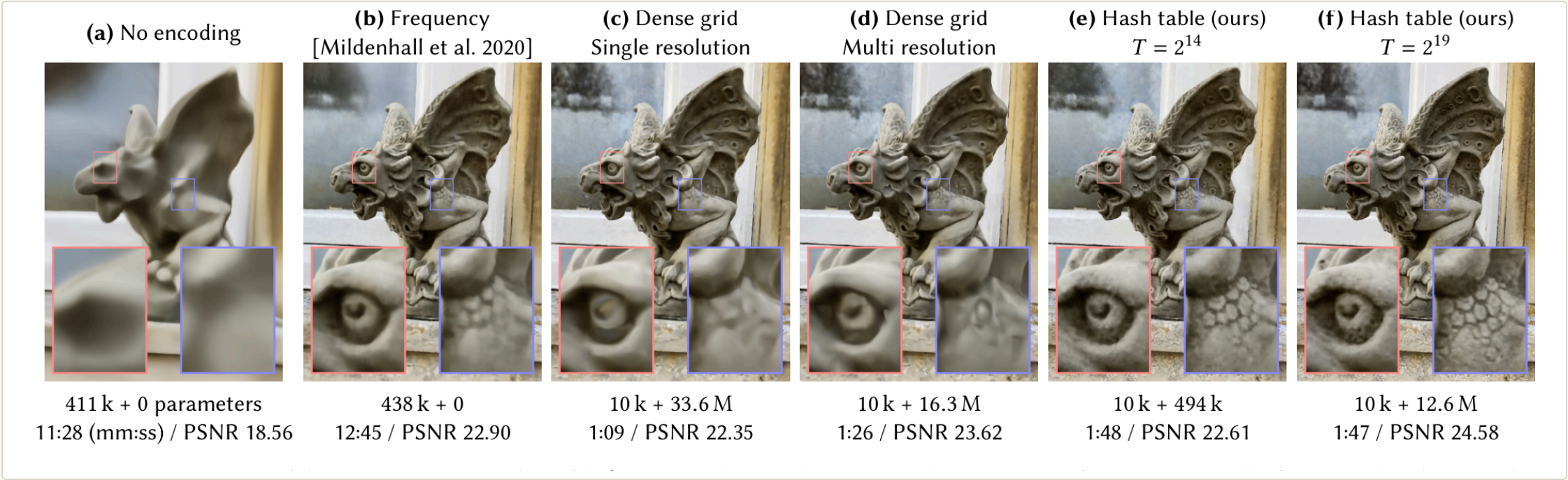
Instant-NGP (2022) keeps the rays but moves most of the work from the network into a learned lookup table. 3D Gaussian splatting (2023) drops both the network and the rays.



Instant-NGP, after 15 seconds of training

3D Gaussian splatting, rendered in real time

Instant-NGP: learned features in a grid, and a tiny network

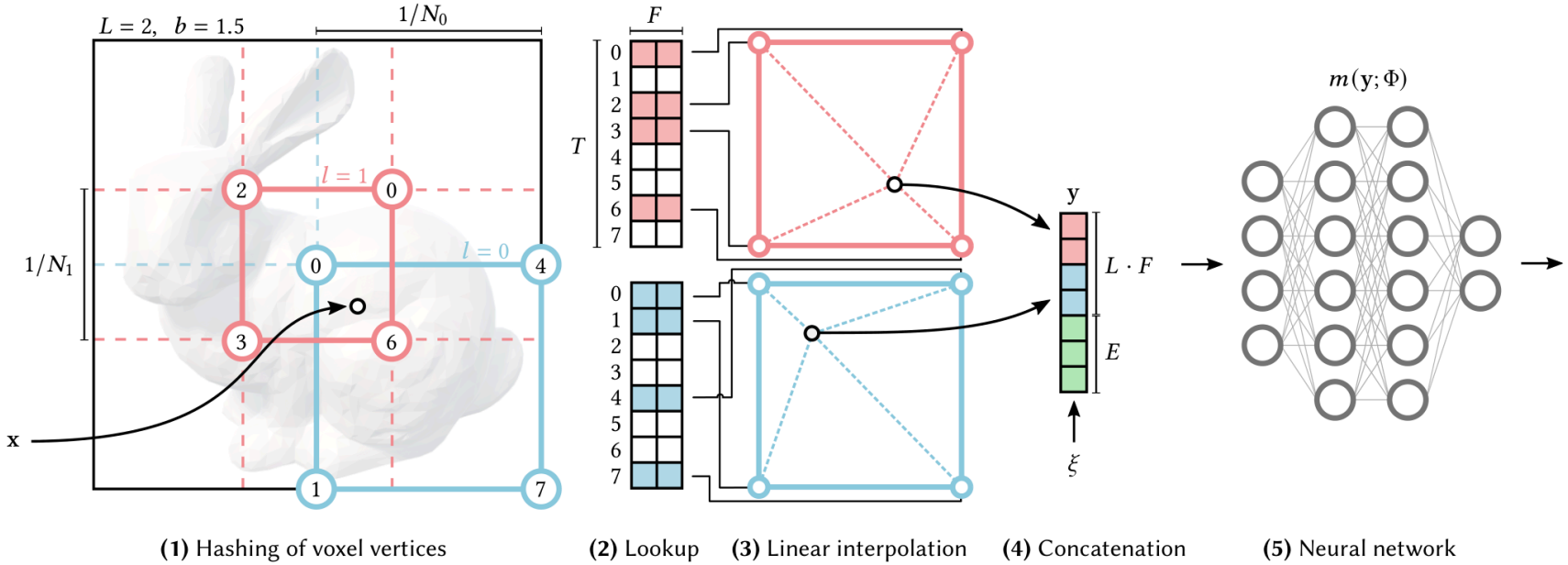


The paper trains the same scene six ways for 11,000 steps each. NeRF's sine encoding (b) needs a large network of 438,000 weights and takes almost 13 minutes.

A grid of learned feature vectors (c, d) lets a network of 10,000 weights do the rest, so training takes about a minute, but a dense grid needs millions of numbers. A hash table (e, f) keeps the speed and gives the best image, 24.6 dB.

Müller et al. 2022, Fig. 2. Under each image: network weights + encoding parameters, training time (minutes:seconds) and PSNR.

Looking a point up in grids from coarse to fine

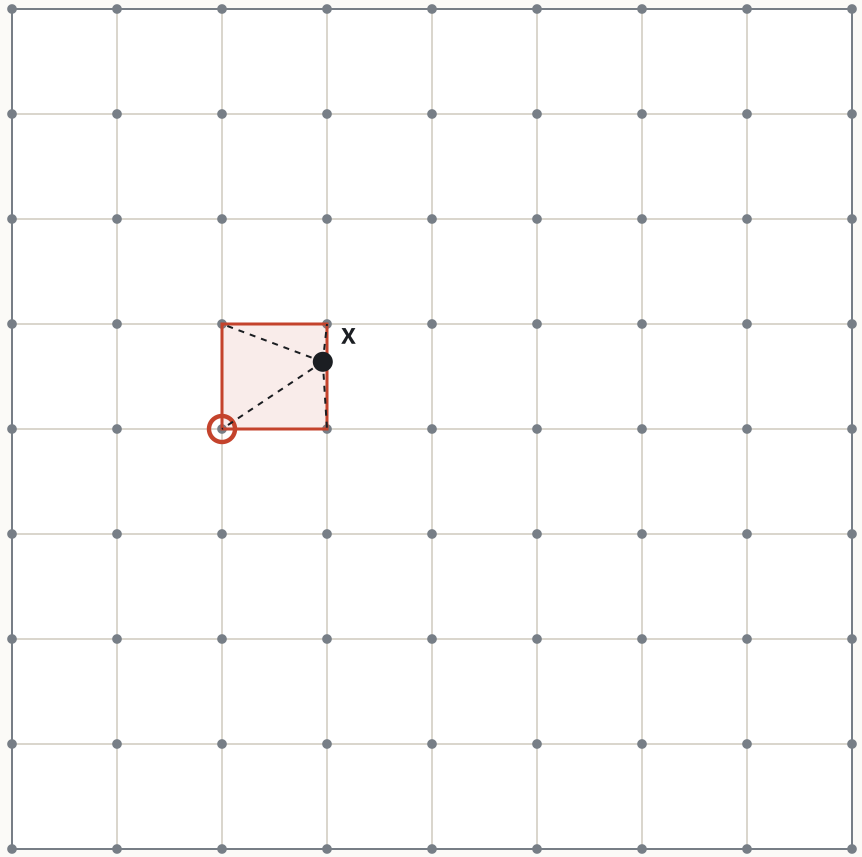


Each of 16 grids, from 16 cells per side to as fine as the photos need, finds the cell around the point (1) and reads the learned vectors stored at its corners (2). They are blended by distance (3), and the 16 results go into a tiny network (4, 5).

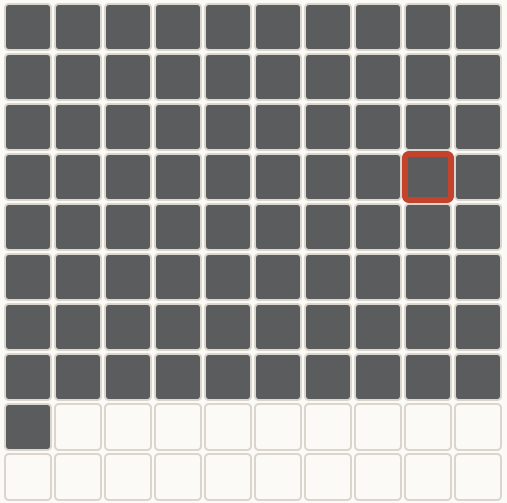
A point touches only 8 corners per grid, so each training step changes only a few table entries. The coarse grids describe the overall shape and the fine ones the detail, like the frequencies of NeRF's encoding, but learned.

Müller et al. 2022, Fig. 3 (drawn in 2D, two levels, red and blue) and Table 1: 16 levels, 2 numbers per entry, coarsest grid 16 cells per side.

Fine grids share one fixed-size table



8 × 8 cells | 16 × 16 cells | 32 × 32 cells

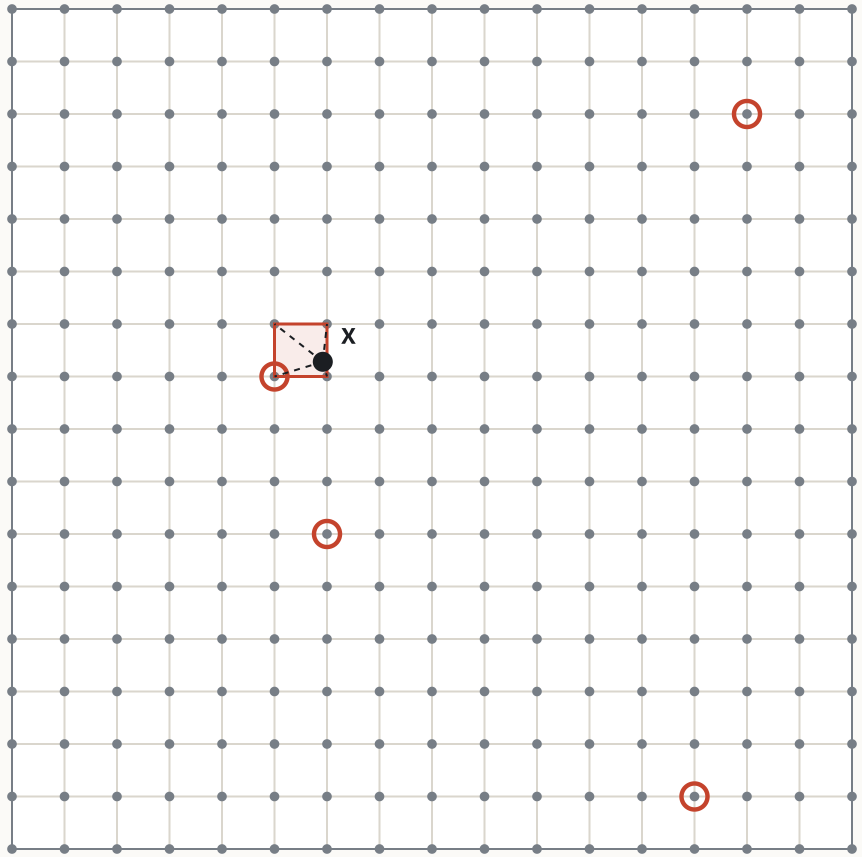


81 corners, **100** table entries.
Every corner has its own entry.

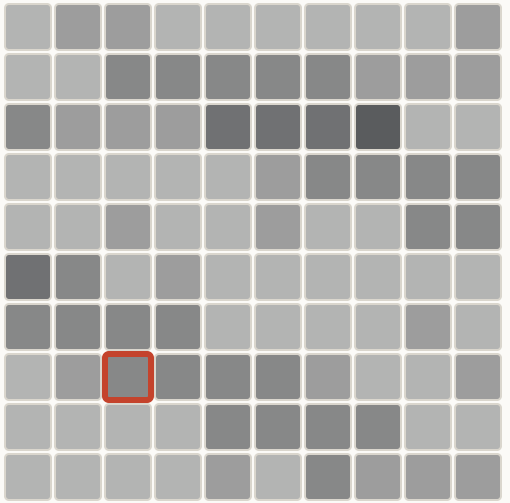
A coarse grid has fewer corners than the table has entries, so every corner simply gets its own entry. The paper does the same for its coarse grids.

Illustration with a table of 100 entries; the paper uses up to 2^{19} = 524,288 entries per grid for NeRF. Hash function of the paper (Eq. 4): corner (x, y) goes to entry ($x \oplus 2{,}654{,}435{,}761 \cdot y$) mod T, where $\oplus$ is bitwise XOR.

Fine grids share one fixed-size table



8 × 8 cells | **16 × 16 cells** | 32 × 32 cells

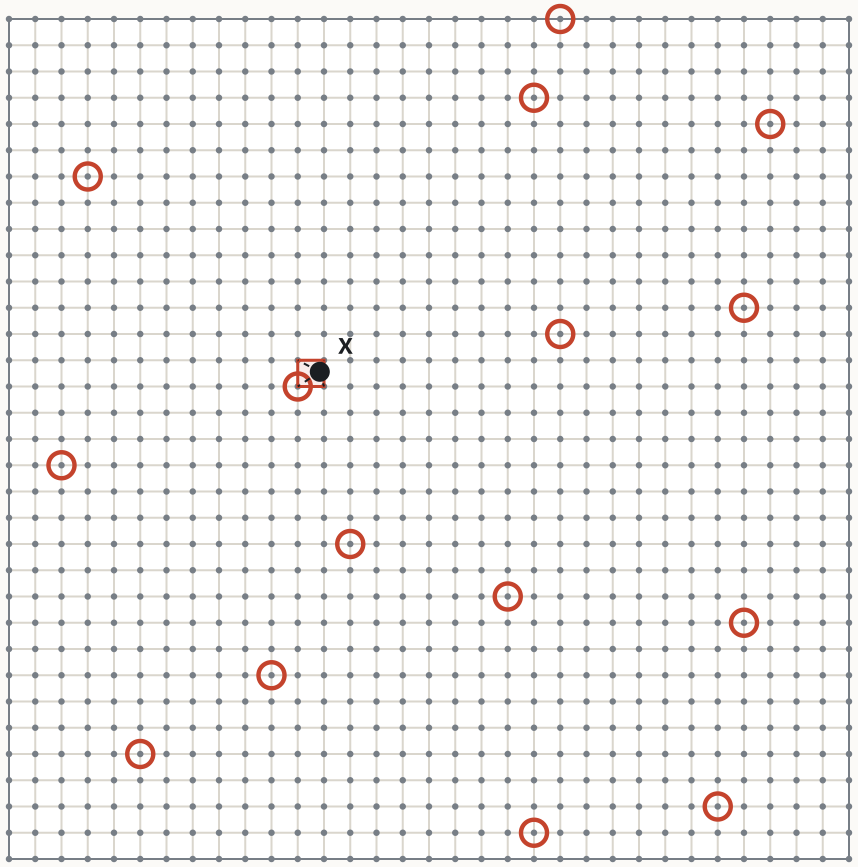


289 corners, **100** table entries.
About **2.9** corners per entry; **4** share the red one.

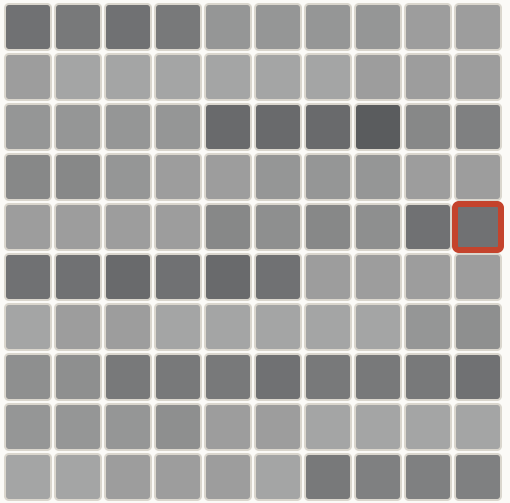
A finer grid has more corners than entries. A hash function scatters them over the table, so several far-apart corners share one entry, circled in red.

Illustration with a table of 100 entries; the paper uses up to 2^{19} = 524,288 entries per grid for NeRF. Hash function of the paper (Eq. 4): corner (x, y) goes to entry $(x \oplus 2{,}654{,}435{,}761 \cdot y) \bmod T$, where $\oplus$ is bitwise XOR.

Fine grids share one fixed-size table



8 × 8 cells | 16 × 16 cells | **32 × 32 cells**

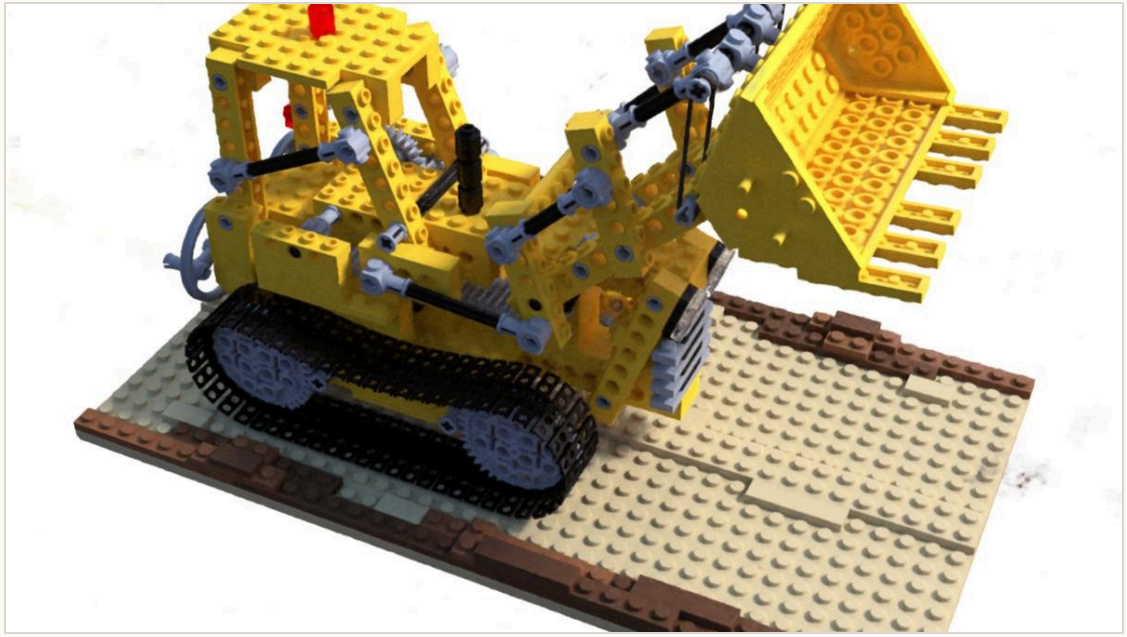


1,089 corners, **100** table entries.
About **10.9** corners per entry; **15** share the red one.

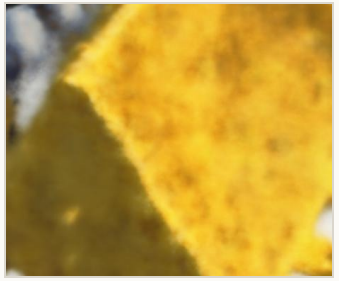
The paper does not resolve these collisions. Training pulls each shared entry mostly towards the corner near a visible surface, and the coarse grids, which never collide, tell the rest apart.

Illustration with a table of 100 entries; the paper uses up to 2^{19} = 524,288 entries per grid for NeRF. Hash function of the paper (Eq. 4): corner (x, y) goes to entry $(x \oplus 2{,}654{,}435{,}761 \cdot y) \bmod T$, where $\oplus$ is bitwise XOR.

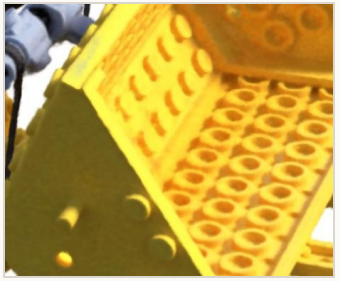
A NeRF scene in seconds instead of days



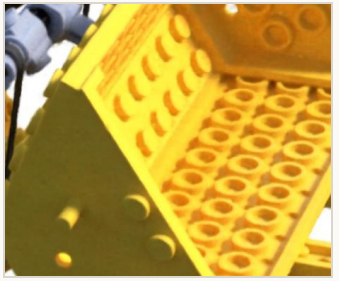
The whole view after 15 seconds of training



1 second



15 seconds



60 seconds

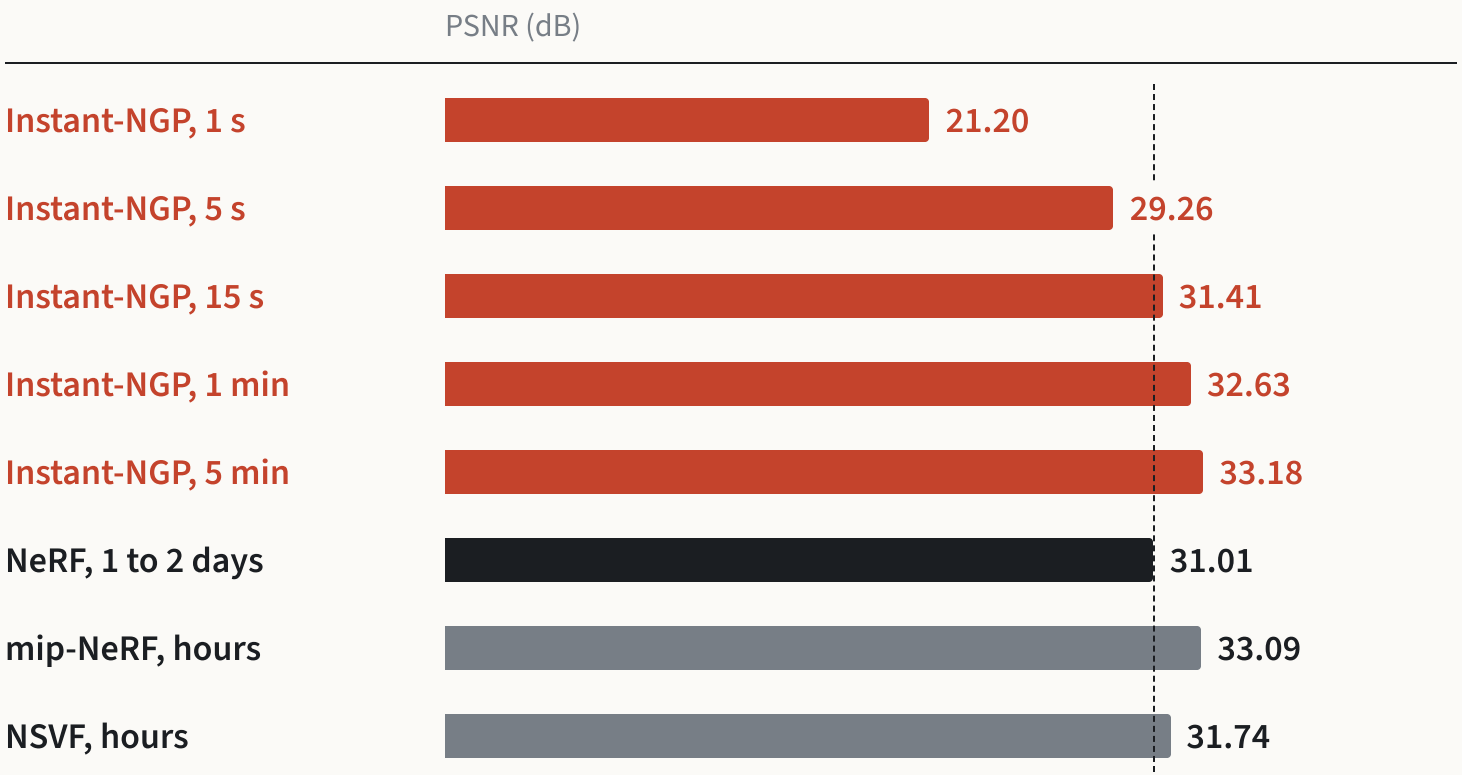
The Lego scene trained from scratch on one RTX 3090 graphics card. After one second the bucket is a yellow blur; after 15 seconds it is sharp, and a minute adds the last details.

NeRF needed one to two days for the same scene on a V100.

Müller et al. 2022, Fig. 1 (NeRF row): a zoom on the bucket of the Lego scene at three training times. NeRF: Mildenhall et al. 2020, Sec. 5.3.

Five minutes of training beat hours of NeRF

Training time | Same code, NeRF's encoding



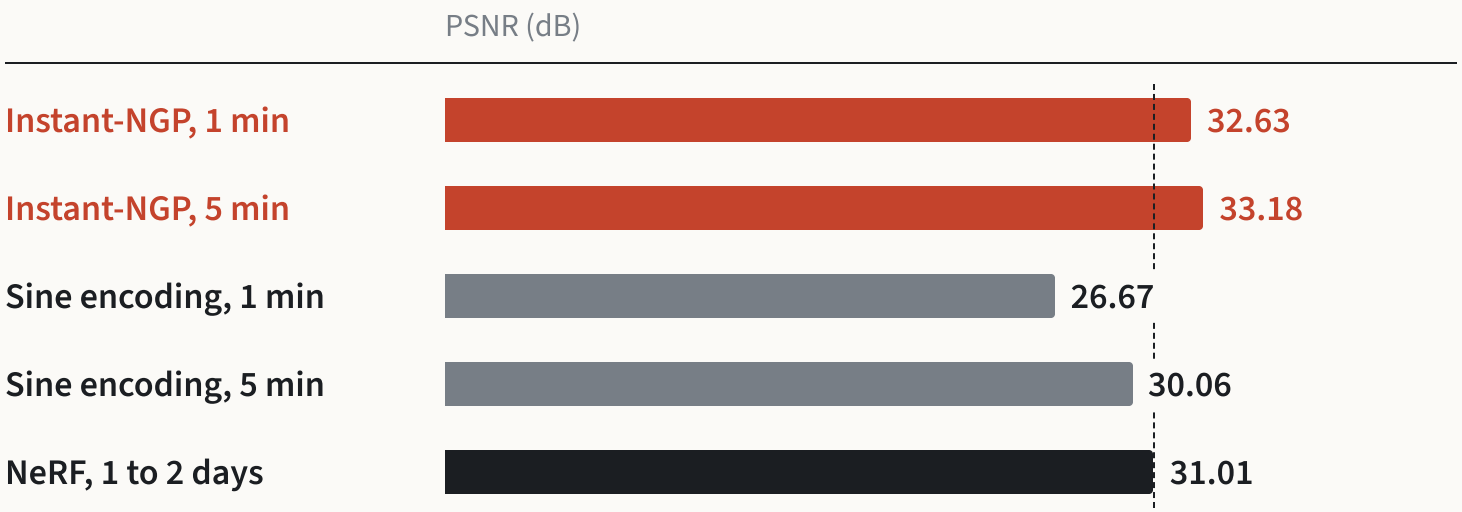
After 15 seconds of training, the average over the eight synthetic scenes passes NeRF's 31.0 dB, which took one to two days.

After five minutes it matches mip-NeRF, the best method of the time, which also trains for hours.

Müller et al. 2022, Table 2: mean PSNR over the eight Realistic Synthetic 360° scenes from the NeRF paper. NeRF, mip-NeRF and NSVF values from their papers; the dashed line is NeRF, 31.01 dB.

Five minutes of training beat hours of NeRF

Training time | **Same code, NeRF's encoding**

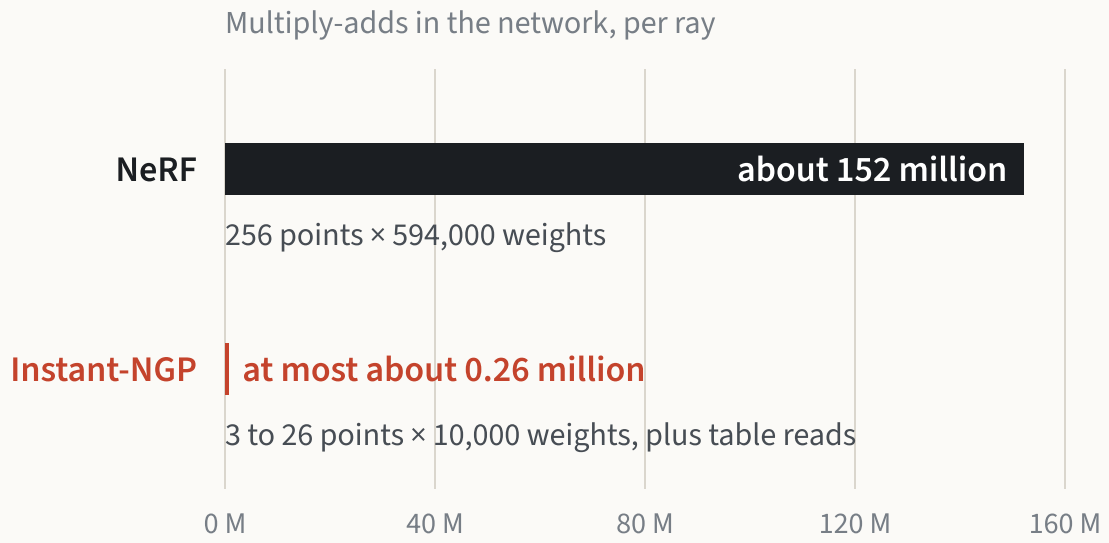


Is it the code or the encoding? In the same fast code with NeRF's sine encoding and big network, five minutes give only 30.1 dB.

The authors credit the hash encoding itself with a 20 to 60 times speed-up.

Müller et al. 2022, Table 2: mean PSNR over the eight Realistic Synthetic 360° scenes from the NeRF paper. NeRF, mip-NeRF and NSVF values from their papers; the dashed line is NeRF, 31.01 dB.

Why it is so much faster, and what it costs



NeRF sends 256 points of every ray through a network of about 600,000 weights. Instant-NGP keeps a coarse grid of occupied cells, so a ray needs only 3 to 26 points, and each point costs a few table reads and a network of 10,000 weights.

That is about 600 times less arithmetic per ray.

Computed from the papers: NeRF evaluates 64 + 192 points per ray with a network of about 594,000 weights (Mildenhall et al. 2020, Fig. 7, Sec. 5.3). Instant-NGP uses 3.1 to 25.7 points per ray (Müller et al. 2022, Table 3) and a network of about 10,000 weights (Fig. 2); 12.6 million table entries with $T = 2^{19}$ (Fig. 2f).

Why it is so much faster, and what it costs

1.2 million
NeRF, weights

12.6 million
Instant-NGP, table entries and weights

The speed is paid for in memory: 12.6 million learned table entries, ten times NeRF's weights.

It also relies on hand-written GPU code that keeps the whole tiny network in the chip's fastest memory, and it still renders by marching rays.

Computed from the papers: NeRF evaluates 64 + 192 points per ray with a network of about 594,000 weights (Mildenhall et al. 2020, Fig. 7, Sec. 5.3). Instant-NGP uses 3.1 to 25.7 points per ray (Müller et al. 2022, Table 3) and a network of about 10,000 weights (Fig. 2); 12.6 million table entries with $T = 2^{19}$ (Fig. 2f).

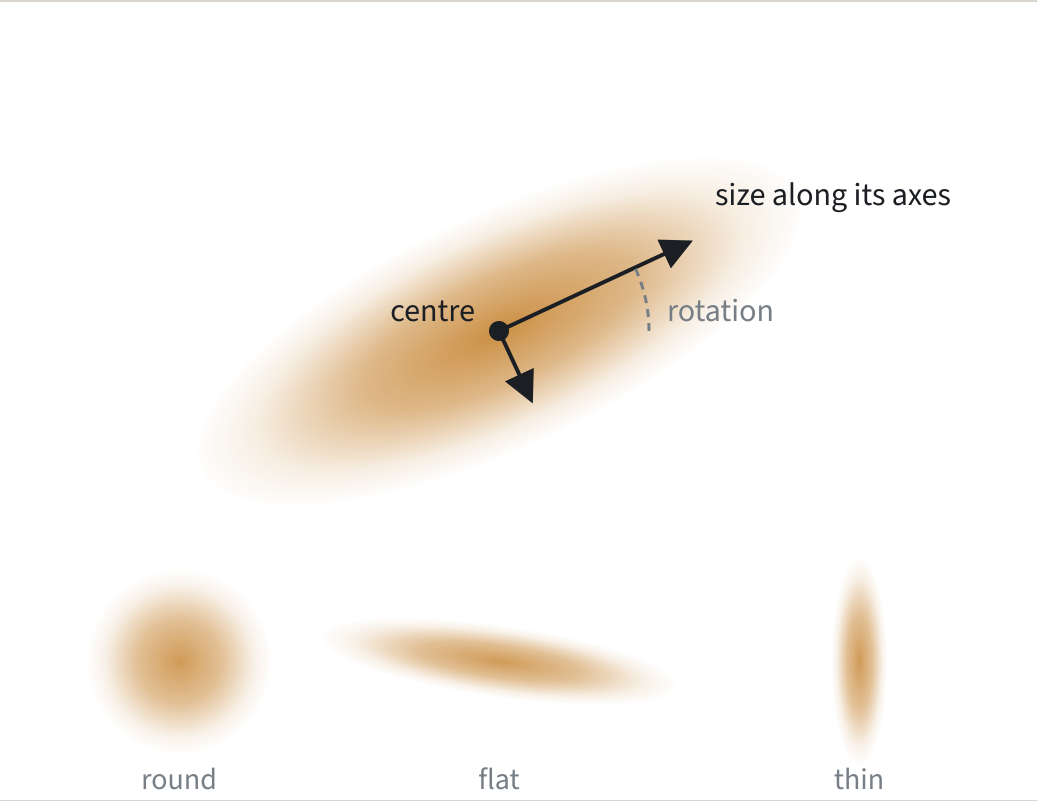
3D Gaussian splatting: no network, no ray marching

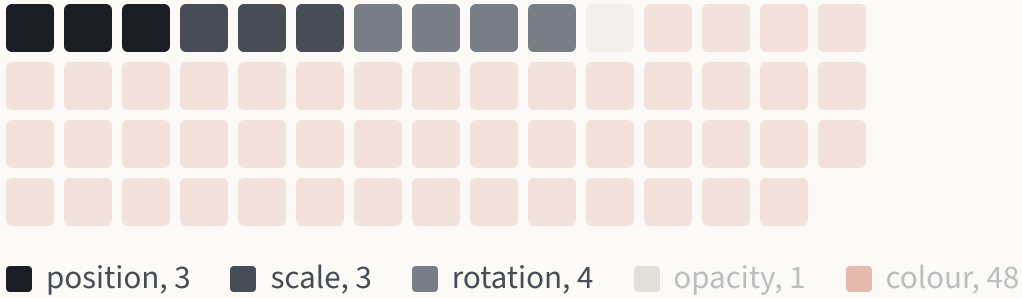
The scene becomes a cloud of a few million small, coloured, see-through blobs called 3D Gaussians. An image is made by projecting every blob onto the screen and blending them, the way a graphics card draws triangles.

On this view, after 6 minutes of training it renders 135 frames per second. After 51 minutes it beats Mip-NeRF 360, which trained for 48 hours and renders one frame every 14 seconds.

Kerbl et al. 2023, Fig. 1: a test view of the Bicycle scene; frame rates and training times on one A6000 GPU. Mip-NeRF 360 (Barron et al. 2022) was the best-quality NeRF variant at the time; Plenoxels (Fridovich-Keil and Yu et al. 2022) is a sparse voxel grid without a network.

What one Gaussian stores



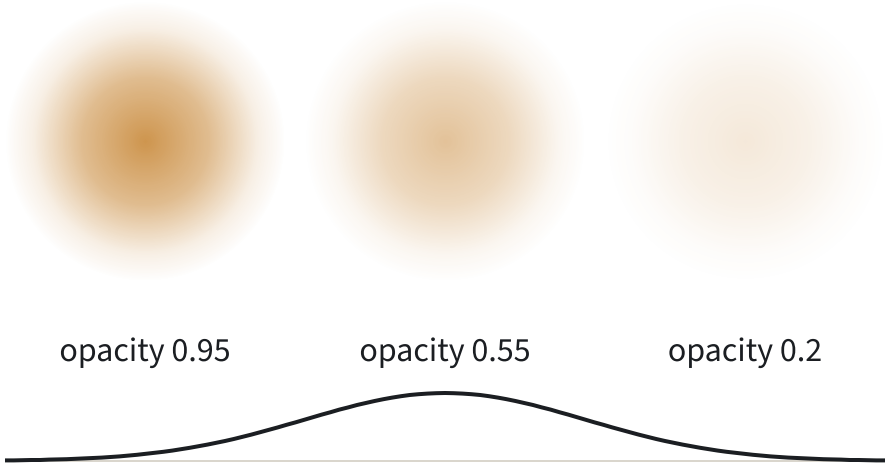


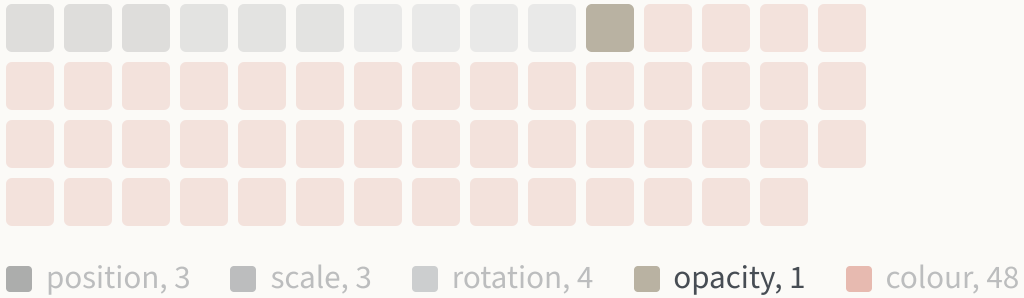
A Gaussian is a soft ellipsoid. Its centre is 3 numbers. Its shape is 3 sizes along its own axes and a rotation of 4 numbers, so it can be a round blob, a flat disc or a thin needle.

> 59 numbers per Gaussian, and 1 to 5 million Gaussians per scene.

Kerbl et al. 2023, Sec. 4: the shape is a covariance $\Sigma = R\,S\,S^{\mathsf{T}}\,R^{\mathsf{T}}$, stored as a scale vector and a rotation quaternion; colour as spherical harmonics up to degree 3 (16 per channel).

What one Gaussian stores



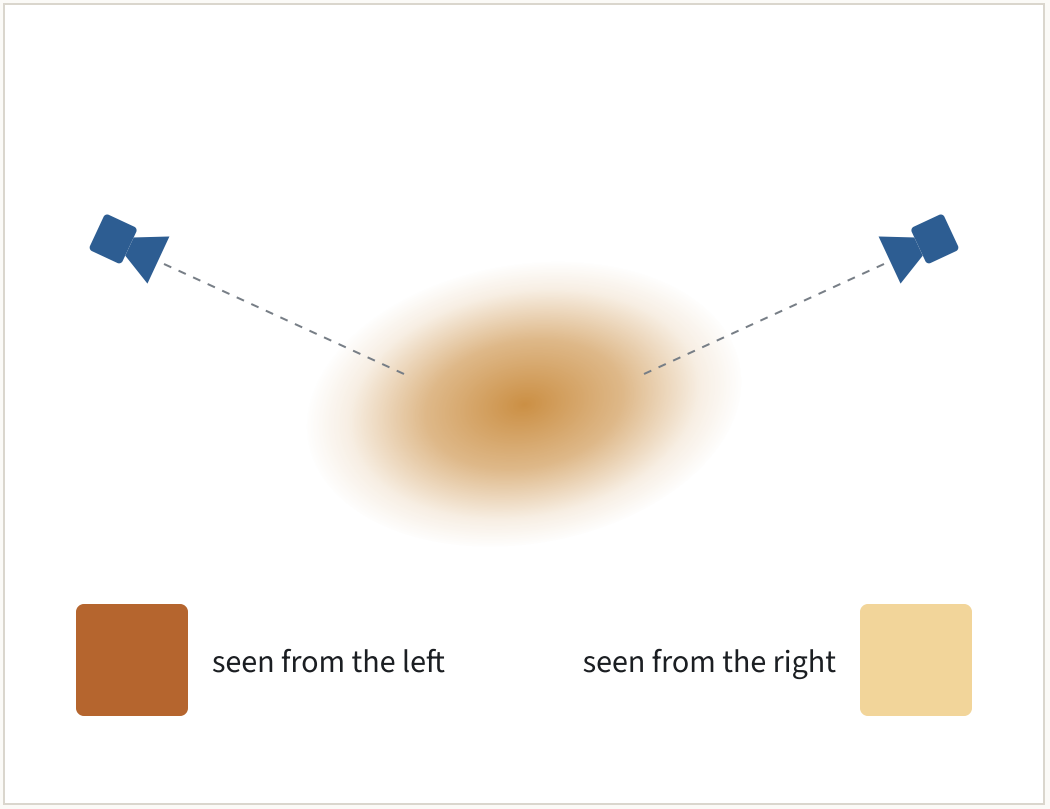


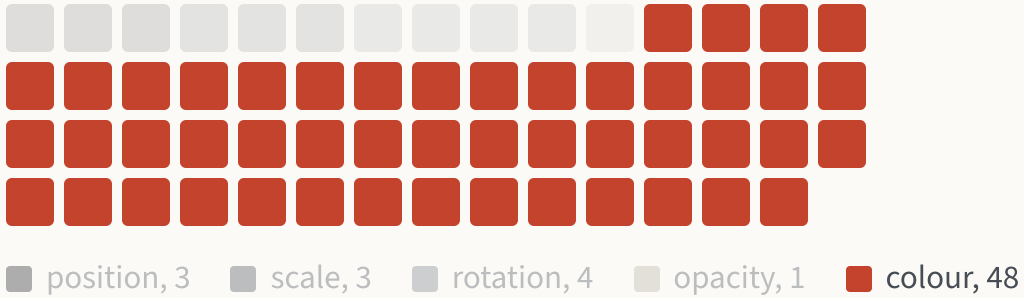
One number says how much light it blocks at its centre. The blocking fades smoothly towards the edges, so neighbouring Gaussians blend into each other.

> 59 numbers per Gaussian, and 1 to 5 million Gaussians per scene.

Kerbl et al. 2023, Sec. 4: the shape is a covariance $\Sigma = R\,S\,S^{\mathsf{T}}\,R^{\mathsf{T}}$, stored as a scale vector and a rotation quaternion; colour as spherical harmonics up to degree 3 (16 per channel).

What one Gaussian stores



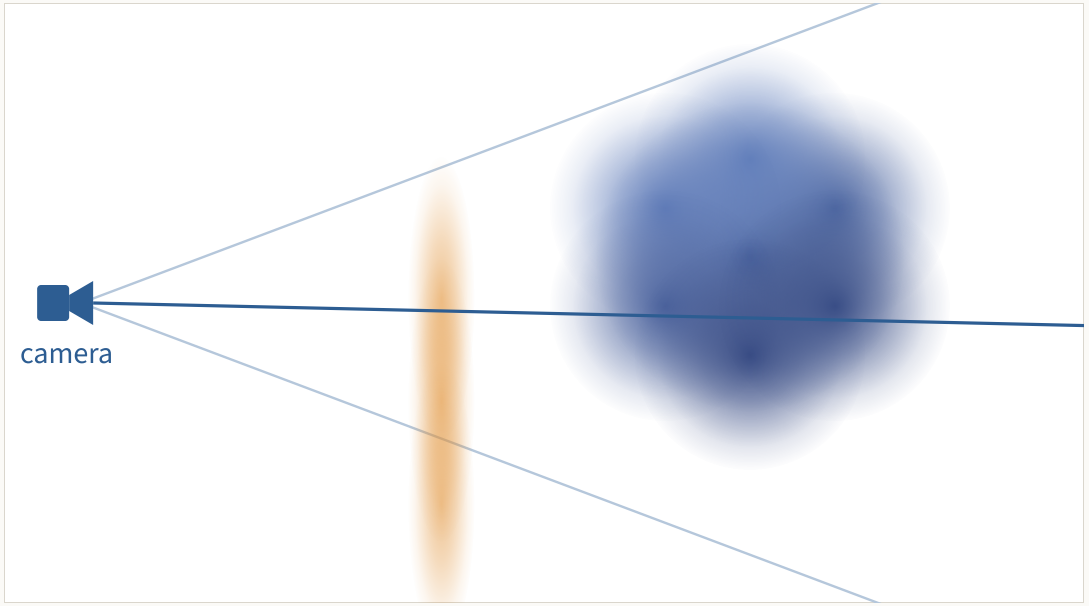


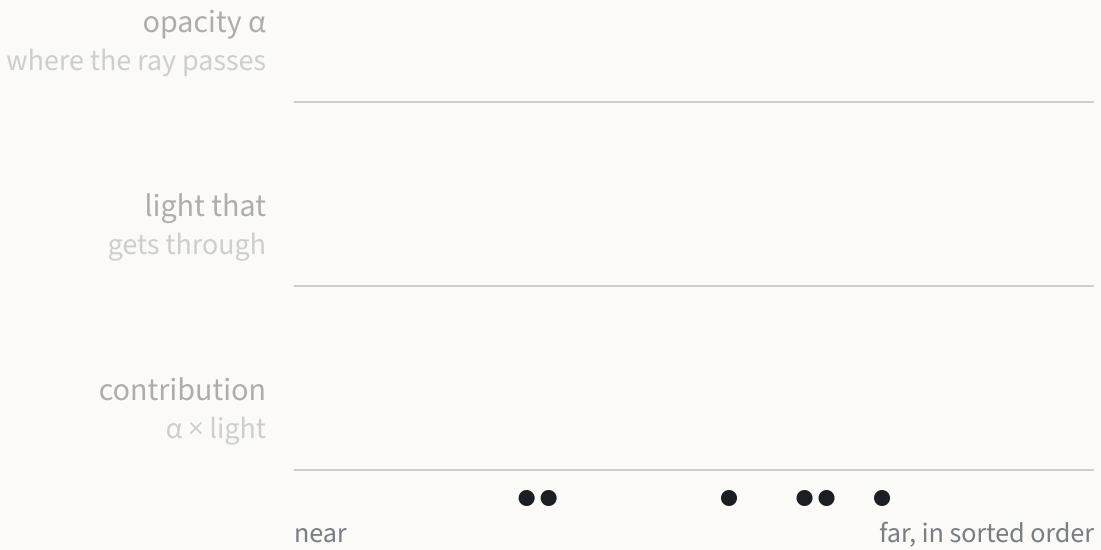
Its colour can change with the viewing direction, as in NeRF. It is stored as 16 spherical-harmonic coefficients per colour channel: 48 numbers, more than everything else together.

> 59 numbers per Gaussian, and 1 to 5 million Gaussians per scene.

Kerbl et al. 2023, Sec. 4: the shape is a covariance $\Sigma = R\,S\,S^{\mathsf{T}}\,R^{\mathsf{T}}$, stored as a scale vector and a rotation quaternion; colour as spherical harmonics up to degree 3 (16 per channel).

Rendering: sort the blobs, then blend them





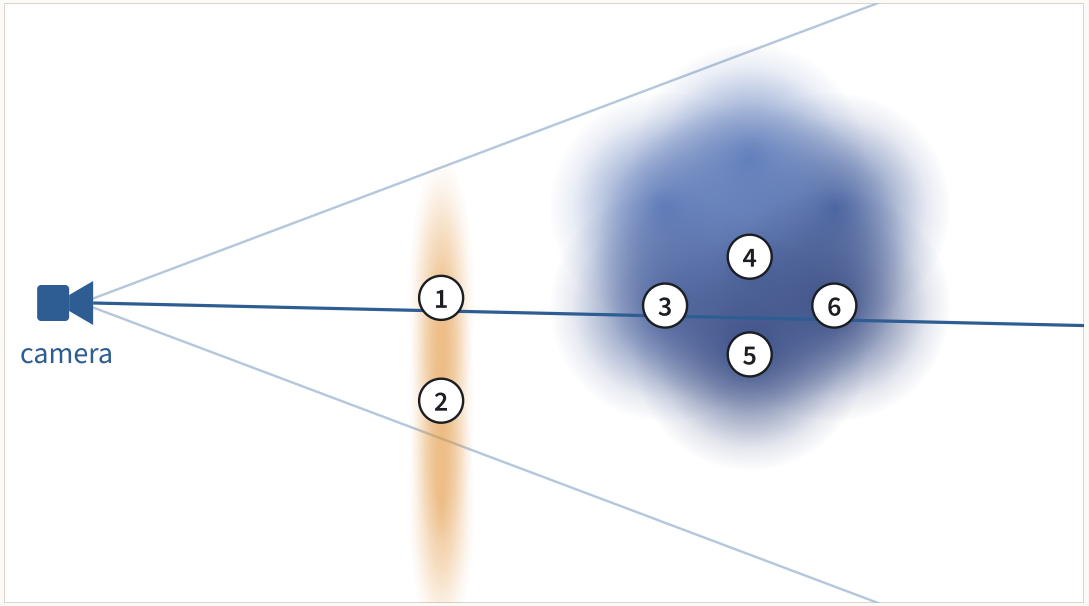
rendered

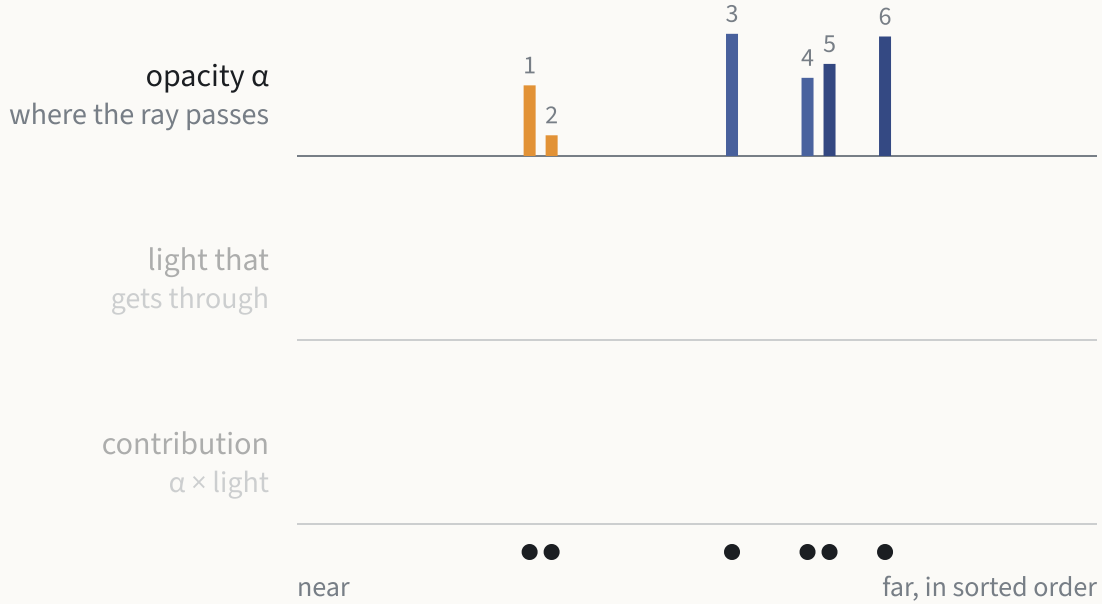
$$C = \Sigma_i \, c_i \, \alpha_i \, \Pi_{j<i} \, (1 - \alpha_j)$$

The toy scene again, now stored as nine Gaussians instead of a network: three thin orange ones for the pane and six blue ones for the ball. The pixel's ray crosses a few of them.

Kerbl et al. 2023, Sec. 2 and 6: the blending sum is NeRF's Eq. 3 with $\alpha_i = 1 - e^{-\sigma_i \delta_i}$; sorting is done once per 16 × 16-pixel tile. Toy scene as in the NeRF part, made of 9 Gaussians; each Gaussian's α is taken where the ray passes closest to its centre.

Rendering: sort the blobs, then blend them



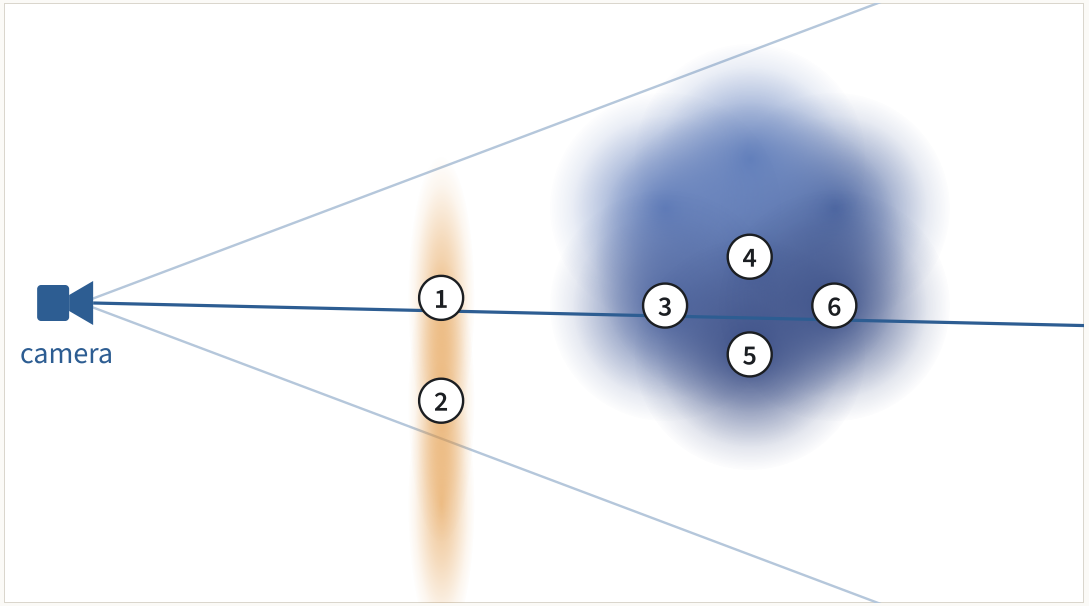


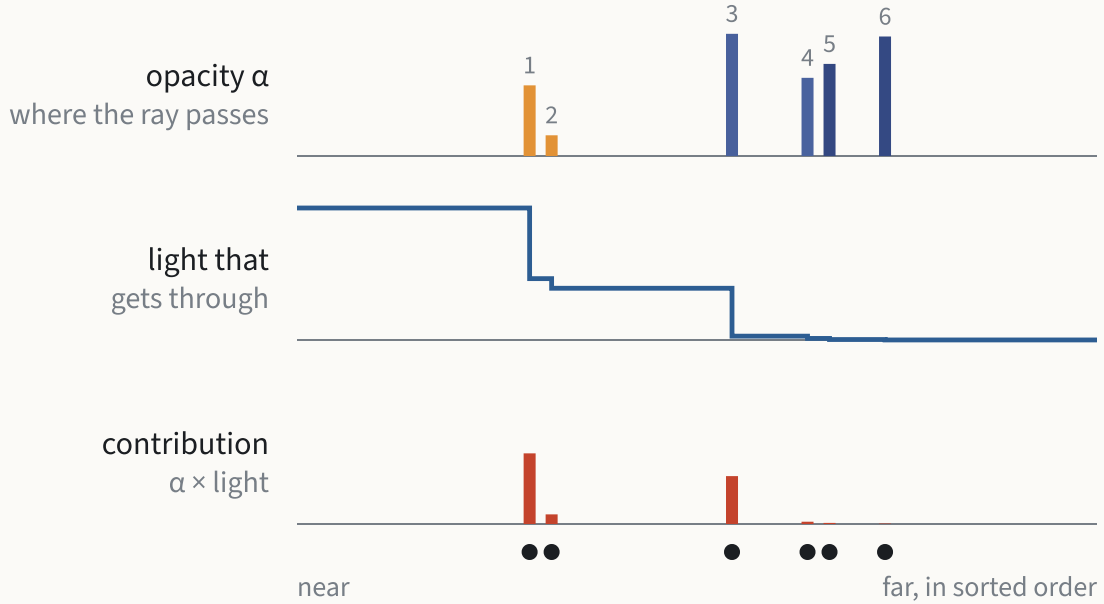
$$C = \Sigma_i \, c_i \, \alpha_i \, \Pi_{j<i} \, (1 - \alpha_j)$$

The Gaussians are sorted by distance from the camera, once for each 16 × 16-pixel tile rather than point by point along each ray. Each one blocks most light where the ray passes close to its centre.

Kerbl et al. 2023, Sec. 2 and 6: the blending sum is NeRF's Eq. 3 with $\alpha_i = 1 - e^{-\sigma_i \delta_i}$; sorting is done once per 16 × 16-pixel tile. Toy scene as in the NeRF part, made of 9 Gaussians; each Gaussian's α is taken where the ray passes closest to its centre.

Rendering: sort the blobs, then blend them





rendered

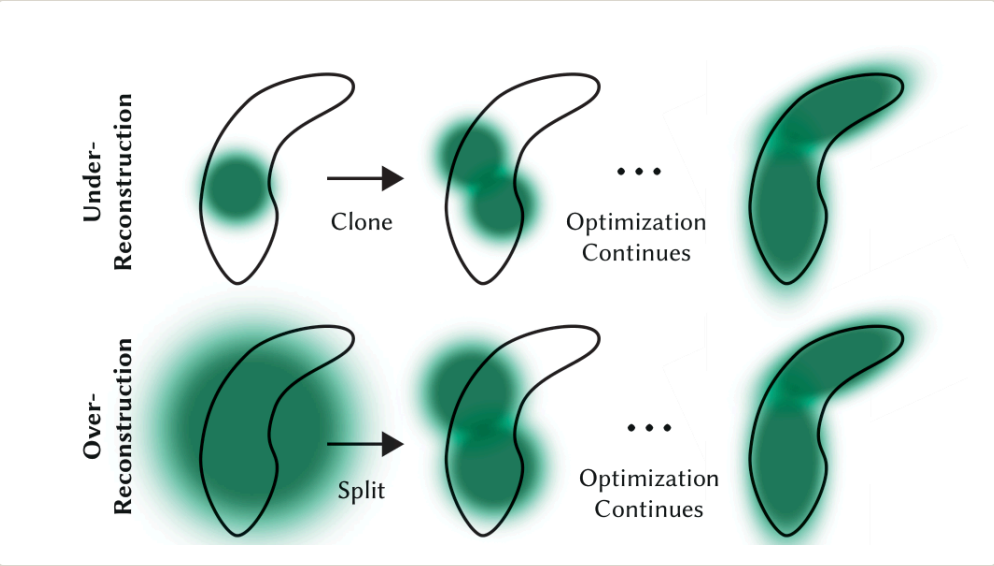
$$C = \Sigma_i \, c_i \, \alpha_i \, \Pi_{j<i} \, (1 - \alpha_j)$$

pixel

Blending front to back is the same sum as NeRF's: colour, times opacity, times the light still getting through. Nothing is queried; every term is read straight from a Gaussian.

Kerbl et al. 2023, Sec. 2 and 6: the blending sum is NeRF's Eq. 3 with $\alpha_i = 1 - e^{-\sigma_i \delta_i}$; sorting is done once per 16 × 16-pixel tile. Toy scene as in the NeRF part, made of 9 Gaussians; each Gaussian's α is taken where the ray passes closest to its centre.

Training moves, clones, splits and removes Gaussians



Training starts from the 3D points COLMAP found while computing the camera positions, each one a small round Gaussian. Rendered images are compared with the photos, as in NeRF, and every number of every Gaussian is adjusted by gradient descent.

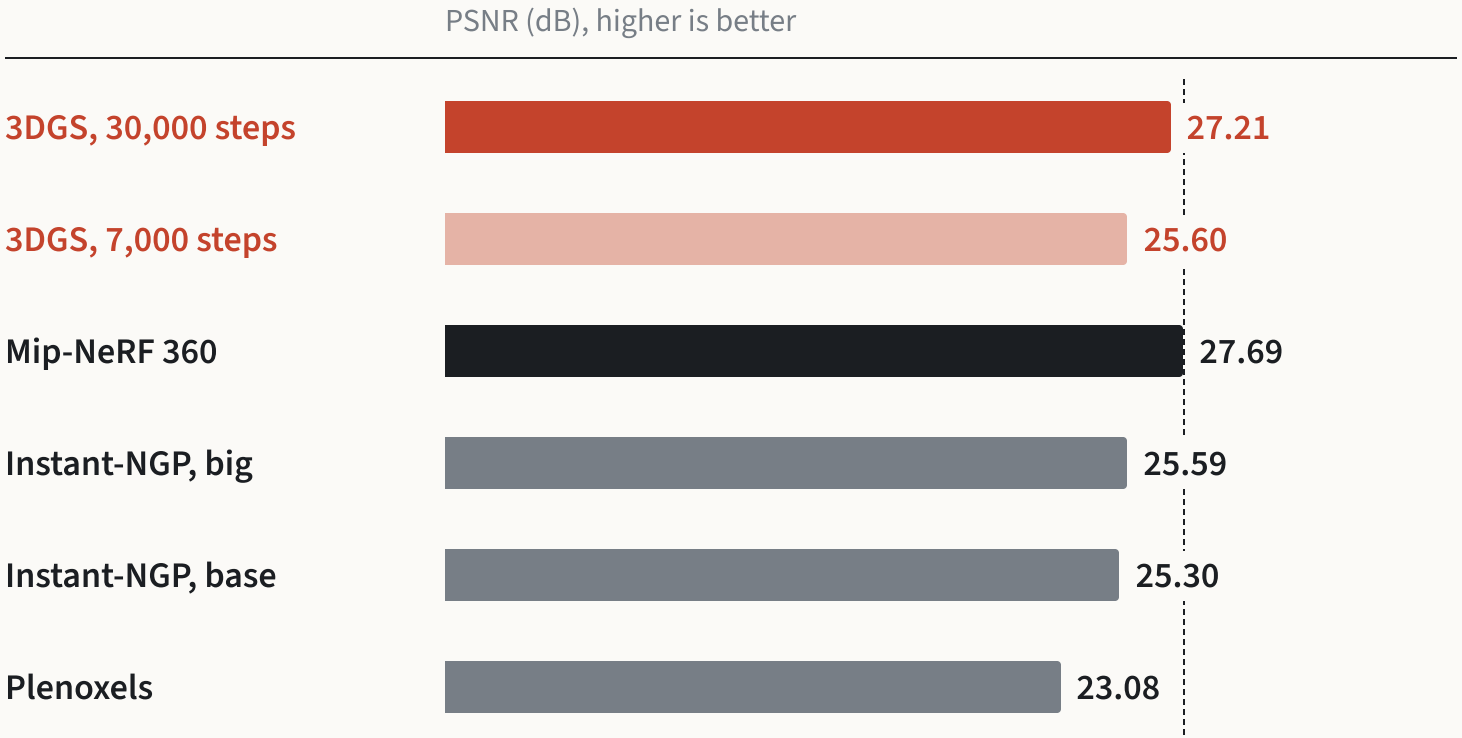
Every 100 steps the set itself changes. Where detail is missing, a small Gaussian is cloned; where one large Gaussian covers fine detail, it is split in two; nearly transparent ones are removed.

After 30,000 steps a scene has 1 to 5 million Gaussians.

Kerbl et al. 2023, Fig. 4 and Sec. 5. The loss is 0.8 × L1 + 0.2 × D-SSIM between the render and the photo. Cloning and splitting are triggered by large position gradients.

NeRF-level quality, rendered in real time

Quality | Training time | Frames per second | Size on disk

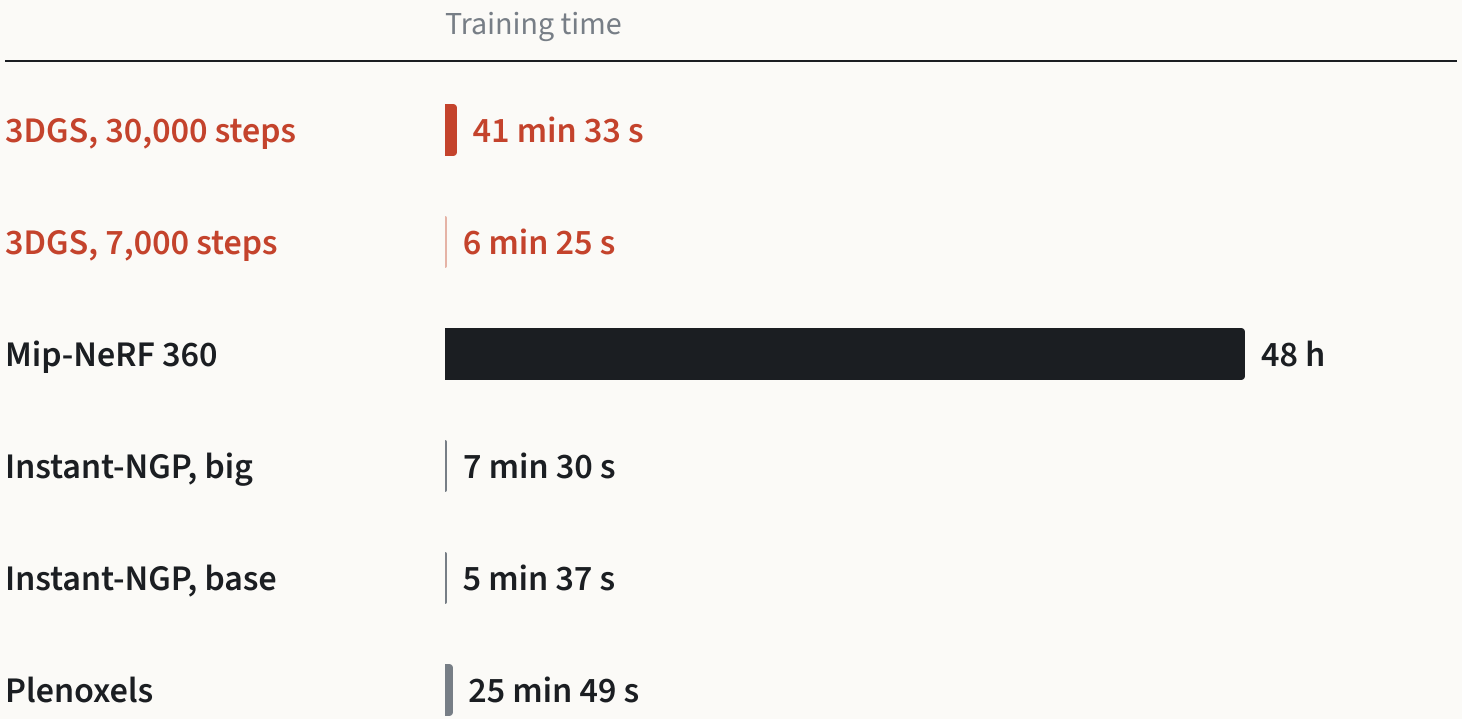


On nine large real scenes, Gaussian splatting reaches 27.2 dB after 30,000 steps, close to Mip-NeRF 360's 27.7 and above both fast methods.

Kerbl et al. 2023, Table 1: mean over the nine Mip-NeRF 360 scenes, one A6000 GPU. Mip-NeRF 360's numbers are from its own paper (Barron et al. 2022); the others are the authors' runs.

NeRF-level quality, rendered in real time

Quality | **Training time** | Frames per second | Size on disk

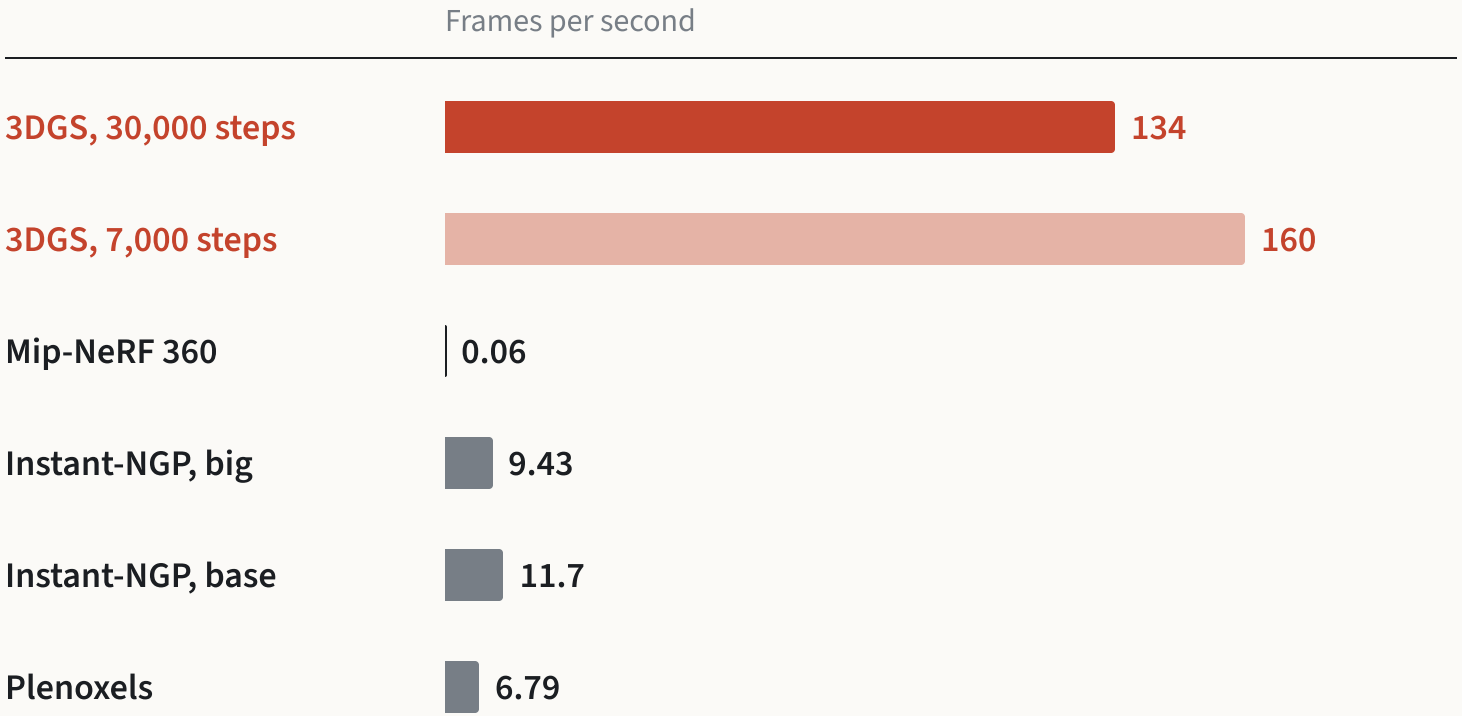


It trains in 42 minutes, against 48 hours for Mip-NeRF 360. After 7,000 steps, about 6 minutes, it already matches Instant-NGP.

Kerbl et al. 2023, Table 1: mean over the nine Mip-NeRF 360 scenes, one A6000 GPU. Mip-NeRF 360's numbers are from its own paper (Barron et al. 2022); the others are the authors' runs.

NeRF-level quality, rendered in real time

Quality | Training time | **Frames per second** | Size on disk

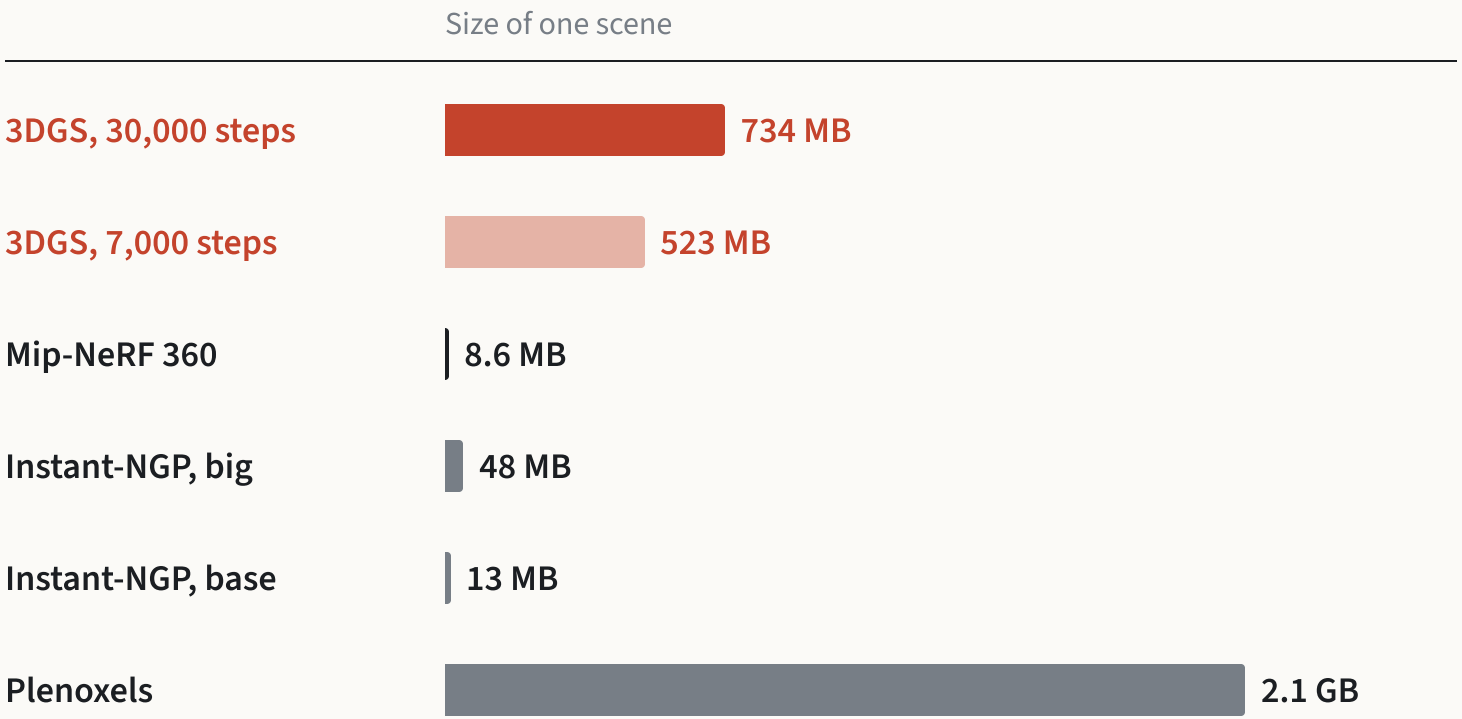


Here it changes everything: 134 frames per second, against 0.06 for Mip-NeRF 360 and about 10 for Instant-NGP.

Kerbl et al. 2023, Table 1: mean over the nine Mip-NeRF 360 scenes, one A6000 GPU. Mip-NeRF 360's numbers are from its own paper (Barron et al. 2022); the others are the authors' runs.

NeRF-level quality, rendered in real time

Quality | Training time | Frames per second | **Size on disk**



The price is size: 734 MB per scene, against 8.6 MB for Mip-NeRF 360 and 13 to 48 MB for Instant-NGP. Plenoxels' voxel grid is even larger.

Kerbl et al. 2023, Table 1: mean over the nine Mip-NeRF 360 scenes, one A6000 GPU. Mip-NeRF 360's numbers are from its own paper (Barron et al. 2022); the others are the authors' runs.

What the difference looks like on real scenes

Bicycle | Garden | Stump | Dr Johnson

Photo

Gaussian splatting

Mip-NeRF 360

Instant-NGP

Plenoxels

Instant-NGP leaves dark floaters on the path, and Plenoxels blurs the grass. Gaussian splatting and Mip-NeRF 360 both stay close to the photo.

Kerbl et al. 2023, Fig. 5: held-out test views, images taken from the paper at full resolution. Bicycle, Garden and Stump are from the Mip-NeRF 360 dataset, Dr Johnson from Deep Blending (Hedman et al. 2018).

What the difference looks like on real scenes

Bicycle | **Garden** | Stump | Dr Johnson

Photo

Gaussian splatting

Mip-NeRF 360

Instant-NGP

Plenoxels

All methods are close on this scene; the fast ones lose some sharpness in the plants and houses at the back.

Kerbl et al. 2023, Fig. 5: held-out test views, images taken from the paper at full resolution. Bicycle, Garden and Stump are from the Mip-NeRF 360 dataset, Dr Johnson from Deep Blending (Hedman et al. 2018).

What the difference looks like on real scenes

Bicycle | Garden | **Stump** | Dr Johnson

Photo

Gaussian splatting

Mip-NeRF 360

Instant-NGP

Plenoxels

Instant-NGP fills the background with ghostly blobs and Plenoxels with coloured smears. Gaussian splatting stays close to the photo.

Kerbl et al. 2023, Fig. 5: held-out test views, images taken from the paper at full resolution. Bicycle, Garden and Stump are from the Mip-NeRF 360 dataset, Dr Johnson from Deep Blending (Hedman et al. 2018).

What the difference looks like on real scenes

Bicycle | Garden | Stump | **Dr Johnson**

Photo

Gaussian splatting

Mip-NeRF 360

Instant-NGP

Plenoxels

On the plain white walls of this indoor scene, Instant-NGP leaves grainy dark noise and Plenoxels cloudy patches. Gaussian splatting stays clean.

Kerbl et al. 2023, Fig. 5: held-out test views, images taken from the paper at full resolution. Bicycle, Garden and Stump are from the Mip-NeRF 360 dataset, Dr Johnson from Deep Blending (Hedman et al. 2018).

What Gaussian splatting costs

8.6 MB
Mip-NeRF 360

48 MB
Instant-NGP, big

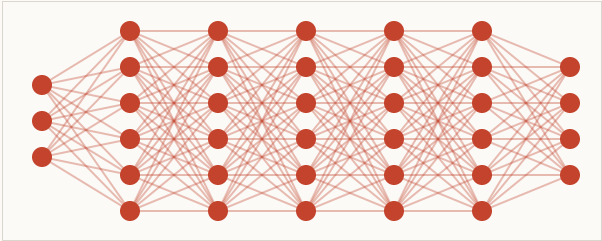
734 MB
Gaussian splatting

Every Gaussian keeps 59 numbers and a scene has millions of them: 734 MB on average for the Mip-NeRF 360 scenes, 85 times the size of Mip-NeRF 360's network. Training a large scene can need more than 20 GB of GPU memory.

Where the photos see little, Gaussians become stretched or blotchy, and large ones can pop in and out as the camera moves. The method also needs good COLMAP points to start from.

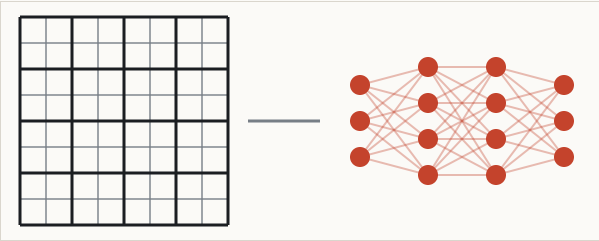
Kerbl et al. 2023, Table 1 (mean size per scene, Mip-NeRF 360 dataset) and Sec. 7.4 (limitations). Squares are sized by area.

Three ways to store the same scene



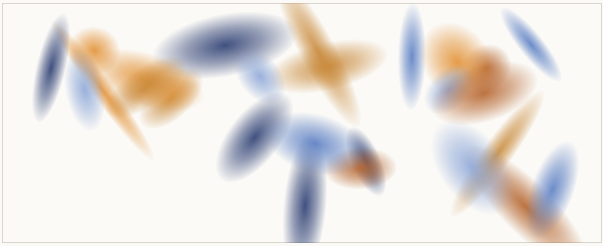
NeRF, 2020

Two networks with 1.2 million weights, 5 MB. One to two days of training on a V100, and about 30 seconds per image, because every pixel queries the network 256 times.



Instant-NGP, 2022

16 hashed grids with 12.6 million learned entries and a network of 10,000 weights. Seconds to minutes of training on an RTX 3090, still rendered by marching rays.



3D Gaussian splatting, 2023

One to five million Gaussians of 59 numbers each, about 734 MB. About 40 minutes of training on an A6000, and over 100 frames per second.

> All three render with the same sum along a ray; they differ in where the density and colour come from.

Numbers from each paper as shown on the previous slides; the GPUs and test scenes differ between papers, so compare orders of magnitude, not exact values.

What else NeRF leaves open

Blur at other distances	One thin ray per pixel misses detail when the camera is closer or further than in training. Mip-NeRF (Barron et al. 2021) traces cones instead, and Mip-NeRF 360 (Barron et al. 2022) extends it to large outdoor scenes.
One network per scene	Nothing is learned across scenes, so each needs many photos. pixelNeRF (Yu et al. 2021) learns a prior from many scenes and works from one or a few photos.
A still scene, fixed light	Moving objects and changing lighting break the assumption that every photo shows the same scene. NeRF in the Wild (Martin-Brualla et al. 2021) handles tourist photos taken at different times.
Medical imaging	CT already measures density along rays. NAF (Zha et al. 2022) keeps only NeRF's density and reconstructs cone-beam CT from few X-ray projections.

Limits from Mildenhall et al. 2020, Sec. 7. Follow-up work as cited; see the references.

My own capture

I filmed a gemstone globe on a brass stand with a phone, walking around it at four heights, to train the paper's model on a real object of my own.

Then I trained 3D Gaussian splatting on the same photos and compared the two.

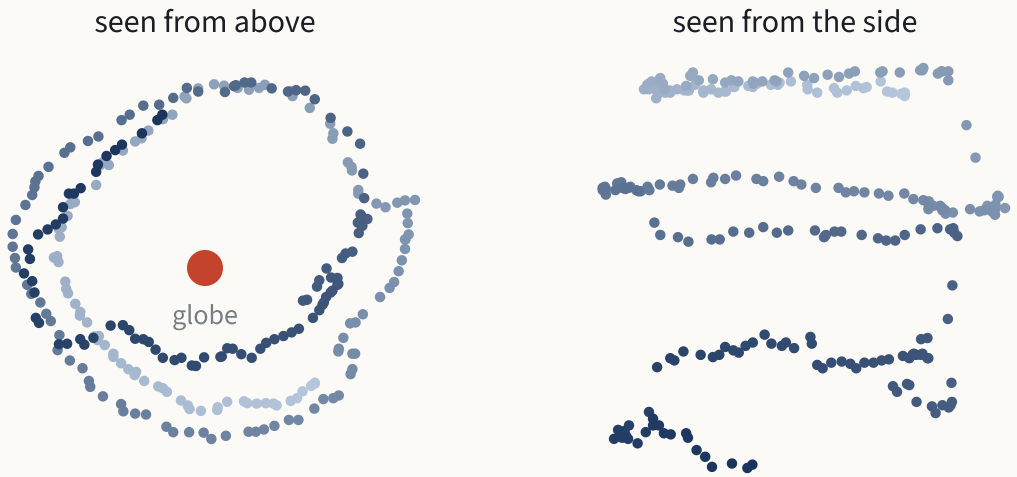
From a phone video to photos with known cameras

0 s

63 s

130 s

157 s



each dot is one camera, light at the start of the video and dark at the end

A 2 min 54 s video at 30 frames per second. I kept the sharpest frame of every ten and gave every second one, 257 frames, to COLMAP at 1080 × 607 pixels.

COLMAP recovered the position and direction of 255 of the 257 cameras, with an average error of 0.58 pixels. This is the input NeRF needs.

My capture: iPhone 17, Blackmagic Camera app, 1/120 s shutter. Camera positions from COLMAP 4.2 (pycolmap), one shared SIMPLE_RADIAL camera; the error is the mean reprojection error.

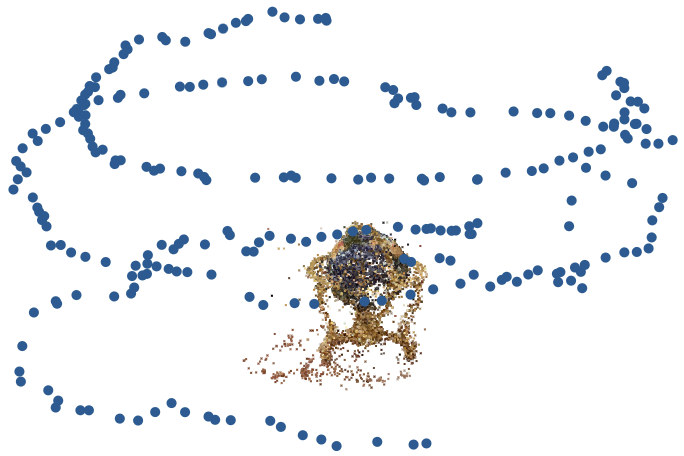
The 3D points COLMAP found on my globe

drag to turn

Besides the camera positions, COLMAP triangulated 41,128 points of the room, each with a colour taken from the photos. 28,184 of them lie on the globe, its stand and the table, shown here.

My capture: COLMAP sparse model of the 257 frames (mean reprojection error 0.58 px). 16,000 of the 28,184 points within 1.2 units of the globe are drawn; the view is orthographic.

The 3D points COLMAP found on my globe



drag to turn

Besides the camera positions, COLMAP triangulated 41,128 points of the room, each with a colour taken from the photos. 28,184 of them lie on the globe, its stand and the table, shown here.

These points are where 3D Gaussian splatting starts: each becomes one small Gaussian before training moves, clones and splits them. NeRF ignores the points and uses only the cameras, shown in blue.

My capture: COLMAP sparse model of the 257 frames (mean reprojection error 0.58 px). 16,000 of the 28,184 points within 1.2 units of the globe are drawn; the view is orthographic.

Keeping only the globe, not my room

The photo, as COLMAP sees it

The mask from SAM 2, room darkened

Neither NeRF nor Gaussian splatting knows what an object is; they rebuild whatever the photos show. COLMAP still gets the full photos, because the room gives it far more to match. To cut out the globe, SAM 2, a video segmentation model, followed it through all 255 frames from one box and two clicks on the first.

My capture, frame 1868 of the video. Masks: SAM 2.1 Hiera-L (Ravi et al. 2024) through Hugging Face transformers, prompted with a box and two points in the first frame, then propagated. White background as in the paper's synthetic scenes.

Keeping only the globe, not my room

The photo, as COLMAP sees it

The mask from SAM 2, room darkened

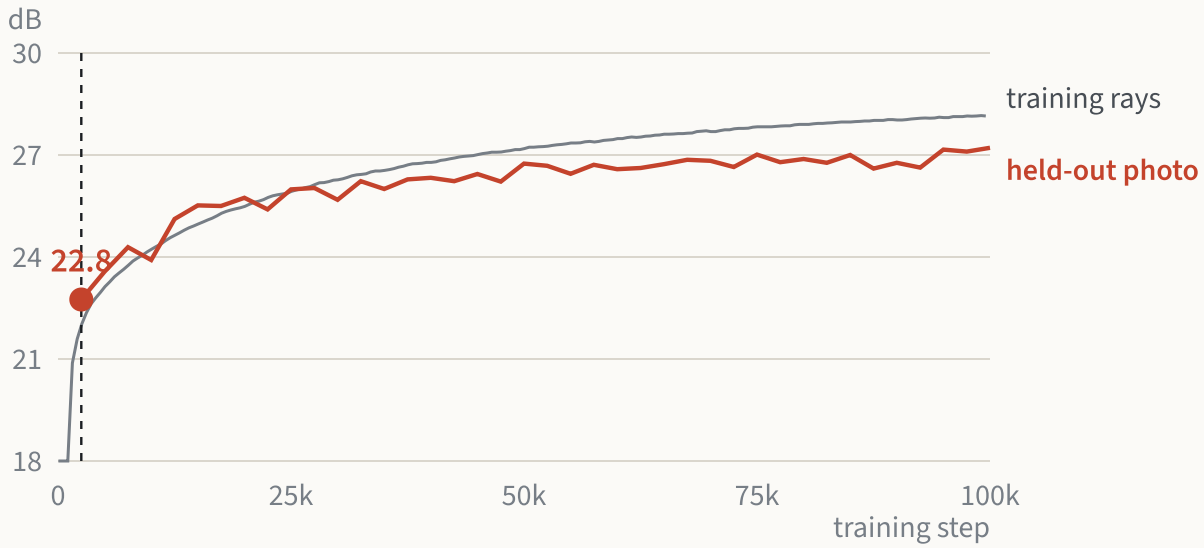
What NeRF is trained on

Neither NeRF nor Gaussian splatting knows what an object is; they rebuild whatever the photos show. COLMAP still gets the full photos, because the room gives it far more to match. To cut out the globe, SAM 2, a video segmentation model, followed it through all 255 frames from one box and two clicks on the first.

Everything outside the mask is painted white, and NeRF composites its rays onto white. The cheapest way to produce white pixels is empty space, so the room never enters the model.

My capture, frame 1868 of the video. Masks: SAM 2.1 Hiera-L (Ravi et al. 2024) through Hugging Face transformers, prompted with a box and two points in the first frame, then propagated. White background as in the paper's synthetic scenes.

Training NeRF on my laptop



The paper's model and settings: 4,096 rays per step, 64 + 128 samples per ray, 100,000 steps. On my laptop's RTX 5070 Ti this took 9.0 hours; the paper needed one to two days on a V100 for 100,000 to 300,000 steps.

2,500	10,000	25,000	100,000

After 2,500 steps the shape and the brass are there, but the ball is still a blur: 22.8 dB on the held-out photo.

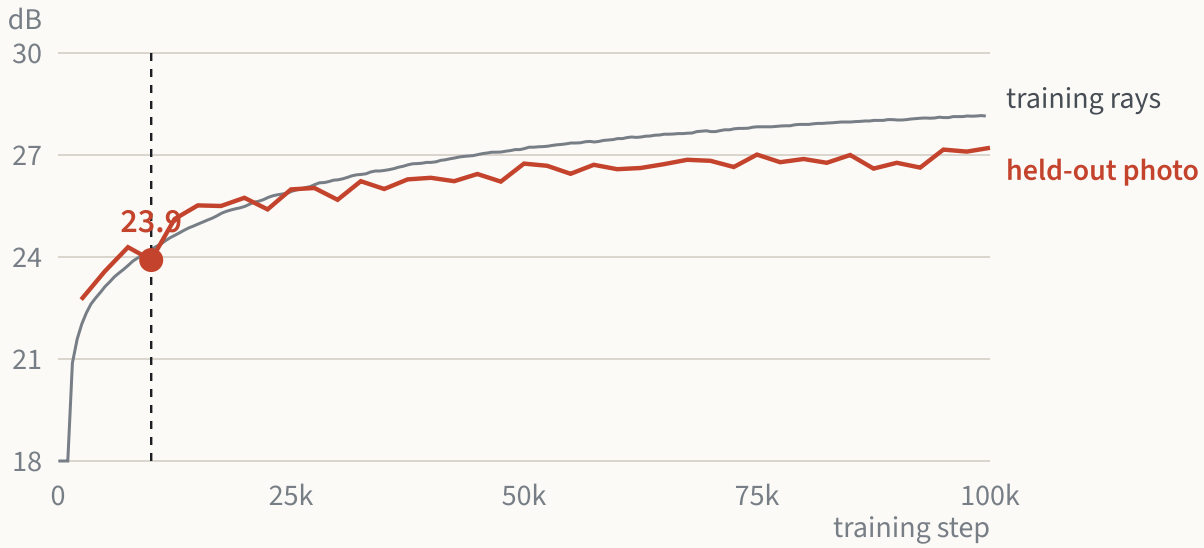
Held-out photo, never trained on

NeRF's render after 2,500 steps

Its depth, near is bright

My run with the nerf-globe notebooks: 223 training photos and 32 held out (every 8th, as in the paper), 1080 × 607 pixels. Test PSNR on one held-out photo at half resolution, every 2,500 steps.

Training NeRF on my laptop



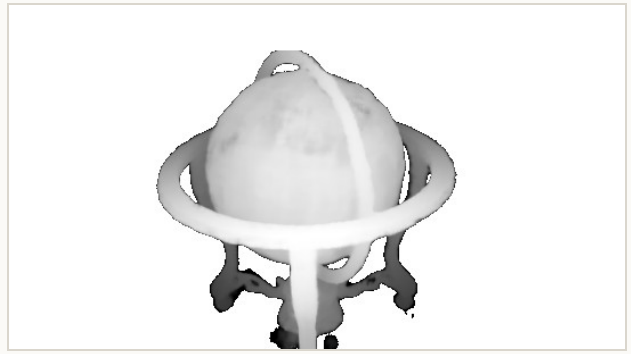
The paper's model and settings: 4,096 rays per step, 64 + 128 samples per ray, 100,000 steps. On my laptop's RTX 5070 Ti this took 9.0 hours; the paper needed one to two days on a V100 for 100,000 to 300,000 steps.

2,500	10,000	25,000	100,000

By 10,000 steps the continents appear, at 23.9 dB.

Held-out photo, never trained on

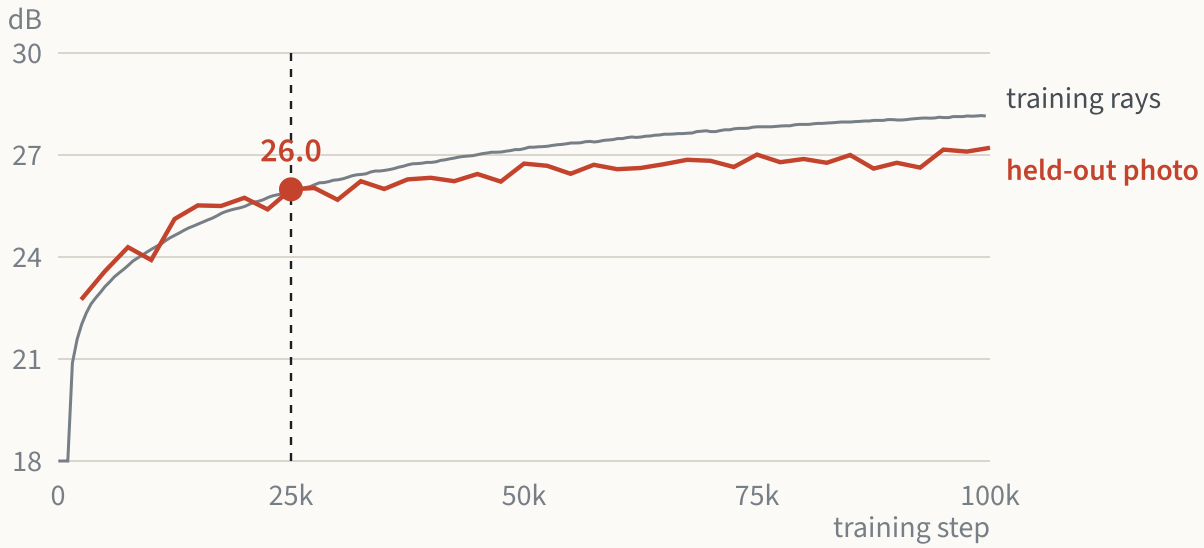
NeRF's render after 10,000 steps



Its depth, near is bright

My run with the nerf-globe notebooks: 223 training photos and 32 held out (every 8th, as in the paper), 1080 × 607 pixels. Test PSNR on one held-out photo at half resolution, every 2,500 steps.

Training NeRF on my laptop



The paper's model and settings: 4,096 rays per step, 64 + 128 samples per ray, 100,000 steps. On my laptop's RTX 5070 Ti this took 9.0 hours; the paper needed one to two days on a V100 for 100,000 to 300,000 steps.

2,500	10,000	**25,000**	100,000

At 25,000 steps most of the detail is there, at 26.0 dB; the remaining 75,000 steps add about one more.

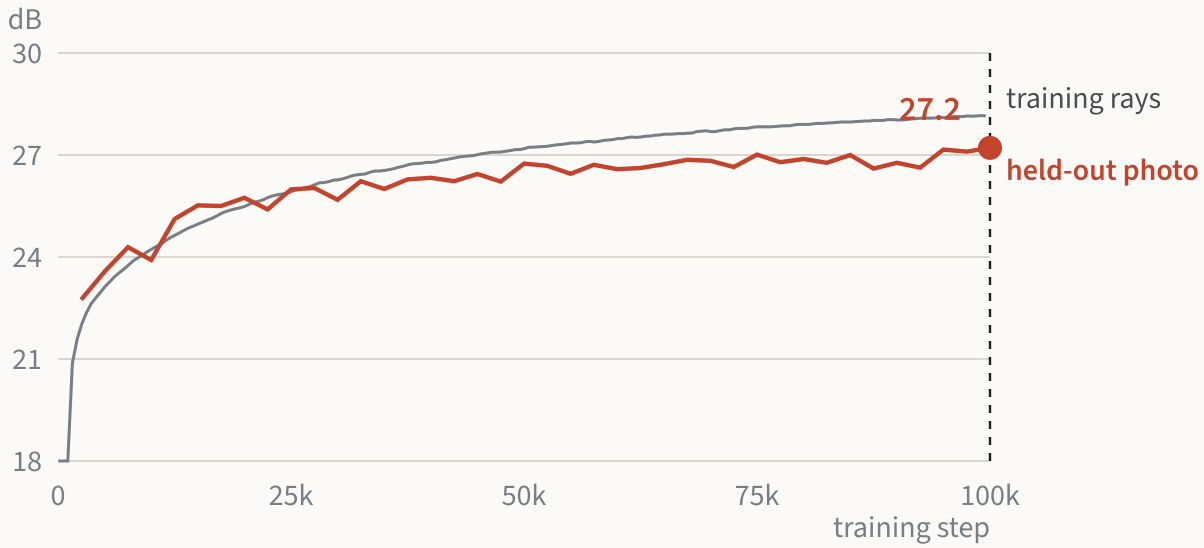
Held-out photo, never trained on

NeRF's render after 25,000 steps

Its depth, near is bright

My run with the nerf-globe notebooks: 223 training photos and 32 held out (every 8th, as in the paper), 1080 × 607 pixels. Test PSNR on one held-out photo at half resolution, every 2,500 steps.

Training NeRF on my laptop



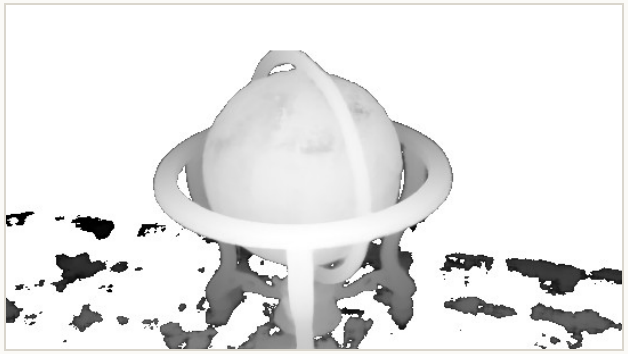
The paper's model and settings: 4,096 rays per step, 64 + 128 samples per ray, 100,000 steps. On my laptop's RTX 5070 Ti this took 9.0 hours; the paper needed one to two days on a V100 for 100,000 to 300,000 steps.

2,500	10,000	25,000	100,000

27.2 dB at the end. The dark specks around the stand in the depth map are floaters: faint fog at the height of the table.

Held-out photo, never trained on

NeRF's render after 100,000 steps



Its depth, near is bright

My run with the nerf-globe notebooks: 223 training photos and 32 held out (every 8th, as in the paper), 1080 × 607 pixels. Test PSNR on one held-out photo at half resolution, every 2,500 steps.

Views of my globe that nobody filmed

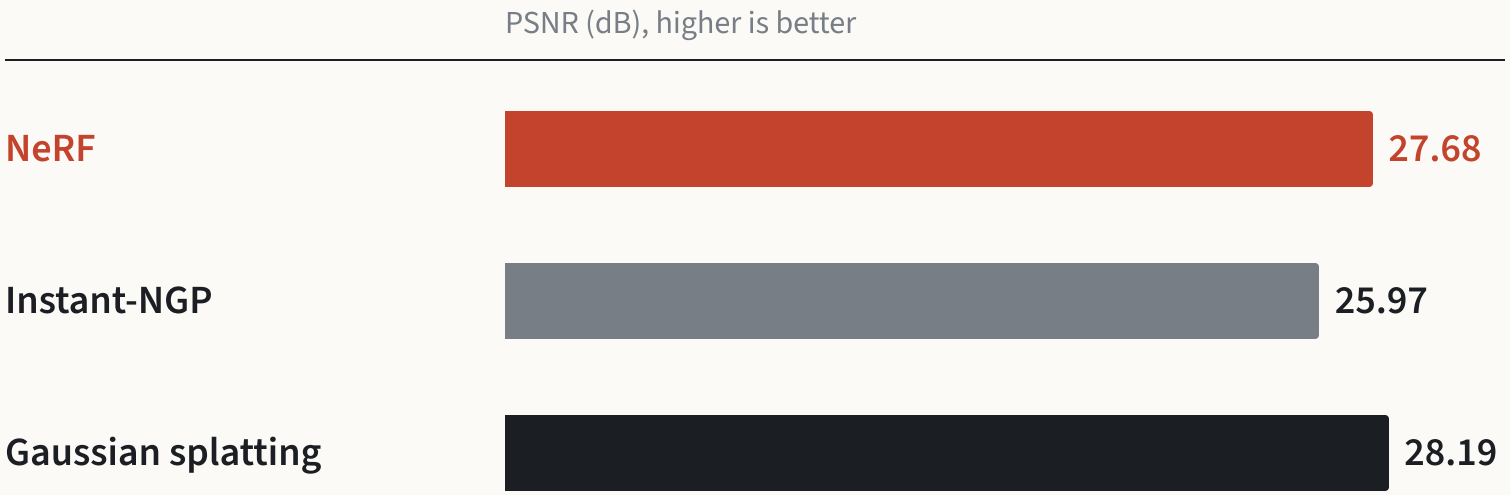
A circle of 120 new cameras, a little further back than mine, rendered by the trained NeRF. On the right is the depth from the same densities; NeRF was never given any depth.

Each frame took about 4 seconds at this half resolution, and 18 seconds at the full 1080 × 607. That is why this is a recording.

My NeRF after 100,000 steps, 120 viewpoints on a circle around the globe, 1.3 times the distance of my cameras, at 540 × 303 pixels per image. Depth is the weighted mean distance along each ray, drawn only where the opacity is above 0.5.

Image quality of the three methods on my photos

PSNR | SSIM | LPIPS



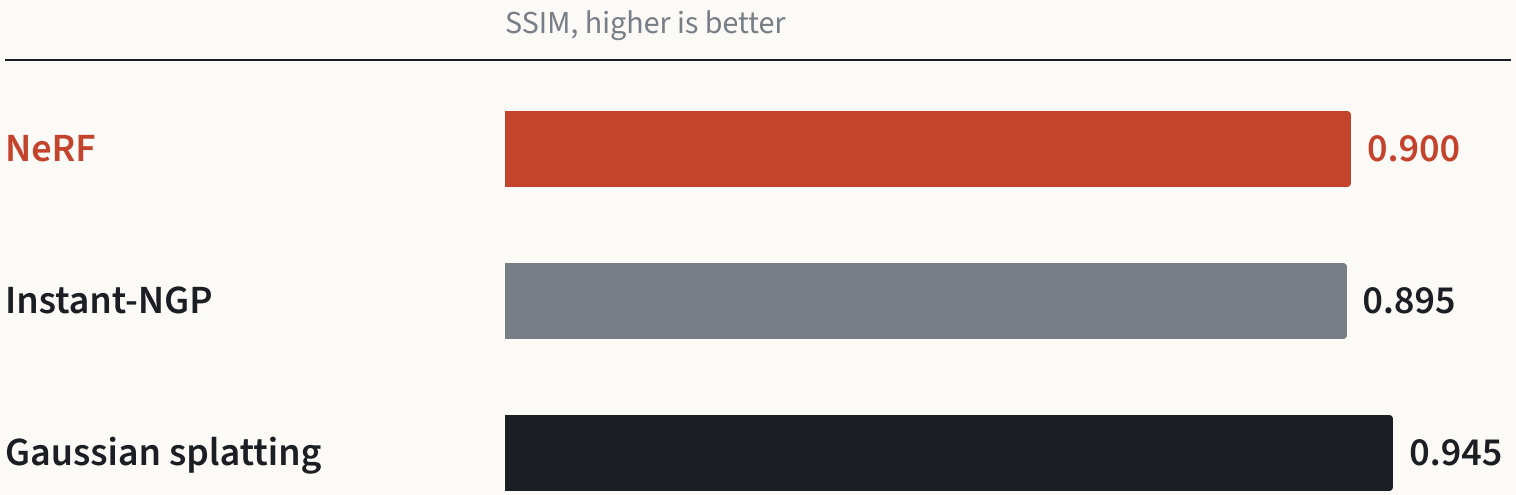
Gaussian splatting has the best average and is the best of the three on 24 of the 32 photos; NeRF on 8, Instant-NGP on 0.

The bars look alike, but the 2.2 dB between first and last is 40% less squared error.

My capture. All three trained on the same 223 photos, cameras and masks, and scored on the same 32 held-out photos after correcting the lens to a pinhole camera (NeRF on the original photos: 27.27 dB). NeRF: the paper's model, 100,000 steps. Instant-NGP: the paper's hash grid and networks, my implementation, 20,000 steps. Gaussian splatting: gsplat 1.5.3 (Ye et al. 2025), 30,000 steps.

Image quality of the three methods on my photos

PSNR | **SSIM** | LPIPS

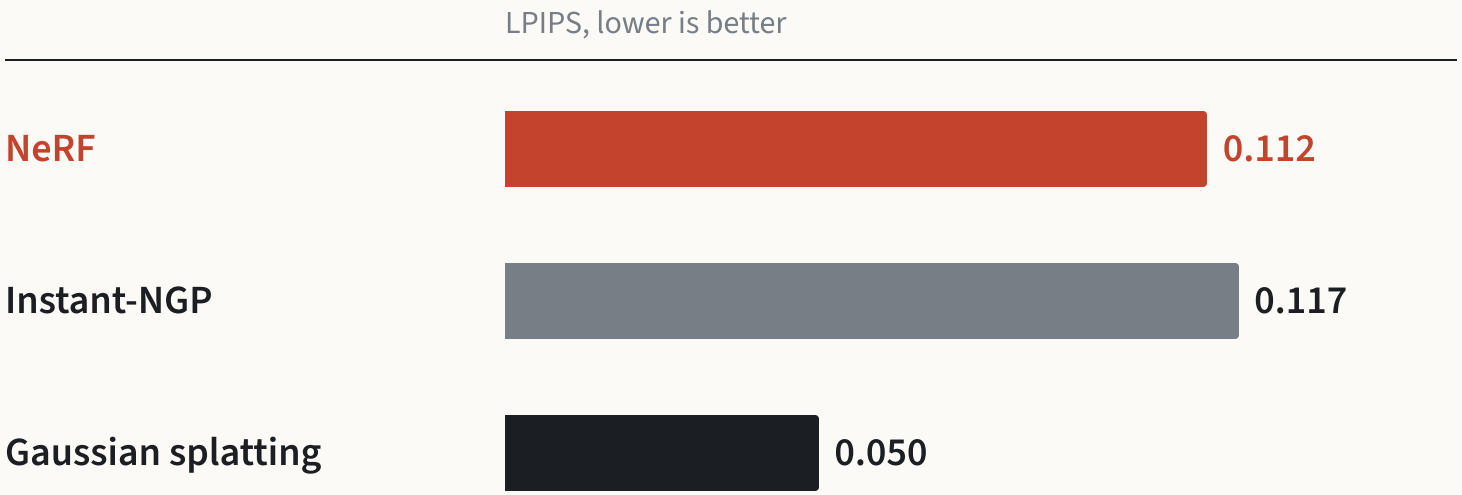


SSIM compares local structure such as edges and texture. Gaussian splatting keeps the most of it.

My capture. All three trained on the same 223 photos, cameras and masks, and scored on the same 32 held-out photos after correcting the lens to a pinhole camera (NeRF on the original photos: 27.27 dB). NeRF: the paper's model, 100,000 steps. Instant-NGP: the paper's hash grid and networks, my implementation, 20,000 steps. Gaussian splatting: gsplat 1.5.3 (Ye et al. 2025), 30,000 steps.

Image quality of the three methods on my photos

PSNR | SSIM | **LPIPS**



LPIPS punishes blur the most. Gaussian splatting has 2.3 times less of it than Instant-NGP.

My capture. All three trained on the same 223 photos, cameras and masks, and scored on the same 32 held-out photos after correcting the lens to a pinhole camera (NeRF on the original photos: 27.27 dB). NeRF: the paper's model, 100,000 steps. Instant-NGP: the paper's hash grid and networks, my implementation, 20,000 steps. Gaussian splatting: gsplat 1.5.3 (Ye et al. 2025), 30,000 steps.

Where each method looks better

Photo

NeRF, 28.3 dB

Instant-NGP, 27.4 dB

Gaussian splatting, 31.0 dB

Close up, Gaussian splatting keeps the grid lines and place names that NeRF blurs. Instant-NGP recovers more of them than NeRF, but its fine hash levels leave a grainy noise over the surface.

Held-out frames 808 and 3685 of my capture, never used in training; PSNR over the whole photo; both rows are enlarged close-ups. Limitation: Kerbl et al. 2023, Sec. 7.4.

Where each method looks better

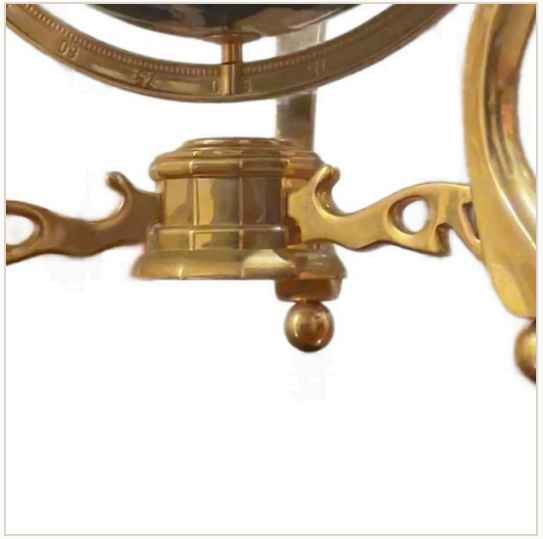
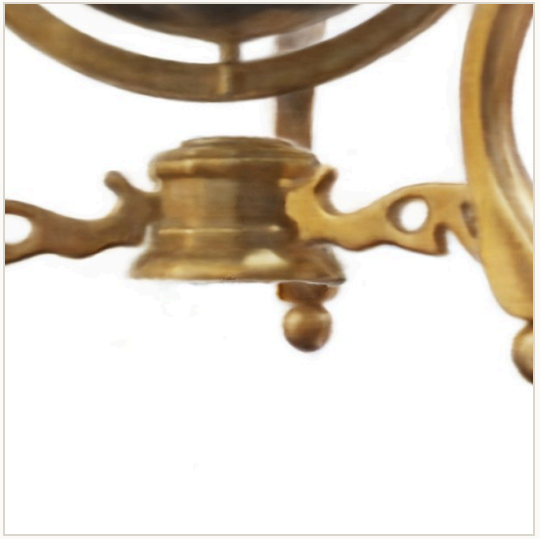
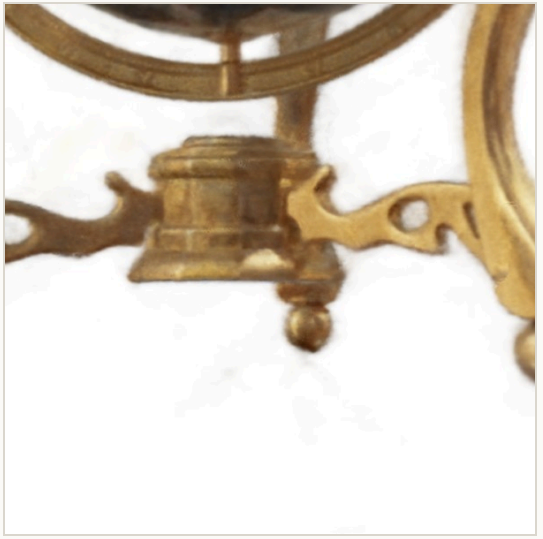


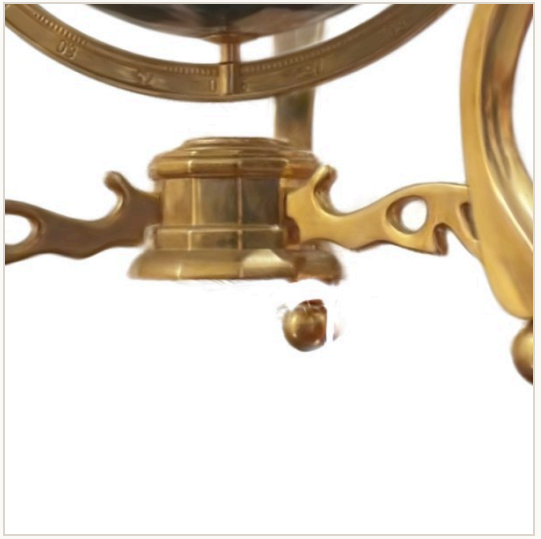
Photo from below



NeRF, 27.9 dB



Instant-NGP, 25.6 dB



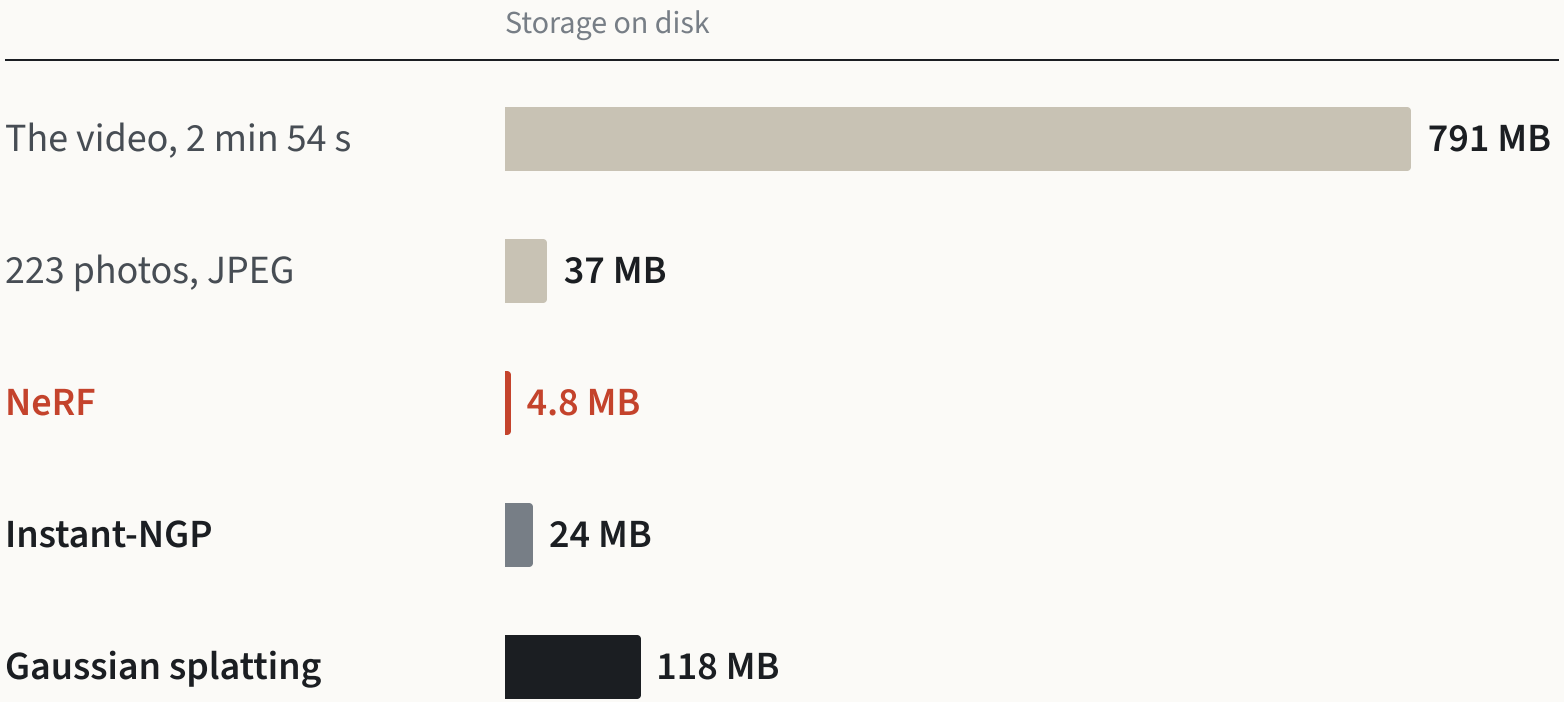
Gaussian splatting, 25.1 dB

From below, where few of my photos look, NeRF is best. Gaussian splatting breaks one brass foot into loose Gaussians, the limitation its paper names; Instant-NGP keeps the foot but leaves a faint haze around the stand.

Held-out frames 808 and 3685 of my capture, never used in training; PSNR over the whole photo; both rows are enlarged close-ups. Limitation: Kerbl et al. 2023, Sec. 7.4.

What each method costs on my globe

Storage | Training | One image



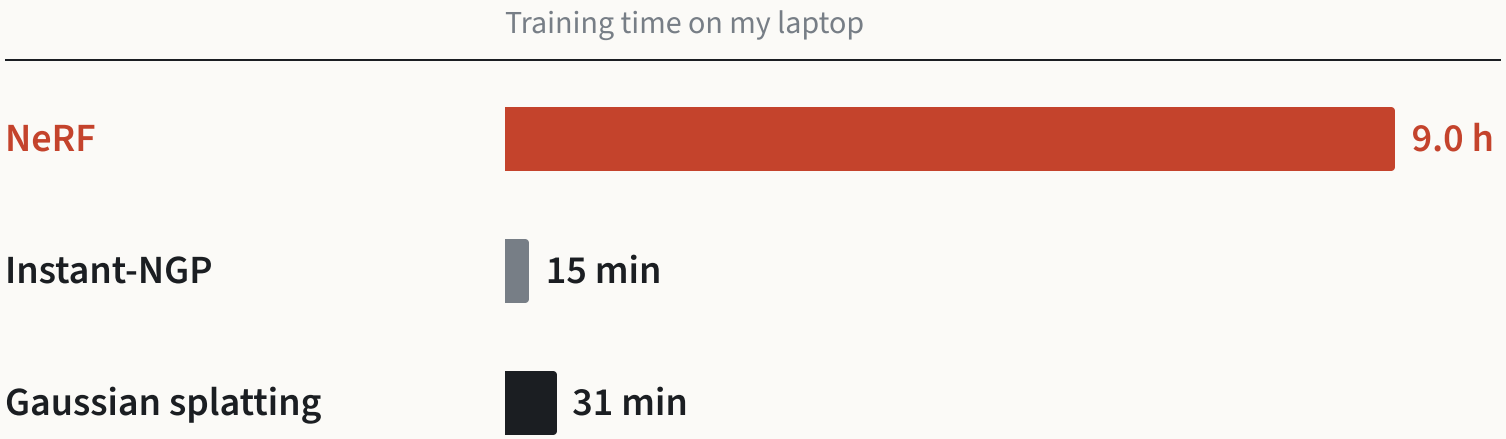
NeRF keeps the whole globe, from every direction, in 4.8 MB: 8 times less than the 223 photos it learned from.

Instant-NGP's hash tables take 24 MB, and Gaussian splatting's 500,482 Gaussians 118 MB.

My capture. Video: 174 s of HEVC at about 36 Mbit/s (its size from the bitrate). Photos: the 223 training frames as 1080 × 607 JPEG (439 MB uncompressed). Models as stored: NeRF 1,187,848 weights at 32 bits, Instant-NGP 12,207,469 numbers at 16 bits as in the paper, Gaussian splatting 29,528,438 numbers at 32 bits (.ply). Same laptop GPU (RTX 5070 Ti) for all times.

What each method costs on my globe

Storage | **Training** | One image



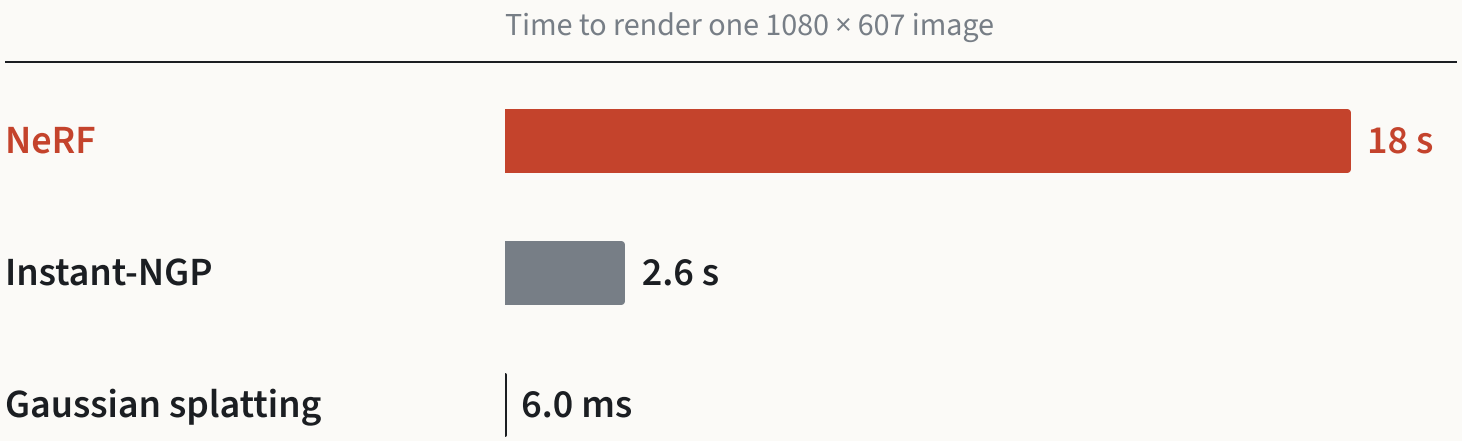
9.0 hours for NeRF, 15 minutes for Instant-NGP and 31 minutes for Gaussian splatting, on the same GPU.

My Instant-NGP runs partly in PyTorch; with the paper's fully fused code it would take seconds to minutes.

My capture. Video: 174 s of HEVC at about 36 Mbit/s (its size from the bitrate). Photos: the 223 training frames as 1080 × 607 JPEG (439 MB uncompressed). Models as stored: NeRF 1,187,848 weights at 32 bits, Instant-NGP 12,207,469 numbers at 16 bits as in the paper, Gaussian splatting 29,528,438 numbers at 32 bits (.ply). Same laptop GPU (RTX 5070 Ti) for all times.

What each method costs on my globe

Storage | Training | **One image**



18 seconds for NeRF, 2.6 s for Instant-NGP and 6.0 milliseconds for Gaussian splatting: about 165 images per second, which is why only it can be shown live.

My capture. Video: 174 s of HEVC at about 36 Mbit/s (its size from the bitrate). Photos: the 223 training frames as 1080 × 607 JPEG (439 MB uncompressed). Models as stored: NeRF 1,187,848 weights at 32 bits, Instant-NGP 12,207,469 numbers at 16 bits as in the paper, Gaussian splatting 29,528,438 numbers at 32 bits (.ply). Same laptop GPU (RTX 5070 Ti) for all times.

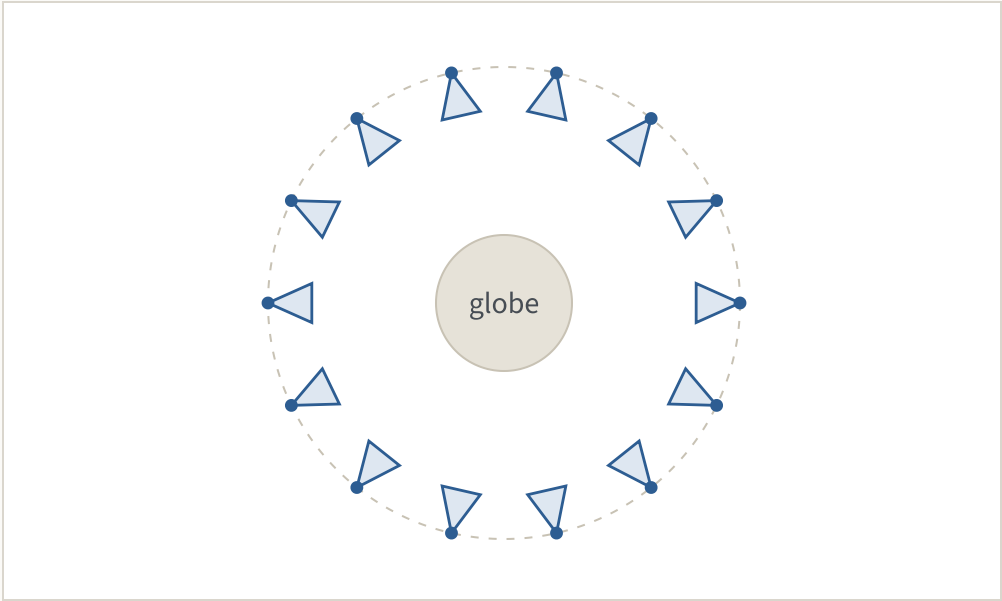
My globe as Gaussians, drawn in real time

The Gaussians from my training, drawn by this page on the laptop's GPU. Every frame they are sorted by depth and blended front to back with the same sum NeRF uses along a ray.

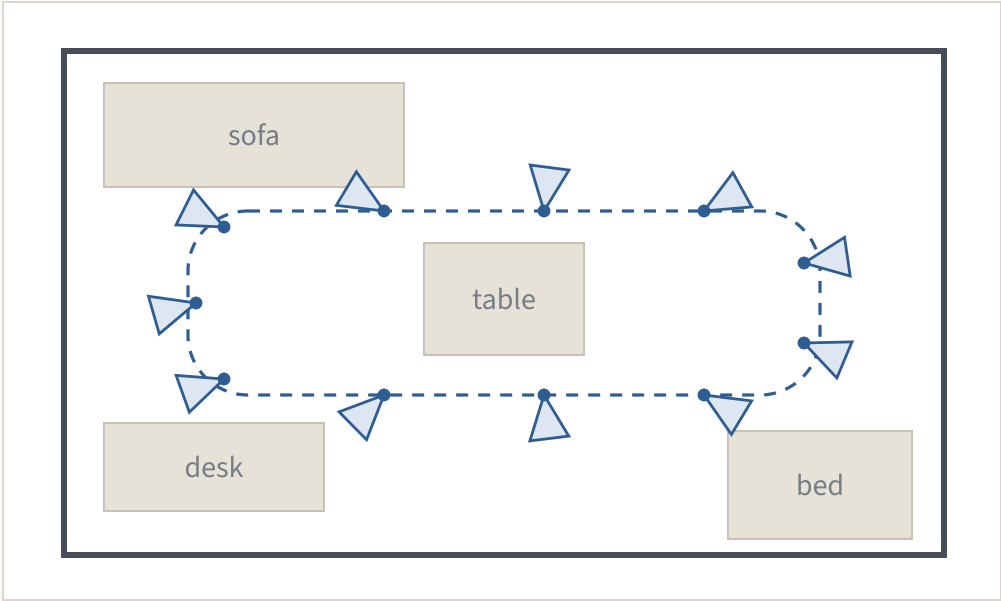
Shown live in the talk; here one frame of it. A frame took about 4 ms to draw on my laptop, while NeRF needs 18 seconds for one image.

My capture. Gaussians trained with gsplat 1.5.3, drawn by a WebGL2 renderer written for this talk, with view-dependent colour up to degree 3; its images match gsplat's to 41–43 dB. Full window.

Capturing a whole room from inside



My globe: cameras around one object, looking in



A room: cameras walking a loop inside it, looking out

A room works, and needs no mask, because everything in it belongs to the scene. The original NeRF assumes the scene lies between a near and a far distance; Mip-NeRF 360 and Gaussian splatting handle scenes that surround the camera, and their test scenes include indoor rooms.

The camera has to walk, here a slow loop at two heights: turning on one spot gives no depth, because nothing shifts against the background. Mirrors, windows and plain white walls stay hard for COLMAP and for both methods.

Barron et al. 2022 (Mip-NeRF 360: unbounded scenes; its dataset includes the indoor scenes room, kitchen, counter and bonsai); Kerbl et al. 2023, Table 1 and Fig. 5 (Dr Johnson, an indoor scene from Hedman et al. 2018).

Summary

5 MB	per scene: the scene is stored as two small networks that give density and colour for any point and direction.
+5.0 dB	over the best earlier method on the realistic synthetic scenes, 31.0 against 26.1 dB, and the highest PSNR on all three test sets.
15 s	of Instant-NGP training pass NeRF's quality on its own synthetic scenes (31.4 against 31.0 dB), with a learned hash grid and a tiny network.
134 fps	for 3D Gaussian splatting on large real scenes, close to Mip-NeRF 360's quality (27.2 against 27.7 dB), at 734 MB per scene.
6 ms	per image for Gaussian splatting on my own globe, against 18 s for NeRF, and 28.2 against 27.7 dB on photos neither had seen.

Mildenhall et al. 2020, Table 1 (Realistic Synthetic 360°: NeRF 31.01 dB, Neural Volumes 26.05 dB) and Sec. 6.3; Müller et al. 2022, Table 2; Kerbl et al. 2023, Table 1 (Mip-NeRF 360 dataset). Last row: my capture, 32 held-out photos, both methods on the same laptop GPU.

References

The three papers

Mildenhall, Srinivasan, Tancik, Barron, Ramamoorthi & Ng 2020. *NeRF: representing scenes as neural radiance fields for view synthesis.* ECCV. arXiv:2003.08934.

Müller, Evans, Schied & Keller 2022. *Instant neural graphics primitives with a multiresolution hash encoding.* SIGGRAPH. arXiv:2201.05989.

Kerbl, Kopanas, Leimkühler & Drettakis 2023. *3D Gaussian splatting for real-time radiance field rendering.* SIGGRAPH. arXiv:2308.04079.

Methods compared in the three papers

Sitzmann, Zollhöfer & Wetzstein 2019. *Scene representation networks: continuous 3D-structure-aware neural scene representations* (SRN). NeurIPS.

Lombardi et al. 2019. *Neural volumes: learning dynamic renderable volumes from images.* SIGGRAPH.

Mildenhall et al. 2019. *Local light field fusion: practical view synthesis with prescriptive sampling guidelines* (LLFF). SIGGRAPH.

Sitzmann et al. 2019. *DeepVoxels: learning persistent 3D feature embeddings.* CVPR.

Liu et al. 2020. *Neural sparse voxel fields* (NSVF). NeurIPS.

Fridovich-Keil, Yu et al. 2022. *Plenoxels: radiance fields without neural networks.* CVPR.

Barron et al. 2021. *Mip-NeRF: a multiscale representation for anti-aliasing neural radiance fields.* ICCV.

Barron et al. 2022. *Mip-NeRF 360: unbounded anti-aliased neural radiance fields.* CVPR.

Methods and tools used

Levoy 1990. *Efficient ray tracing of volume data.* ACM Transactions on Graphics.

Park et al. 2019. *DeepSDF: learning continuous signed distance functions for shape representation.* CVPR.

Rahaman et al. 2019. *On the spectral bias of neural networks.* ICML.

Schönberger & Frahm 2016. *Structure-from-motion revisited* (COLMAP). CVPR.

Ravi et al. 2024. *SAM 2: segment anything in images and videos.* arXiv:2408.00714.

Ye et al. 2025. *gsplat: an open-source library for Gaussian splatting.* JMLR.

Datasets and metrics

Hedman et al. 2018. *Deep blending for free-viewpoint image-based rendering.* SIGGRAPH Asia.

Wang, Bovik, Sheikh & Simoncelli 2004. *Image quality assessment: from error visibility to structural similarity* (SSIM). IEEE TIP.

Zhang, Isola, Efros, Shechtman & Wang 2018. *The unreasonable effectiveness of deep features as a perceptual metric* (LPIPS). CVPR.

Other follow-up work

Yu, Ye, Tancik & Kanazawa 2021. *pixelNeRF: neural radiance fields from one or few images.* CVPR.

Martin-Brualla et al. 2021. *NeRF in the wild: neural radiance fields for unconstrained photo collections.* CVPR.

Zha, Zhang & Li 2022. *NAF: neural attenuation fields for sparse-view CBCT reconstruction.* MICCAI.